

VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ BAKALAURO STUDIJŲ PROGRAMA

Kodo kokybės vertinimo, taikant didelius kalbos modelius, tyrimas

Reserach of Code Quality Assessment Using Large Language Models

Bakalauro baigiamasis darbas

Atliko:	Tadas Baltrūnas
Darbo vadovas:	Doc. dr. Asta Slotkienė
Darbo recenzentas:	Lekt. Irus Grinis

Vilnius – 2025

Santrauka

Sparčiai vystantis programinės įrangos kūrimo industrijai, kodo kokybė tapo esminiu aspektu, užtikrinančiu programų efektyvumą ir patikimumą. Tradiciniai kodo kokybės vertinimo metodai, tokie kaip statinė analizė ir kodo peržiūros, yra plačiai taikomi, tačiau jų veiksmingumą riboja žmogiškosios klaidos, laiko sąnaudos ir ribotas statinių analizės įrankių gebėjimas interpretuoti kodo kontekstą. Dideli kalbos modeliai siūlo naujas galimybes automatizuoti šiuos procesus, tačiau jų pritaikymas dar nėra pakankamai ištirtas.

Šio darbo tikslas – įvertinti, kaip efektyviai dideli kalbos modeliai gali vertinti programinio kodo kokybę, pagal statines metrikas ir ISO/IEC 25010:2023 standarto apibrėžiamas kodo palaikomumo subcharakteristikas. Literatūros analizė parodė, kad ankstesni tyrimai daugiausia analizavo kodo skaitomumą ar saugumą, o kiekybinis vertinimas pagal metrikas ir palyginimas su analizės įrankiais ar ekspertais – nebuvo atliktas.

Eksperimente įvertinti Qwen2.5-Coder, CodeGemma ir CodeLlama modeliai, taikant dešimt statinių metrikų ir keturis kodo palaikomumo kriterijus. Naudotas 310 C# metodų duomenų rinkinys. Rezultatai parodė, kad Qwen2.5-Coder buvo tiksliausias daugumoje metrikų, turėjo mažiausias paklaidas ir stipriausią koreliaciją su analizės įrankiais bei ekspertais. Tačiau kai kurios metrikos, ypač susijusios su kodo apimtimi, liko iššūkiu visiems modeliams. Tyrimo rezultatai rodo, kad nors dideli kalbos modeliai gali pateikti statistiškai artimus įverčius tradiciniams kodo kokybės vertinimo metodams, jų taikymas kodo kokybės vertinimui šiuo metu dar nėra pakankamai patikimas. Siekiant tikslesnių rezultatų, didelius kalbos modelius rekomenduojama naudoti kaip papildomą priemonę kartu su tradiciniais statinės analizės įrankiais.

Raktiniai žodžiai: dideli kalbos modeliai, kodo kokybės vertinimas, produkto kokybė

Summary

With the rapid development of the software development industry, code quality has become an essential aspect ensuring the efficiency and reliability of applications. Traditional methods of assessing code quality, such as static analysis and code reviews, are widely applied but are limited by human errors, time consumption, and the constrained ability of static analysis tools to interpret code context. Large language models offer new possibilities to automate these processes, but their application has not yet been sufficiently explored.

The aim of this work is to evaluate how effectively large language models can assess program code quality based on static metrics and maintainability sub-characteristics defined in the ISO/IEC 25010:2023 standard. The literature review showed that previous studies mainly analyzed code readability or security, while quantitative evaluation based on metrics and comparison with analysis tools or experts had not been conducted.

The experiment evaluated the Qwen2.5-Coder, CodeGemma, and CodeLlama models using ten static metrics and four maintainability criteria. A dataset of 310 C# methods was used. The results showed that Qwen2.5-Coder was the most accurate across most metrics, had the lowest errors, and the strongest correlation with analysis tools and experts. However, some metrics, especially those related to code volume, remained challenging for all models. The findings indicate that although large language models can produce statistically close estimates to traditional code quality assessment methods, their application for code quality evaluation is not yet sufficiently reliable. For more accurate results, large language models are recommended to be used as a supplementary tool alongside traditional static analysis tools.

Keywords: large language models, code quality evaluation, product quality

Turinys

SĄVOKŲ APIBRĖŽIMAI	6
ĮVADAS	8
1. DIDELI KALBOS MODELIAI IR JŲ YPATUMAI	10
1.1. Dideli kalbos modeliai.....	10
1.1.1. Didelio kalbos modelio samprata.....	10
1.1.2. Didelių kalbos modelių raida	10
1.1.3. Didelių kalbos modelių klasifikacija.....	10
1.2. Didelių kalbos modelių apmokymo duomenys.....	12
1.2.1. Duomenų rinkimas ir apdorojimas	12
1.2.1.1. Treniravimo duomenų šaltiniai	12
1.2.1.2. Duomenų filtravimas ir kokybės užtikrinimas.....	13
1.3. Didelių kalbos modelių užklausų tipų apžvalga	13
1.3.1. Užklausos be pavyzdžių	14
1.3.2. Užklausos su keliais pavyzdžiais	14
1.3.3. Minčių grandinės metodas.....	14
1.3.4. Minčių medžio metodas	15
1.4. Didelių kalbos modelių analizė	15
1.4.1. GPT-4 modelis.....	15
1.4.2. Codex modelis	16
1.4.3. CodeT5 modelis	16
1.4.4. StarCoder modelis	17
1.4.5. CodeLlama modelis.....	17
1.4.6. CodeGemma modelis	18
1.4.7. Qwen2.5-Coder modelis.....	18
1.4.8. Modelių palyginimas.....	18
1.5. Skyriaus apibendrinimas ir išvados	20
2. KODO KOKYBĖS VERTINIMO METODAI	21
2.1. Kodo kokybės vertinimo metodai ir jų efektyvumas	21
2.1.1. Rankinės kodo peržiūros.....	21
2.1.2. Automatizuota kodo kokybės analizė	21
2.2. Kiekybiniai kodo vertinimo aspektai.....	21
2.2.1. Statinės ir dinaminės kodo kokybės metrikos.....	21
2.2.2. Statinių kodo kokybės metrikų klasifikavimas.....	22
2.3. Statinių kodo kokybės metrikų analizė	23
2.3.1. Sudėtingumo metrikos.....	23
2.3.1.1. Ciklomatinis sudėtingumas	24
2.3.1.2. Kognityvinis sudėtingumas.....	25
2.3.2. Apimties metrikos.....	26
2.3.3. Objektiškai orientuotos metrikos.....	26
2.4. Subjektyvūs kodo kokybės kriterijai	28
2.5. Metrikų pasirinkimo kriterijai kodo kokybei vertinti su dideliais kalbos modeliais ..	29
2.6. Skyriaus apibendrinimas ir išvados	30
3. KITŲ AUTORIŲ TYRIMŲ ANALIZĖ	31
3.1. Literatūros šaltinių pasirinkimo aprašymas	31
3.2. Literatūros šaltinių analizė ir rezultatai	32

4. EKSPERIMENTINIS TYRIMAS IR REZULTATAI	35
4.1. Duomenų rinkinio sudarymas	35
4.1.1. Duomenų paieška ir atrankos kriterijai	35
4.1.2. Duomenų filtravimas ir paruošimas analizei	36
4.2. Techninės eksperimento aplinkos ir įrankių pasiruošimas	36
4.2.1. SciTools Understand konfigūracija	36
4.2.2. SonarQube konfigūracija	37
4.2.3. Užklauso sudarymas	37
4.2.4. Didelių kalbos modelių paruošimas	38
4.2.5. Ekspertų profilis ir įtraukimas	39
4.3. Statinių kodo kokybės metrikų vertinimo rezultatai ir analizė	40
4.3.1. Vertinimo rezultatų struktūra ir apdorojimas	40
4.3.2. Duomenų paruošimas statistinei analizei	41
4.3.3. Rezultatų analizė ir palyginimas	41
4.3.3.1. Modelių vertinimo tikslumo analizė	41
4.3.3.2. Statistinių paklaidų ir koreliacijos analizė	44
4.3.3.3. Vertinimų anomalijos ir pastebėjimai	45
4.4. Subjektyvių kodo kokybės metrikų vertinimo rezultatai ir analizė	47
4.4.1. Vertinimo rezultatų struktūra ir apdorojimas	47
4.4.2. Rezultatų analizė ir palyginimas	47
4.4.2.1. Statistinių paklaidų analizė	47
4.4.2.2. Koreliacijų analizė	50
4.5. Eksperimento rezultatai ir apibendrinimas	52
REZULTATAI IR IŠVADOS	53
ŠALTINIAI	55

Sąvokų apibrėžimai

Atraminių vektorių klasifikatorius (angl. Support Vector Machines). Mašininio mokymosi algoritmas skirtas klasifikuoti duomenims

Audiovizualiniai duomenys (angl. Audiovisual Data). Duomenų tipas, apimantis garso ir vaizdo informaciją

Automatinio kodavimo modeliai (angl. Code Auto-completion Models). Modeliai pritaikyti natūralios kalbos supratimo užduotims analizuojantys visą sekos kontekstą vienu metu

Autoregresiniai modeliai (angl. Autoregressive Models). Modeliai, prognozuojantys sekos elementus remiantis ankstesniais elementais

Dėmesio mechanizmas (angl. Attention Mechanism). Giliojo mokymosi metodas, leidžiantis modeliams sutelkti dėmesį į svarbiausias duomenų dalis

Dinaminės kodo metrikos (angl. Dynamic Code Metrics). Metrikos, apskaičiuojamos vykdamas kodą

Dideli kalbos modeliai (angl. Large Language Models, DKM). Dirbtinio intelekto tipas, išmokytas apdoroti ir generuoti tekstą natūralia kalba, naudojant didelius duomenų rinkinius ir giliojo mokymosi metodus

Dvipusio mokymosi metodas (angl. Bidirectional Learning Method). Dirbtinio intelekto mokymo metodas, leidžiantis modeliams analizuoti duomenis iš abiejų kryptų – nuo kairės į dešinę ir nuo dešinės į kairę

Euristinės technikos (angl. Heuristic Techniques). Metodai, naudojami didelių kalbos modelių duomenims filtruoti, remiantis apibrėžtomis taisyklėmis, siekiant greitai pašalinti nekokybiškus ar nereikalingus duomenis

Gilieji mokymosi metodai (angl. Deep Learning Methods). Dirbtinio intelekto metodai, pagrįsti dirbtiniais neuroniniais tinklais, naudojami mokyti didelius kalbos modelius

Integruota kūrimo aplinka (angl. Integrated Development Environment, IDE). Programos, turinčios daug priemonių, palengvinančių ir pagreitinančių programinės įrangos kūrimo procesą

Kodo metrika (angl. Code Metric). Kiekybinis rodiklis, naudojamas konkrečiam kodo aspektui vertinti

Kodo peržiūros (angl. Code Reviews). Procesas, kurio metu tikrinama kodo kokybė rankiniu būdu

Maksimalios entropijos modeliai (angl. Maximum Entropy Models). Statistiniai modeliai, naudojami klasifikavimo užduotyse, užtikrinantys kuo vienodesnę tikimybių paskirstymą

Maskavimo metodas (angl. Masking Method). Metodas, naudojamas modelio treniravimo metu, kai dalis įvesties duomenų yra paslepiama, siekiant išmokyti modelį atkurti trūkstamą informaciją ir geriau suprasti kontekstą

Minčių grandinės metodas (angl. Chain-of-Thought Method, CoT). Užklausų formavimo metodas, pagrįstas užduočių skaidymu į tarpinius žingsnius, primenančius žmogaus loginę mąstymo seką

Minčių medžio metodas (angl. Tree-of-Thought Method, ToT). Užklausų formavimo

metodas, struktūruojantis užduotis kaip medžio formos duomenų struktūrą, leidžiantį įvertinti įvairias alternatyvas

Multimodaliniai duomenys (angl. Multimodal Data). Duomenys, vienu metu apimantys kelių skirtingų tipų informaciją, pvz., tekstą, vaizdus, garsą ar kitus formatus

Multimodaliniai modeliai (angl. Multimodal Models). Modeliai, gebantys apdoroti skirtingų tipų duomenis pavyzdžiui, tekstą ir vaizdus

N-gramos (angl. N-grams). Fiksuoto ilgio žodžių ar simbolių sekos, naudojamos natūralios kalbos apdorojimo užduotyse analizuoti ir prognozuoti žodžių ar frazių pasikartojimus bei tikimybes

Natūralios kalbos tekstas (angl. Natural Language Text). Tekstinė informacija, parašyta žmonių vartojama kalba

Parametrai (angl. Parameters). Modelio mokymo metu išmoktos reikšmės, tiesiogiai lemiančios modelio užduočių atlikimo tikslumą

Paslėptieji Markovo modeliai (angl. Hidden Markov Models). Statistiniai modeliai, naudojami sekoms prognozuoti remiantis tikimybinėmis prielaidomis

Sentimentų analizės mechanizmas (angl. Sentiment Analysis Mechanism). Algoritmai, analizuojantys duomenis ir nustatantys jų emocinį turinį

Sekos į seką modeliai (angl. Sequence-to-Sequence Models). Giliojo mokymosi modeliai, naudojami sekos transformacijai, pavyzdžiui, kalbos vertimui ar kodo generavimui

Statinės kodo metrikos (angl. Static Code Metrics). Metrikos, apskaičiuojamos nevykdant kodo

Statistiniai kalbos modeliai (angl. Statistical Language Models). Kalbos modeliai, naudojančys statistinius metodus žodžių sekų prognozavimui

Šlamšto filtravimo mechanizmas (angl. Spam Filtering Mechanism). Algoritmai, naudojami automatiškai aptikti ir filtruoti nepageidaujamą informaciją

Transformatorių architektūra (angl. Transformer Architecture). Giliojo mokymosi architektūra, paremta dėmesio mechanizmais, naudojama daugelyje didelių kalbos modelių

Treniravimo duomenys (angl. Training Data). Duomenų rinkiniai, naudojami apmokant dirbtinius intelekto modelius

Užklausa (angl. Prompt). Tekstinė ar struktūrizuota įvestis, pateikiama dirbtinio intelekto modeliui, siekiant inicijuoti norimą atsakymą ar veiksmą. Užklausa gali būti klausimas, užduotis ar komandų seka, nurodanti modeliui, kaip jis turėtų interpretuoti duomenis ir generuoti rezultatą

Užklauskos be pavyzdžių (angl. Zero-shot Prompts). Užklauskų formavimo metodas, pagrįstas užduočių pateikimu be pavyzdžių, remiantis tik modelio turimomis žiniomis

Užklauskos su keliais pavyzdžiais (angl. Few-shot Prompts). Užklauskų formavimo metodas, pagrįstas užduočių pateikimu su keliais pavyzdžiais, siekiant pagerinti modelio supratimą

Užklauskų inžinerija (angl. Prompt Engineering). Procesas, kurio metu kuriamos ir optimizuojamos užklauskos, siekiant pagerinti dirbtinio intelekto modelių atsakymų kokybę

Įvadas

Tyrimo objektas ir aktualumas

Pastaraisiais metais, sparčiai vystantis ir plečiantis programinės įrangos kūrimo industrijai, vis didesnis dėmesys skiriamas tinkamam turimų išteklių panaudojimui bei kuriamų produktų kokybės ir patikimumo užtikrinimui. Programinio kodo kokybė yra vienas esminių veiksnų, lemiančių programinės įrangos palaikomumą, saugumą ir ilgalaikį patikimumą [EOA24]. Aukštos kokybės kodas ne tik padeda išvengti klaidų, bet ir palengvina sistemų priežiūrą, modifikavimą bei integraciją.

Programinio kodo kokybė yra vertinama pagal tiksliai apibrėžtas kodo metrikas, leidžiančias objektyviai analizuoti tokias kodo savybes kaip skaitomumas, sudėtingumas ir suprantamumas [NPM⁺17]. Šiuolaikiniai kodo kokybės užtikrinimo metodai, tokie kaip statinė ir dinaminė analizė, kodo peržiūros bei automatizuotų įrankių taikymas, padeda identifikuoti klaidas ir įvertinti kodą pagal įvairius kriterijus, tačiau jų taikymas yra ribotas dėl žmogiškųjų klaidų, laiko sąnaudų ir išsamumo stokos [RSW⁺24].

Šiuo metu vis labiau besivystančios didelių kalbos modelių (DKM) technologijos atveria naujas galimybes automatizuojant programinės įrangos kūrimo ir analizės procesų etapus, įskaitant kodo kokybės vertinimą [SV24], [JSS25], [TB25]. Naujausi DKM geba generuoti kodą, aptikti klaidas ir vertinti kodo struktūrą. Nepaisant to, jų gebėjimas vertinti kodo kokybę metodo lygmeniu, remiantis kodo kokybės metrikomis, vis dar nėra pakankamai ištirtas.

Norint nustatyti DKM tinkamumą kodo kokybės analizei, yra poreikis atlikti praktinį tyrimą, kuriame būtų įvertinta, kurie modeliai yra efektyviausi pagal konkrečius kodo kokybės kriterijus, bei kaip jų rezultatai skiriasi nuo tradicinių analizės įrankių ir ekspertų įžvalgų.

Darbo tikslas

Ištirti didelių kalbos modelių taikymo galimybes vertinant C# programavimo kalbos kodo kokybę, remiantis kodo kokybės kriterijais, įtakojančiais programų sistemos produkto kokybę.

Keliami uždaviniai ir laukiami rezultatai

Uždaviniai

Tikslui pasiekti keliami šie uždaviniai:

1. Išnagrinėti didelius kalbos modelius ir kodo kokybės vertinimo metodus, metrikas.
2. Išanalizuoti kitų autorių tyrimus kodo kokybei vertinti naudojant didelius kalbos modelius.
3. Sudaryti C# programinio kodo duomenų rinkinį.
4. Atlikti eksperimentinį tyrimą, siekiant ištirti didelių kalbos modelių efektyvumą vertinant kodo kokybę ir palyginti jų rezultatus su tradiciniais kodo analizės įrankiais.
5. Parengti išvadas ir rekomendacijas didelių kalbos modelių taikymui vertinant kodo kokybę.

Laukiami rezultatai

1. Išnagrinėta mokslinė literatūra analizuojanti didelius kalbos modelius ir atliktas septynių modelių palyginimas.
2. Atlikta kodo kokybės vertinimo metodų ir metrikų analizė.
3. Atlikta literatūros analizė, apimanti mokslinius tyrimus, kuriuose nagrinėtas didelių kalbos modelių taikymas kodo kokybės vertinimui.
4. Sudarytas C# programinio kodo duomenų rinkinys, kodo kokybės vertinimui taikant didelius kalbos modelius.
5. Ištirtas didelių kalbos modelių kodo kokybės vertinimo efektyvumas pagal pasirinktas kodo kokybės metrikas ir gauti rezultatai palyginti su tradiciniais kodo analizės įrankiais.
6. Parengtos išvados ir rekomendacijos kodo kokybės vertinimui taikant didelius kalbos modelius.

1. Dideli kalbos modeliai ir jų ypatumai

1.1. Dideli kalbos modeliai

1.1.1. Didelio kalbos modelio samprata

Dideli kalbos modeliai – tai dirbtinio intelekto tipas, sukurtas suprasti ir generuoti natūralios kalbos tekstą [WLX⁺23]. Šie modeliai dažnai naudoja giliaisiais mokymosi metodais paremtą transformatorių architektūrą tam, kad išmokytų ir perprastų sudėtingas žmogiškos kalbos tendencijas bei struktūras. Pagrindinis didelių kalbos modelių bruožas – gebėjimas apdoroti didžiulius duomenų kiekius ir fiksuoti semantinius ryšius tarp žodžių bei frazių. Šie modeliai taip pat sugeba apdoroti vaizdo, garso, audiovizualinius ir multimodalinius duomenis, bei sugeba išmokti semantinius ryšius tarp tokių duomenų [HTS⁺24].

1.1.2. Didelių kalbos modelių raida

Didelių kalbos modelių raida prasidėjo nuo statistinių kalbos modelių [WCD⁺24], tokių kaip n-gramos, paslėptųjų Markovo modelių ir maksimalios entropijos modelių atsiradimo. Šie įrankiai leido atlikti paprasto teksto generavimą, tačiau jų galimybės suprasti sudėtingesnes natūralios kalbos struktūras ir semantinius ryšius buvo ribotos [NNN24].

Pereidami prie duomenimis grįstų metodikų, mokslininkai pradėjo taikyti mašininio mokymosi algoritmus, tokius kaip atraminių vektorių klasifikatoriai [CGR⁺20]. Tai leido modeliams geriau pastebėti dėsningumus dideliuose tekstų rinkiniuose ir paspartino tokių mechanizmų, kaip šlamšto filtravimas [CKP⁺15] ir sentimentų analizė vystimasi [NR13].

Giliojo mokymosi atsiradimas buvo vienas iš kertinių atradimų prisidėjusių prie didelė kalbos modelių kūrimo. Pasikartojantys ir ilgalaikės-trumpalaikės atminties neuroniniai tinklai įgijo ypatingai didelę reikšmę. Šios giliojo mokymosi architektūros pagerino modelių gebėjimą suprasti kontekstą ir suteikė galimybę taikyti modelius tokioms užduotims, kaip mašininis vertimas ir kalbos atpažinimas [LLA22]. Tačiau giliojo mokymosi metodų naudojimas taip pat turėjo ir savų iššūkių. Iškilo nykstančių gradientų ir ilgalaikių priklausomybių problemos [Li17].

Reikšmingas postūmis įvyko 2017 metais, kuomet „Google“ kompanijos tyrėjai straipsnyje „Dėmesys yra viskas, ko jums reikia“ pristatė transformatorių architektūrą [VBS⁺17]. Transformatoriai, naudojančys dėmesio mechanizmus stipriai pagerino modelių efektyvumą, kadangi leido modeliams suprasti visą sakinio ar dokumento kontekstą. Transformatorių architektūra tapo pagrindu tokiems modeliams kaip OpenAI GPT ir Google BERT, kurie parodė itin gerus rezultatus [HTS⁺24].

1.1.3. Didelių kalbos modelių klasifikacija

Dideli kalbos modeliai yra kuriami spręsti specifines užduotis, kurios gali būti labai įvairios – dialogų formavimas, teksto klasifikacija, kalbų vertimas ar skirtingų duomenų tipų apdorojimas [HQS⁺23]. Modelio tipas parenkamas atsižvelgiant į tai, kokias užduotis jis turės spręsti, todėl svarbu įvertinti užduočių pobūdį, reikiamą funkcionalumą ir techninius apribojimus. Tokiu

būdu užtikrinama, kad modelis bus pritaikytas optimaliai veikti konkrečiame kontekste. Dideli kalbos modeliai yra kuriami remiantis keliomis pagrindinėmis architektūromis, kurios skiriasi savo veikimo principais ir pritaikymu [SBK⁺24].

Autoregresiniai modeliai, tokie kaip GPT-3 ir LLaMA 2 yra specializuoti atlikti generatyvio pobūdžio užduotis. Pagrindinis šių modelių veikimo principas yra sekančio sekos elemento nuspėjimas pagal prieš tai buvusias elementus. Dėl to šie modeliai geba itin gerai generuoti natūralios kalbos tekstą ir yra naudojami dialogų simuliacijai, teksto apibendrinimui ir kodo generavimui. Jie efektyviai apdoroja įvairaus ilgio duomenis ir gali spręsti užduotis net su minimaliu pavyzdžių kiekiu [XLH⁺24]. Tačiau jų pagrindinis trūkumas yra nesugebėjimas analizuoti viso pateikto konteksto vienu metu, todėl jie nėra tinkami užduotims, reikalaujančioms pilno teksto supratimo, kaip teksto klasifikacija, interpretacija ar multimodaliųjų duomenų apdorojimas [LJL⁺20].

Automatinio kodavimo modeliai, tokie kaip BERT yra pritaikyti užduotims, kurios reikalauja natūralios kalbos supratimo. Šie modeliai naudoja dvipusio mokymosi ir maskavimo metodus, leidžiančius analizuoti visą sekos kontekstą vienu metu. Dėl šios savybės jie yra ypač tinkami užduotims, susijusioms su teksto supratimu ar klasifikavimu [SBK⁺24]. Tačiau, skirtingai nei autoregresiniai modeliai, automatinio kodavimo modeliai nėra pritaikyti teksto generavimui. Be to, jie gali apdoroti tik fiksuoto ilgio įvesties sekas ir reikalauja išsamaus treniravimo, kad galėtų efektyviai atlikti tam tikras užduotis [GHW⁺23].

Sekos į seką modeliai apjungia autoregresinių ir automatinio kodavimo modelių privalumus, todėl geba ne tik generuoti natūralios kalbos tekstą, bet ir atlikti užduotis susijusias su konteksto supratimu. Pavyzdžiui, modeliai, tokie kaip T5 ar GLM, geba efektyviai apdoroti įvesties sekas ir sugeneruoti fiksuoto ilgio išvesties sekas, todėl yra labai naudingi atliekant tokias užduotis kaip teksto apibendrinimas ar kalbų vertimas [SJS⁺17]. Nepaisant šių pranašumų, sekos į seką modeliai turi ir trūkumų. Jie reikalauja daugiau resursų, nes žymiai padidėja modelio parametrų skaičius. Be to, skirtingai nei automatinio kodavimo ar autoregresiniai modeliai, sekos į seką modeliai dažniau kuriami konkrečioms užduotims, todėl jų pritaikymas gali būti mažiau universalus [SBK⁺24].

Multimodaliniai modeliai yra orientuoti apdoroti skirtingų tipų duomenis, tokius kaip paveikslėliai, tekstas, garso ir vaizdo įrašai. Vienas iš pažangiausių šios kategorijos modelių yra GPT-4, galintis priimti ir analizuoti tekstinę, garsinę bei vaizdinę medžiagą [WGC⁺23]. Multimodaliniai modeliai taip pat atveria plačias pritaikymo galimybes tokiose srityse kaip robotika [WWL⁺24], medicina [Mes23] ir vaizdo įrašų generavimas [KFS23]. Tačiau šie modeliai taip pat turi problemų, jų treniravimo procesas yra sudėtingesnis ir žymiai brangesnis nei vien tik teksto generavimu paremtų modelių [WGC⁺23]. Be to, dabartiniai multimodaliniai modeliai susiduria su problemomis apdorojant didelio masto duomenis ir vykdant sudėtingesnes užduotis. Jų sugeneruoti vaizdai neretai būna iškreipti, o garso įrašai – neperteikia tinkamų intonacijų bei skamba robotiškai [YFZ⁺24].

1.2. Didelių kalbos modelių apmokymo duomenys

1.2.1. Duomenų rinkimas ir apdorojimas

Didelių kalbos modelių efektyvumas ir generuojamų atsakymų kokybė stipriai priklauso nuo duomenų, kuriais modelis buvo treniruotas. Aukštos kokybės duomenys padeda modeliams geriau suprasti natūralios kalbos dėsningumus, gramatines taisykles ir lingvistines struktūras, taip užtikrinant tikslesnius ir rišlesnius atsakymus [NKQ⁺23]. Tuo tarpu žemos kokybės duomenys, kuriuose gausu gramatinių ar semantinių klaidų bei netikslios informacijos, gali ne tik neigiamai paveikti DKM atsakymų tikslumą, bet ir apsunkinti modelio treniravimo procesą. Modeliai treniruoti prastos kokybės duomenimis dažnai pasižymi tokiomis problemomis, kaip nenuoseklių ar klaidinančių atsakymų generavimu, šališkos informacijos pateikimu, nerišliu ar daug gramatinių klaidų turinčiu atsakymų tekstu [JWL⁺24].

1.2.1.1. Treniravimo duomenų šaltiniai

Informacijos pasirinkimas ir treniravimo duomenų rinkinių sudarymas yra itin svarbus etapas norint gerai apmokyti didelius kalbos modelius. Siekiant užtikrinti modelio gebėjimą suprasti įvairias kalbos struktūras bei kontekstus, naudojami platūs ir įvairūs duomenų rinkiniai, surinkti iš skirtingų šaltinių. Vienas iš dažniausiai naudojamų didelės apimties informacijos šaltinių yra „Common Crawl” – atvirojo tipo duomenų rinkinys, apimantis duomenis, surinktus iš daugiau nei 250 milijardų interneto puslapių nuo 2008 metų [Baa24]. Skaičiuojama, kad šio duomenų rinkinio dydis siekia apie 11 milijonų gigabaitų ir nuolat auga, nes kiekvieną mėnesį įtraukiama apie 3–5 milijardus naujų puslapių [AEX⁺24].

Dar vienas didelės apimties viešai prieinamas duomenų rinkinys, specifiskai orientuotas didelių kalbos modelių treniravimui, yra „The Pile”. Šis 2020 metų pabaigoje pavišintas duomenų rinkinys sudarytas iš 22 įvairių, aukštos kokybės informacijos šaltinių. Lyginant su „Common Crawl”, „The Pile” yra palyginti mažas, jo dydis siekia apie 825 gigabaitus. Tačiau šio rinkinio pagrindinis tikslas – apjungti kuo įvairesnę ir aukštos kokybės informaciją, nes pastebėta, kad modelių treniravimas naudojant įvairesnius duomenis reikšmingai pagerina modelių gebėjimą spręsti plataus spektro problemas [GBB⁺20], [LCL⁺24].

Be to, verta paminėti ir keletą mažiau populiarių, tačiau reikšmingų duomenų rinkinių, kurie atlieka svarbų vaidmenį didelių kalbos modelių treniravimo srityje. Vienas tokių pavyzdžių yra „C4 (Colossal Clean Crawled Corpus)” – duomenų rinkinys, sukurtas iš „Common Crawl” duomenų, kuriame pašalinti žemos kokybės tekstai ir pasikartojanti informacija, taip užtikrinant aukštesnę treniravimo duomenų kokybę [RSR⁺19]. Kitas reikšmingas pavyzdys yra RedPajama duomenų rinkinys, skirtas didesnio skaidrumo ir prieinamumo užtikrinimui treniravimo duomenyse. Jo pagrindinis tikslas – atkurti Meta LLaMA modelio treniravimo duomenų sudėtį ir pateikti šiuos duomenis atvirojo kodo formatu. Tai suteikia modelių kūrėjams galimybę geriau suprasti, kokie duomenys naudojami modelių treniravimui [WFA⁺24]. Taip pat dažnai pasitelkiami ir labiau specializuoti duomenų šaltiniai, pavyzdžiui, GitHub kodų saugyklos, kurios padeda treniruoti

modelius atlikti programavimo užduotis. Kiti svarbūs šaltiniai yra mokslinių publikacijų archyvai, kurie prisideda prie techninio pobūdžio informacijos apdorojimo gebėjimų tobulinimo [WZW⁺23].

1.2.1.2. Duomenų filtravimas ir kokybės užtikrinimas

Didelių kalbos modelių treniravimui yra reikalingi itin dideli kiekiai duomenų – naujausiems modeliams jų poreikis siekia trilijonus tekstinės informacijos vienetų. Vienas iš pavyzdžių yra Meta sukurtas LLaMA 3 modelis, išleistas 2024 metų balandį, kurio treniravimui prireikė maždaug 15 trilijonų tekstinės informacijos vienetų, surinktų iš įvairių viešai prieinamų šaltinių [GDJ⁺24]. Iš tokio didelio informacijos kiekio atrinkti tinkamiausius duomenis yra sudėtingas ir brangus procesas, todėl dažnai taikomi įvairūs filtravimo metodai, kurių pasirinkimas priklauso nuo konkrečių modelio techninių sprendimų [AEX⁺24].

Vienas dažniausiai taikomų filtravimo kriterijų yra kalbos, su kuriomis modelis dirbs. Kalbų filtravimas leidžia atskirti informaciją kuri yra pateikiama kalbomis aktualiomis treniruojamam modeliui. Tai ypač aktualu daugiakalbių modelių atveju, kai reikalinga ne tik atmesti nereikalingas kalbas, bet ir sekti kiekvienos kalbos duomenų kiekį. Programavimo kalbų atveju dažniausiai naudojami tokie praktiniai sprendimai kaip dokumentų filtravimas pagal failų plėtinius (pvz., „.py“ Python programavimo kalbai) [CND⁺22]. Šie metodai yra veiksmingi specifiniuose duomenų rinkiniuose, tokiuose kaip GitHub kodų saugyklos, tačiau natūralios kalbos tekstuose esantis kodas dažnai išlieka neaptiktas [AEX⁺24].

Kitas plačiai taikomas duomenų filtravimo metodas – euristinės technikos. Į DKM treniravimo duomenų rinkinius neretai patenka neapdoroti interneto duomenys, kurie gali apimti daug nereikalingos informacijos, pavyzdžiui, šabloninius tekstus ar klaidų pranešimus [NKQ⁺23]. Euristikų tikslas yra efektyviai pašalinti nepageidaujamą informaciją pagal tokius parametrus kaip teksto ilgis, pasikartojimų skaičius ar konkrečių elementų egzistavimas. Pavyzdžiui, euristinės funkcijos gali būti tokios: „Pašalink dokumentus, kuriuose yra mažiau nei penki žodžiai“, „Pašalink eilutes, kuriose žodžiai kartojasi keturis kartus iš eilės“, „Pašalink eilutes, prasidedančias žodžiais 'Prisijungti:'“ [KFS23].

Interneto duomenų rinkiniuose taip pat pasitaiko dubliuotų ar beveik identiškų duomenų [EBM⁺24]. Ruošiant duomenis DKM treniravimui, tikslinga yra pašalinti informaciją kuri kartojasi. Pasikartojančios informacijos pašalinimas gali sumažinti treniravimo laiką [SGS⁺22] ir pagerinti modelio tikslumą atliekant užduotis [LIN⁺22], [TSA⁺23].

Aptarti duomenų filtravimo metodai yra vieni iš dažniausiai taikomų DKM treniravimo metu. Tačiau egzistuoja ir kitos technikos, kurios pritaikomos priklausomai nuo konkrečių modelio poreikių ir treniravimo tikslų.

1.3. Didelių kalbos modelių užklausų tipų apžvalga

Pagrindinė sąsaja tarp vartotojo ir didelio kalbos modelio yra instrukcijos, pateikiamos natūralia kalba. Šios instrukcijos, nusakančios, kokią užduotį modelis turi atlikti, vadinamos užklausomis. Užklausa formuluotė tiesiogiai veikia modelio generuojamo atsakymo kokybę ir tikslumą. Jei užklausa pateikiama kartu su pavyzdžiais, modelis geba tiksliau suprasti užduoties

kontekstą ir pateikti kokybiškesnį atsakymą. Tuo tarpu abstrakčios užklauskos be pavyzdžių dažnai lemia mažiau tikslius rezultatus [SSS⁺24].

Užklauskų formavimas, dar vadinamas užklauskų inžinerija, yra svarbus didelių kalbos modelių panaudojimo aspektas. Užklauskų inžinerija leidžia pritaikyti DKM specifinių sričių užduotims nekeičiant techninių jų parametrų. Mokslinėje literatūroje yra priskaičiuojami 39 skirtingi užklauskų formavimo metodai [VD24].

1.3.1. Užklauskos be pavyzdžių

Užklauskos be pavyzdžių yra užklauskų formavimo metodas, kai DKM pateikiamas užduoties aprašymas be papildomų pavyzdžių, kaip turėtų atrodyti užduoties rezultatas. Patekiant tokio tipo užklauskas, gaunami atsakymai priklauso tik nuo modelio jau turimų žinių ir gebėjimo interpretuoti kontekstą. Šis metodas yra itin efektyvus nesudėtingų užduočių atlikimui, nes nereikalauja ilgų užklauskų ir papildomų vartotojo pastangų pateiant išsamius pavyzdžius [SSS⁺24].

Tačiau jis dažnai yra ribotas, kai atliekamos sudėtingesnės užduotys, nes modelis gali nepilnai suprasti specifinį užklauskos kontekstą. Be to, modelis gali susidurti su sunkumais, jei jo treniravimo duomenyse trūksta reikiamos informacijos, būdingos konkrečiai užduočiai [KGR⁺22].

1.3.2. Užklauskos su keliais pavyzdžiais

Užklauskos su keliais pavyzdžiais yra užklauskų formavimo metodas, kai DKM pateikiamas ne tik užduoties aprašymas, bet ir keli pavyzdžiai, aiškiai parodantys, kaip atlikti norimą užduotį. Šie pavyzdžiai leidžia modeliui geriau suprasti užduoties struktūrą ir kontekstą, o net ir nedidelis aukštos kokybės pavyzdžių skaičius gali ženkliai pagerinti modelio gebėjimą spręsti konkrečios srities užduotis. Šis metodas dažnai naudojamas sudėtingesnėms užduotims [SSS⁺24].

Tačiau, pateikiant užklauskas su keliais pavyzdžiais padidėja bendras užklauskos ilgis, dėl ko šio metodo panaudojimas gali taip nepraktišku jeigu užduočiai atlikti naudojamo DKM konteksto ilgio riba yra mažesnė nei visas užklauskos ilgis [PKC21].

Be to, siekiant užtikrinti aukščiausios kokybės rezultatus, svarbu kruopščiai suformuluoti ir parinkti pavyzdžius. Neteisingai suformuluoti ar netinkamai parinkti pavyzdžiai gali paveikti modelio pateikiamus atsakymus ir įvesti šališkumą [Gao23].

1.3.3. Minčių grandinės metodas

Minčių grandinės metodas (CoT) yra užklauskų formavimo metodas, skirtas pagerinti didelių kalbos modelių gebėjimą atlikti loginį samprotavimą. Taikant CoT, užduoties užklausa nėra pateikiama kaip vientisas aprašymas, o suskaidoma į mažesnius, nuoseklius žingsnius, primenančius žmogaus loginę mąstymo seką. Šis metodas padeda modeliui geriau suprasti, kaip atlikti loginį samprotavimą specifinės srities kontekste [VD24].

CoT metodas yra itin naudingas sprendžiant užduotis, kuriose reikalingas sudėtingas logikos pritaikymas, pavyzdžiui, spręsti matematines problemas, kurti algoritmus ar analizuoti programinio kodo logiką. Dalinant užduotį į atskiras, mažesnes apimties užduotis, yra sumažinama potencialių klaidų tikimybė [VD24].

Pagrindinis šio metodo privalumas yra atsakymų tikslumo ir nuoseklumo pagerinimas. CoT leidžia modeliui pateikti išsamią užduoties sprendimo eigą, užuot apsiribojus galutiniu rezultatu. Vis dėlto, CoT metodas gali būti mažiau efektyvus užduotyse, kuriose nėra reikalinga sudėtinga logika, nes papildomi žingsniai padidina užklauskos ilgį ir prailgina užklauskos apdorojimo laiką [Gao23].

1.3.4. Minčių medžio metodas

Minčių medžio metodas (ToT) yra pažangus užklauskų formavimo būdas, kuris remiasi CoT metodo principais, tačiau papildomai pateikia modeliui instrukcijas generuoti ir analizuoti alternatyvias sprendimų sekas. Užuoat sekęs vieną tiesinę sprendimo eigą, modelis struktūrizuoja užduotį kaip medžio formos duomenų struktūrą, kurioje kiekvienas žingsnis sukuria naujas galimų sprendimų šakas [YYZ⁺24].

Šis metodas yra naudingas sudėtingoms užduotims, kuriose reikia apsvarstyti kelias alternatyvas, pavyzdžiui, ieškant optimaliausio algoritmo ar sprendžiant sudėtingus matematinius uždavinius. ToT leidžia dideliame kalbos modeliui įvertinti kelis sprendimų kelius ir pasirinkti efektyviausią variantą, atsižvelgiant į iš anksto nustatytus kriterijus ar vartotojo pateiktus nurodymus [VD24].

Nors ToT metodas savo efektyvumu ir rezultatų tikslumu pralenkia CoT metodą, jis taip pat reikalauja daugiau skaičiavimo išteklių ir gali užimti itin daug konteksto atminties, ypač jei nagrinėjama užduotis yra labai sudėtinga [YYZ⁺24].

1.4. Didelių kalbos modelių analizė

1.4.1. GPT-4 modelis

GPT-4 yra vienas pažangiausių „OpenAI“ sukurtų didelių kalbos modelių, pristatytas 2023 metais. Šis modelis yra multimodalinis, gebantis apdoroti ir generuoti ne tik tekstą, bet ir vaizdus, o tai ženkliai praplečia jo pritaikymo galimybes. GPT-4 modelis turi daugiau nei 1 trilijoną parametrų ir remiasi autoregresine architektūra, kuri leidžia generuoti aukštos kokybės, natūralios kalbos tekstą [BD23].

Pagrindiniai GPT-4 modelio funkcionalumai apima: sudėtingų klausimų atsakinėjimą, kūrybinio ir techninio turinio generavimą, teksto vertimą ir santraukų sudarymą, programinio kodo generavimą, analizę, bei klaidų identifikavimą ir taisymą, vaizdų aprašymų kūrimą ir informacijos iš vaizdų išgavimą [SLM⁺24].

Modelis palaiko daugelį programavimo kalbų, į kurias įeina: Python, Java, JavaScript, C++, C#, HTML, CSS, SQL, Bash, PHP, Ruby, ir Go [AAA⁺23]. Be to, GPT-4 geba dirbti su įvairiomis integruotomis kūrimo aplinkomis (IDEs) įskaitant: „Visual Studio Code“, „Visual Studio“, „Vim“, „Neovim“, „JetBrains“, „Azure Data Studio“ [Git21].

Vis dėl to modelio pritaikymo galimybės yra ribotos, kadangi GPT-4 treniravimo duomenys ir informacija apie vidinę architektūrą nėra viešai prieinama [Kal24].

1.4.2. Codex modelis

OpenAI Codex yra 12 milijardų parametrų turintis didelis kalbos modelis, sukurtas 2021 metais, remiantis GPT-3 architektūra. Šis modelis yra orientuotas į užduotis, susijusias su programiniu kodu, ir buvo apmokytas naudojant daugiau nei 179 GB Python programavimo kalbos duomenų, surinktų iš daugiau nei 54 milijonų viešai prieinamų „GitHub“ saugyklų. Modelio treniravimui taip pat buvo naudojami dideli kiekiai kitų programavimo kalbų duomenų, tačiau galutinis treniravimo duomenų rinkinio dydis ir detalės nėra viešinami dėl „OpenAI“ taikomos privatumo politikos [XAN⁺22].

Codex modelis yra pritaikytas Python programavimo kalbai, tačiau taip pat palaiko ir kitas programavimo kalbas, tokias kaip T-SQL, Markdown, HTML, JSON, SCSS, Less, Ruby, Julia, C#, PHP, Swift, C++, Go, Perl, R, PowerShell, Rust, SQL, CSS, Java, JavaScript, TypeScript. Pagrindiniai Codex modelio funkcionalumai yra: kodo testų kūrimas, kodo pertvarkymas, kodo užbaigimas ir kodo generavimas [JWS⁺24].

Modelis taip pat palaiko darbą su IDEs, tokiomis kaip „JetBrains IDE“, „Visual Studio“, „VS Code“ ir „Neovim“. Šio modelio pagrindu buvo sukurtas įrankis „GitHub Copilot“, kuris leidžia programuotojams efektyviai rašyti kodą pasitelkiant natūralios kalbos užklausas [Ope22], [JWS⁺24].

Nors Codex modelis pasižymi savo gebėjimais atliekant užduotis susijusias su kodo generavimu, tačiau šio modelio pritaikymas yra sudėtingas. Codex modelio parametrų skaičius ir treniravimo duomenys nėra viešai prieinami, todėl nėra galimybės atlikti detalios modelio derinimo ar pritaikymo specifinėms sritims ir užduotims analizės. Dėl šios priežasties Codex naudojimas išlieka orientuotas į bendro pobūdžio programavimo užduotis. Taip pat dėl vis naujesnių ir geresnių GPT modelių atsiradimo, prieiga prie Codex modelio buvo apribota ir modelį galima pasiekti tik moksliniais tikslais gavus specialią prieigą [XAN⁺22].

1.4.3. CodeT5 modelis

CodeT5 yra 220 milijonų parametrų turintis didelis kalbos modelis, sukurtas 2021 metais, remiantis pažangia T5 modelio architektūra. Šį modelį pristatė „Salesforce Research“ – organizacija [Res24]. CodeT5 modelis yra atvirojo kodo tipo, todėl suteikia galimybę nuodugniai analizuoti jo pritaikymo sritis ir potencialias galimybes. Modelis buvo sukurtas atlikti su programiniu kodu susijusias užduotis, įskaitant kodo apibendrinimą, generavimą, vertimą tarp skirtingų programavimo kalbų, defektų identifikavimą ir jų taisymą [ZFX⁺23], [WWJ⁺21].

Šio modelio apmokymo procese buvo pasitelkti duomenys iš „CodeSearchNet“ ir „Google BigQuery“ duomenų rinkinių. CodeT5 buvo apmokytas naudojant 8,35 milijono programinio kodo vienetų: 3,16 milijono dvimodalių (funkcija ir jos komentaras anglų kalba) ir 5,19 milijono vienimodalių (tik funkcijos) pavyzdžių. Šie duomenys apima aštuonias programavimo kalbas: Python, Java, JavaScript, PHP, Ruby, Go, C ir C# [Res21].

Vienas iš pagrindinių „CodeT5“ privalumų yra jo sekos į seką architektūra, leidžianti vienodai efektyviai atlikti tiek kodo analizės, tiek generavimo užduotis. Be to, CodeT5 palaiko darbą su „VS Code“ integruota kūrimo aplinka, todėl yra paprastai ir greitai pritaikomas [WWJ⁺21], [Res21].

1.4.4. StarCoder modelis

„StarCoder“ yra 15,5 milijardo parametrų turintis didelis kalbos modelis, sukurtas 2023 metais „BigCode“ bendruomenės, kuri siekia atsakingo dirbtinio intelekto kūrimo programiniam kodui. Modelis yra viešai prieinamas ir remiasi autoregresine architektūra, leidžiančia efektyviai atlikti kodo generavimo užduotis. Viena iš išskirtinių „StarCoder“ savybių yra jo gebėjimas apdoroti iki 8 tūkstančių simbolių kontekstą, kas suteikia galimybę dirbti su ilgesniais kodo fragmentais. Pagrindiniai šio modelio funkcionalumai yra generuoti ir pertvarkyti kodą, identifikuoti klaidas ir pasiūlyti jų sprendimus, tačiau modelis nėra apmokytas atsakinėti į natūralios kalbos užklausas ir yra labiausiai pritaikomas kodo generavimo užduočių sprendimui [LAZ⁺23].

Modelio treniravimo duomenys buvo surinkti iš „The Stack“ duomenų rinkinio [KLA⁺22], iš kurio modelio treniravimui buvo atrinkta daugiau nei 1 trilijonas informacijos vienetų. Pasirinkti „The Stack“ duomenys apimė 86 programavimo kalbas. Papildomai, modelio mokymui buvo naudojami viešai prieinami „GitHub“ saugyklų duomenys bei projekto „Jupyter Notebook“ duomenys [YSD⁺24]. Siekiant padidinti Python programavimo kalbos užduočių atlikimo efektyvumą, modelis papildomai buvo optimizuotas, naudojantis 35 milijardus Python informacijos vienetų [ZNW⁺23].

Pagrindinės programavimo kalbos, kurias palaiko „StarCoder“, yra C++, C#, Java, Ruby, Python, JavaScript, PHP, Rust, Go, TypeScript ir SQL. Be to, „StarCoder“ yra suderinamas su įvairiomis integruotomis kūrimo aplinkomis, įskaitant „Visual Studio Code“, „IntelliJ IDEA“ ir „Eclipse“ [LAZ⁺23], [IS24].

1.4.5. CodeLlama modelis

„CodeLlama“ yra didelis kalbos modelis, pristatytas „Meta“ kompanijos 2023 metais. Šis modelis turi keturias skirtingų dydžių versijas: 7 milijardų, 13 milijardų, 34 milijardų ir 70 milijardų parametrų. Iš šių versijų, 70 milijardų parametrų modelis yra didžiausias ir pažangiausias [RGG⁺23].

Code Llama modelis remiasi autoregresine Llama 2 modelio architektūra, leidžiančia efektyviai atlikti tokias užduotis kaip kodo generavimas, užbaigimas, pertvarkymas, klaidų identifikavimas. Taip pat buvo pristatyta 7 milijardų parametrų modelio versija „Instruct“, kuri papildomai buvo apmokyta suprasti ir atsakinėti natūralios kalbos tekstu į su kodu susijusius klausimus [TLI⁺23]. 70 milijardų parametrų turintis Code Llama modelis buvo apmokytas naudojant 1 trilijoną informacijos vienetų iš viešai prieinamų duomenų šaltinių. Modelis yra apmokytas daugeliu programavimo kalbų, tačiau geriausiai geba dirbti su Python, C#, C++, Java, TypeScript, PHP ir Bash kalbomis [RGG⁺23]. Modelis yra suderinamas su integruota kūrimo aplinka „Visual Studio Code“ [Met23].

Code Llama modelis yra atvirojo kodo tipo, todėl jo techninė informacija yra pasiekama viešai ir gali būti laisvai analizuojama, tačiau modelio treniravimo duomenų rinkiniai nėra vieši [RGG⁺23].

1.4.6. CodeGemma modelis

CodeGemma yra „Google“ sukurto Gemma didelio kalbos modelio variantas, pristatytas 2024 metais ir paremtas Gemma modelio architektūra bei pritaikytas su programiniu kodu susijusioms užduotims spręsti. CodeGemma modelis yra autoregresinio tipo ir atvirojo kodo, todėl yra lengvai prieinamas bei suteikia galimybę plačiai juo naudoti ir pritaikyti įvairioms užduotims. CodeGemma modelis pasižymi gebėjimu generuoti ir užbaigti kodą, atlikti natūralios kalbos užduotis bei sekti instrukcijas [TZH⁺24].

CodeGemma modelis turi tris pagrindines versijas: ypač lengvą ir nereikalaujančią didelių skaičiavimo resursų CodeGemma 2B – 2 milijardų parametrų modelį, skirtą greitam kodo užbaigimui ir generavimui; CodeGemma 7B – 7 milijardų parametrų modelį, apmokytą naudojant 500 milijardų informacijos vienetų, įskaitant programinį kodą, matematikos ir natūralios kalbos duomenis, gebantį atlikti kodo generavimo, užbaigimo ir natūralios kalbos užduotis; ir CodeGemma 7B Instruct – 7 milijardų parametrų modelį, papildomai apmokytą sekti instrukcijas ir palaikyti pokalbį natūralia kalba apie programinį kodą bei matematikos užduotis [TZH⁺24].

Visi CodeGemma modelio variantai palaiko „fill-in-the-middle“ (FIM) funkcionalumą, leidžiantį užpildyti trūkstamas kodo dalis, pateikus pradžios ir pabaigos kontekstą. Modeliai taip pat palaiko įvairias programavimo kalbas, įskaitant Python, JavaScript, Java, C++, C#, Rust, Go, TypeScript, Bash, C, HTML, CSS, ir Kotlin. Be to, CodeGemma modeliai palaiko darbą su Visual Studio Code (VS Code) kūrimo aplinka [SDL⁺24].

1.4.7. Qwen2.5-Coder modelis

Qwen2.5-Coder yra atvirojo kodo didelio kalbos modelio variantas, sukurtas „Alibaba Cloud“ ir pristatytas 2024 metais. Šis modelis remiasi Qwen2.5 architektūra ir yra orientuotas į darbą su programiniu kodu, matematikos užduotimis ir natūralios kalbos apdorojimu. Modelis buvo apmokytas naudojant 5,5 trilijono informacijos vienetų turintį įvairių duomenų rinkinį, apimantį viešai prieinamą GitHub platformoje esantį programinį kodą, sintetinę informaciją, matematikos ir tekstines užduotis iš tokių duomenų rinkinių, kaip McEval-Instruct[HYC⁺24].

Qwen2.5-Coder seriją sudaro šešios skirtingo dydžio versijos: nuo 0,5 iki 32 milijardų parametrų (0.5B, 1.5B, 3B, 7B, 14B, 32B). Modelio architektūra yra autoregresinė, su iki 128 tūkstančių žodžių palaikančiu konteksto langu. Qwen2.5-Coder palaiko 92 programavimo kalbas, įskaitant Python, Java, C++, C, TypeScript, JavaScript, Go, Bash, PHP, HTML, SQL, Rust, Shell ir kitas [HYC⁺24].

Modelis yra visiškai suderinamas su „Visual Studio Code“, leidžiantis jį naudoti kaip kodo asistento įrankį ar integruoti į redagavimo ir analizės sistemas. Dėl atvirojo kodo licencijos, modelis gali būti lengvai pritaikomas moksliniams tyrimams ar individualiems sprendimams[Tea24].

1.4.8. Modelių palyginimas

1.4 skyriuje buvo analizuoti ir aprašyti septyni dideli kalbos modeliai: GPT-4, Codex, CodeT5, StarCoder, CodeLlama, CodeGemma ir Qwen2.5-Coder. Analizės metu surinkta informacija buvo susisteminta pagal apibrėžtus kriterijus, įskaitant modelio tipą, jį sukūrusią kompaniją,

sukūrimo metus, treniravimo duomenų šaltinius ir kiekius, modelio parametrų skaičių, specifikaciją, palaikomas programavimo kalbas ir integruotas kūrimo aplinkas. Taip pat buvo nustatyta, kurie modeliai yra atvirojo kodo.

1 lentelė. Didelių kalbos modelių palyginimas

Kriterijus	GPT-4	Codex	CodeT5	StarCoder	CodeLlama	CodeGemma	Qwen2.5-Coder
Modelio tipas	Autoregresinis	Autoregresinis	Sekos į seką	Autoregresinis	Autoregresinis	Autoregresinis	Autoregresinis
Kompanija	OpenAI	OpenAI	Salesforce Research	BigCode	Meta	Google	Alibaba Cloud
Metai	2023	2021	2021	2023	2023	2024	2024
Treniravimo duomenų šaltiniai	Neskelbiama	GitHub	CodeSearchNet, BigQuery	The Stack, GitHub	Neskelbiama	GitHub	McEval-Instruct, GitHub
Treniravimo duomenų kiekis	Neskelbiama	Neskelbiama	8.35 mln.	≈1 trilijonas	≈1 trilijonas	500 mlrd.	5.5 trilijonų
Parametrai	≈1 trilijonas	12 mlrd.	220 mln.	15.5 mlrd.	7– 70 mlrd.	2 ir 7 mlrd.	0.5 – 32 mlrd.
Specifikacija	Bendro naudojimo	Kodo generavimas	Kodo generavimas	Kodo generavimas	Kodo generavimas, kodo analizė	Kodo generavimas, kodo analizė	Kodo generavimas, kodo analizė
Programavimo kalbos	Python, Java, JS, C++, C#, HTML, CSS, SQL, Bash, PHP, Ruby, Go ir daugiau	Python, TSQL, Markdown, HTML, JSON, SCSS, Less, Ruby, Julia, C#, PHP, Swift, C++, Go, Perl, R, PowerShell, Rust, SQL, CSS, Java, JS, TS	Python, Java, JS, PHP, Ruby, Go, C, C#	C++, C#, Java, Ruby, Python, JS, PHP, Rust, Go, TS, SQL ir daugiau	Python, C#, C++, Java, TS, PHP, Bash ir daugiau	Python, JavaScript, Java, C++, C#, Rust, Go, TypeScript, Bash, C, HTML, CSS, Kotlin	Python, Java, C++, C#, TypeScript, JS, Go, Bash, PHP, HTML, SQL, Rust, Shell ir daugiau
Palaikomos IDEs	VS Code, Visual Studio, Vim, Neovim, JetBrains, Azure Data Studio	JetBrains IDE, Visual Studio, VS Code, Neovim	VS Code	VS Code, IntelliJ IDEA, Eclipse	VS Code	VS Code	VS Code
Atvirojo kodo	Ne	Ne	Taip	Taip	Taip	Taip	Taip

Nagrinėtų didelių kalbos modelių skirtingų aspektų palyginimai, remiantis informacija surinkta moksliniuose darbuose ir techninėse dokumentacijoje yra pateikiami 1 lentelėje. Lentelė atspindi pasirinktas modelių vertinimo sritis ir leidžia geriau suprasti šių modelių technines detales, panašumus ir skirtumus.

Šeši iš septynių nagrinėtų DKM (GPT-4, Codex, StarCoder, CodeLlama, CodeGemma, Qwen2.5-Coder) yra autoregresiniai, o vienintelė išimtis yra CodeT5, kuris naudoja sekos į seką architektūrą.

Codex ir CodeT5 modeliai buvo pristatyti 2021 metais, GPT-4, StarCoder ir CodeLlama – 2023 metais, o naujausi modeliai – CodeGemma ir Qwen2.5-Coder – buvo pristatyti 2024 metais. Codex modelio prieiga dabar yra ribota ir pasiekama tik moksliniams tikslams gavus specialų leidimą, dėl ko šio modelio praktinis pritaikymas yra ribotas. GPT-4, kaip vienintelis bendros paskirties ir multimodalinis DKM, turi gebėjimą apdoroti tekstą ir vaizdus, o likę modeliai yra skirti specifinėms su programiniu kodu susijusioms užduotims.

OpenAI sukurti modeliai (GPT-4, Codex) nėra atvirojo kodo tipo, todėl jų treniravimo duomenų šaltiniai ir techninė informacija nėra viešai prieinama. Tuo tarpu CodeT5, StarCoder, CodeLlama, CodeGemma ir Qwen2.5-Coder yra atvirojo kodo modeliai, leidžiantys detalesnę šių modelių analizę ir pritaikymą. Visgi CodeLlama, kaip ir OpenAI modeliai, neskelbia savo treniravimo duomenų šaltinių.

Treniravimo duomenų kiekiai taip pat skiriasi: Didžiausiu treniravimo duomenų kiekiu buvo apmokytas Qwen2.5-Coder modelis, kurio treniravimo duomenų rinkinį sudarė apytiksliai 5,5 trilijonų informacijos vienetų, StarCoder ir CodeLlama modeliai buvo apmokyti naudojant apytiksliai 1 trilijoną informacijos vienetų, CodeGemma – 500 milijardų, o CodeT5 – vos 8,35 milijono vienetų. GPT-4, Codex ir Qwen2.5-Coder neskelbia tikslių treniravimo duomenų kiekių, dėl ko jų palyginimas su kitais modeliais yra sudėtingesnis.

Parametrų skaičius taip pat varijuoja. GPT-4 yra didžiausias modelis, turintis apytiksliai 1 trilijoną parametrų, po jo eina CodeLlama (iki 70 mlrd.), StarCoder (15,5 mlrd.), Codex (12 mlrd.), CodeGemma (2–7 mlrd.), Qwen2.5-Coder (0,5–32 mlrd.) ir mažiausias – CodeT5 (220 mln.).

Visi nagrinėti DKM buvo apmokyti bent su 8 skirtingomis programavimo kalbomis, o dažniausiai palaikomos programavimo kalbos buvo Python, Java, JavaScript, PHP ir C#. Pagal nagrinėtų modelių viešai skelbiamą informaciją StarCoder ir Qwen2.5-Coder palaiko daugiausiai programavimo kalbų – atitinkamai 86 ir 92. Visi nagrinėti modeliai integruojasi su Visual Studio Code kūrimo aplinka, o trys iš penkių analizuotų modelių (GPT-4, Codex, StarCoder) taip pat palaiko ir kitas aplinkas tarp kurių yra IntelliJ IDEA, Eclipse ir JetBrains.

1.5. Skyriaus apibendrinimas ir išvados

Atsižvelgiant į atliktą didelių kalbos modelių literatūros analizę ir pasirinktų septynių modelių palyginimą, CodeGemma, CodeLlama ir Qwen2.5-Coder išsiskiria kaip labiausiai logiški pasirinkimai ir tarpusavyje gerai lyginami modeliai įvairiems tyrimams. Šie modeliai yra vieni naujausių didelių kalbos modelių srityje, o jų dydis ir techninės savybės, tokios kaip parametrų skaičius, užtikrina efektyvumą sprendžiant įvairias užduotis. Visi trys modeliai yra apmokyti plačiu programavimo kalbų rinkiniu, įskaitant Python, Java, C# ir daugelį kitų, kas padidina jų pritaikomumą įvairiose su programiniu kodu susijusiose srityse.

Tuo tarpu, Codex modelis, kuris taip pat buvo analizuotas, yra senesnis, pristatytas dar 2021 metais ir pasižymi ribota prieiga, o tai apsunkina jo pritaikymą. CodeT5 modelis, nors ir atvirojo kodo tipo, yra mažiausias su vos 220 mln. parametrų ir apmokytas naudojant tik 8,35 mln. treniravimo duomenų, dėl ko apsunkina jo lyginimą su kitais modeliais. GPT-4 modelis pasižymi itin aukštomis techninėmis savybėmis ir yra lyderiaujantis DKM rinkoje, tačiau nėra atvirojo kodo ir viešai prieinamas testavimams. StarCoder modelis yra pritaikomas užduotims susijusioms su kodo generavimu, tačiau nėra apmokytas atsakinėti į natūralios kalbos užklausas ir analizuoti kodo, todėl jo pritaikymo galimybės taip pat yra ribotos.

Dideli kalbos modeliai CodeGemma, CodeLlama ir Qwen2.5-Coder lyginant su nagrinėtomis alternatyvomis labiau atitinka dabartinių tyrimų poreikius.

2. Kodo kokybės vertinimo metodai

2.1. Kodo kokybės vertinimo metodai ir jų efektyvumas

2.1.1. Rankinės kodo peržiūros

Kodo peržiūros yra vienas seniausių ir efektyviausių kodo kokybės analizės metodų. Šis metodas leidžia identifikuoti klaidas, kurios gali būti nepastebimos automatizuotoms priemonėms, pavyzdžiui logikos trūkumus ar kodo semantinius neatitikimus [SSC⁺18]. Peržiūros metu kodas yra analizuojamas kolegų ar ekspertų siekiant identifikuoti klaidas, struktūrinius trūkumus ir optimizavimo galimybes. Šis metodas yra ne tik efektyvus kokybės gerinimui, bet ir skatina komandinį darbą bei žinių dalijimąsi [BB13]. Tačiau, kodo peržiūros taip pat reikalauja nemažai laiko ir išteklių, ypač kai yra vykdomos didelės apimties projektuose. Efektyvių peržiūrų vykdymui yra svarbu užtikrinti, kad komandos nariai turėtų pakankamai žinių ir patirties tiksliai identifikuoti problemas ir pasiūlyti tinkamus sprendimus. Be to, kodo peržiūroms yra reikalingas aiškiai apibrėžtas vykdymo procesas bei komunikacijos standartai, kurių tikslas yra padėti išvengti subjektyvumo vertinimo metu [BB13].

2.1.2. Automatizuota kodo kokybės analizė

Automatizuoti kodo analizės įrankiai, tokie kaip SonarQube, PMD, ir SciTools Understand, leidžia greitai ir efektyviai analizuoti didelius kodų kiekius [GEG19]. Šie įrankiai analizuoja kodą pagal statines kodo kokybės metrikas, siekiant aptikti kodavimo standartų pažeidimus, kodo struktūros klaidas bei potencialias sudėtingumo ar apimties problemas [GEG19]. Pagrindiniai automatizuotų įrankių naudojimo privalumai yra jų greitis, objektyvumas ir galimybė juos lengvai pritaikyti didelėse programinės įrangos sistemose [AAM23]. Nepaisant to, šių įrankių efektyvumas yra gana ribotas, kai reikia suprasti sudėtingą kodo logiką ar aptikti semantines klaidas, kurios gali būti nepastebimos be žmogaus įsikišimo. Be to, įrankių diegimas ir naudojimas reikalauja ne tik techninių žinių, bet ir strateginio supratimo apie jų veikimą ir galimybes. Dėl neteisingai sukonfigūruoto įrankio arba svarbių kodo konteksto aspektų ignoravimo įrankis gali pateikti klaidinančius rezultatus [THG21]. Vis dėlto, naudojami tikslingai automatizuoti įrankiai yra nepakeičiama priemonė užtikrinant kodo kokybę. Jie leidžia įvertinti kodo kokybę daugeliu aspektu nereikalaudami didelių laiko ir resursų sąnaudų, todėl jų naudojimas tampa būtinas šiuolaikiniuose programinės įrangos kūrimo procesuose [GEG19].

2.2. Kiekybiniai kodo vertinimo aspektai

2.2.1. Statinės ir dinaminės kodo kokybės metrikos

Žemo lygio perspektyvoje, visos programinės įrangos metrikos gali būti suskirstytos į dvi pagrindines kategorijas: statinės ir dinaminės metrikos [Ves15]. Statinės metrikos vertina kodo struktūrą ir sintaksę, paties kodo nevykdant. Jos vertinamos ankstyvuose programinės įrangos kūrimo etapuose ir padeda nustatyti kodo sudėtingumą, palaikomumą bei atitikimą gerosioms praktikoms. Kodo analizė naudojantis statinėmis metrikomis, dar vadinama statine kodo analize,

yra gana nesudėtingas procesas ir dažniausiai gali būti automatizuotas naudojant tokius įrankius kaip SonarQube ar SciTools Understand [OMS⁺18].

Tuo tarpu dinaminės metrikos vertina kodo kokybę, programinės įrangos vykdymo metu. Jos vertinamos vėlyvuose programinės įrangos kūrimo etapuose ir suteikia informacijos apie kodo našumą, saugumą ir tikslingą atminties panaudojimą. Tačiau, lyginant su statine kodo analize, dinaminė kodo analizė yra daug sudėtingesnis procesas, kuris paprastai reikalauja specialių testavimo metodų, bei papildomų įrankių programos stebėjimui jos veikimo metu [GC08].

Kadangi dideli kalbos modeliai neturi galimybės vykdyti programinės įrangos paleidimo, jie negali atlikti dinaminės kodo analizės. Todėl toliau šiame darbe bus analizuojamos statinės kodo metrikos, jų ypatumai ir detalesnė klasifikacija.

2.2.2. Statinių kodo kokybės metrikų klasifikavimas

Nors nėra vieningos ir konkrečiai apibrėžiamos statinių kodo kokybės metrikų klasifikavimo sistemos, įvairūs moksliniai tyrimai ir programinės įrangos analizės įrankiai siūlo skirtingus būdus suskirstyti šias metrikas. Vienas iš tokių įrankių yra SciTools Understand. Šis statinės kodo analizės įrankis naudoja įvairias kodo metrikas, leidžiančias įvertinti kodo struktūrą, sudėtingumą ir apimtį. SciTools Understand ne tik padeda programinės įrangos kūrėjams geriau suprasti ir prižiūrėti kodą, bet ir teikia viešai prieinamą išsamų metrikų rinkinį, pritaikytą skirtingoms kodo analizės sritims.

SonarQube yra dar vienas plačiai naudojamas įrankis, siūlantis išsamų kodo kokybės metrikų rinkinį, skirtą analizuoti kodo sudėtingumą, aiškumą, klaidas ir kitus kokybės aspektus. Jo pateikiamos metrikos dažnai naudojamos praktikoje siekiant įvertinti ir pagerinti bendrą programinės įrangos kokybę.

Šio darbo kontekste apibrėžiama statinių kodo kokybės metrikų klasifikavimo sistema, pagrįsta SciTools Understand [Sci24] ir SonarQube [Son24] metrikų dokumentacijomis bei 2017 metais atliktu kodo metrikų tyrimu [NPM⁺17]. Ši klasifikacija suskirstyta į tris pagrindines kategorijas: sudėtingumo metrikos, apimties metrikos ir objektiškai orientuotos metrikos.

2 lentelė. Statinių kodo kokybės metrikų klasifikacija

Sudėtingumo metrikos	Apimties metrikos	Objektiškai orientuotos metrikos
Ciklominis sudėtingumas (Cyclomatic Complexity)	Pasikartojančių eilučių skaičius (Duplicated Lines)	Klasių sanglauda (Coupling Between Objects)
Vidutinis ciklominis sudėtingumas (Average Cyclomatic Complexity)	Komentarų ir kodo santykis (Comment to Code Ratio)	Klasėje apibrėžtų metodų skaičius (Weighted Methods per Class)
Kognityvinis sudėtingumas (Cognitive Complexity)	Vidutinis eilučių skaičius (Average Lines)	Atsakas į klasę (Response for a Class)
Esminis sudėtingumas (Essential Complexity)	Vidutinis kodo eilučių skaičius (Average Code Lines)	Paveldimumo gylis (Depth of Inheritance Tree)
Vidutinis esminis sudėtingumas (Average Essential Complexity)	Vidutinis tuščių eilučių skaičius (Average Blank Lines)	Metodų sanglaudos stoka (Lack of Cohesion in Methods)
Kodo gylis (Nesting Depth)	Vidutinis komentarų eilučių skaičius (Average Comment Lines)	Apsaugotų metodų skaičius (Protected Methods)
Kodo kelių skaičius (Paths)	Neaktyvių eilučių skaičius (Inactive Lines)	Klasės metodų skaičius (Class Methods)
Kodo mazgų skaičius (Knots)	Komentarų eilučių skaičius (Comment Lines)	Klasės kintamųjų skaičius (Class Variables)
	Kodo eilučių skaičius (Lines of Code)	Vaikų skaičius (Number of Children)
	Tuščių eilučių skaičius (Blank Lines)	Privačių metodų skaičius (Private Methods)
	Eilučių skaičius (Lines)	Viešų metodų skaičius (Public Methods)
		Įeinančių elementų skaičius (Fan in)
		Išeinančių elementų skaičius (Fan Out)

Nagrinėta statinių metrikų klasifikacija padeda suprasti, kaip skirtingos metrikos prisideda prie kodo kokybės vertinimo. 2 lentelėje pateikiamos pagrindinės statinės kodo kokybės metrikos, suskirstytos į tris kategorijas: sudėtingumo metrikos, leidžiančios įvertinti kodo struktūros sudėtingumą, apimties metrikos, analizuojančios kodo kiekybinius aspektus ir objektiškai orientuotos metrikos, vertinančios objektų tarpusavio ryšius bei priklausomybes.

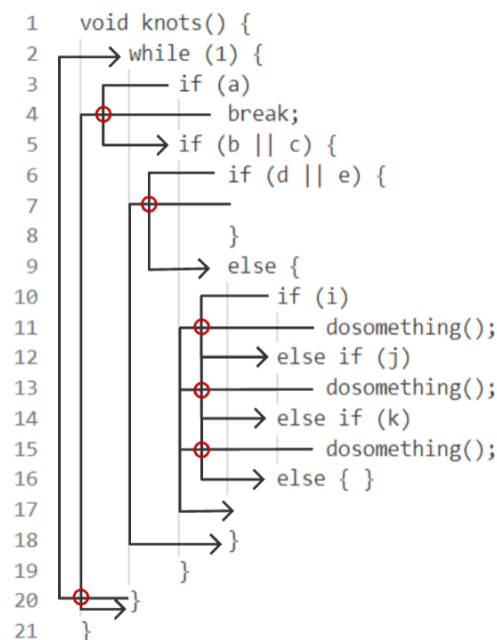
2.3. Statinių kodo kokybės metrikų analizė

2.3.1. Sudėtingumo metrikos

Sudėtingumo metrikos yra rodikliai, skirti įvertinti programinės įrangos ar jos dalių sudėtingumą. Jos padeda nustatyti, kaip sudėtinga yra suprasti, pakeisti ar prižiūrėti analizuojamą kodą, jo struktūrą ir logiką [HWY09].

Kodo sudėtingumas priklauso nuo tokių veiksnių, kaip ciklų ir sąlyginių sakinių skaičiaus, jų įdėjimo gylis bei naudojamų kintamųjų skaičiaus. Kuo daugiau kodas turi sudėtingų struktūrų ir elementų, tuo sunkiau yra analizuoti ir suprasti programos logiką, o tai didina klaidų tikimybę. Taip pat įtakos kodo sudėtingumui turi bendras kodo struktūravimas ir organizavimas [Gha15].

Skirtingos kodo sudėtingumo metrikos, leidžia vertinti kodo kokybę pagal įvairius aspektus. Tarp dažniausiai naudojamų yra ciklominio sudėtingumo ir kognityvinio sudėtingumo metrikos, kurios analizuoja kodą iš skirtingų perspektyvų: ciklominis sudėtingumas matuoja kodo loginių kelių skaičių, o kognityvinis sudėtingumas vertina, kaip sudėtinga žmogui suprasti kodo struktūrą [WH21]. Tarp kitų svarbių, tačiau paprasčiau apskaičiuojamų sudėtingumo metrikų verta paminėti kodo gylio ir kodo mazgų skaičiaus metrikas. Kodo gylis nusako, kiek kartų loginiai blokai, tokie kaip if, while, for, switch yra įdėti vienas į kitą. Kuo didesnis įdėjimo gylis, tuo sudėtingiau sekti programos loginę eigą ir atlikti kodo pakeitimus. Mazgų skaičiaus metrika, vizuoliai pateikta 1 paveikslėlyje, parodo, kaip susipynęs yra programos valdymo srautas. Ji matuoja, kiek kartų valdymo srauto komandos (if, else if, for, while), susikerta tarpusavyje vykstant kodo šuoliui. Kiekvienas toks susikirtimas yra laikomas mazgu. Kuo daugiau mazgų, tuo painesnis kodo valdymas ir tuo sunkiau jį modifikuoti, bei testuoti [Sci24].



1 pav. Kodo mazgų skaičius

2.3.1.1. Ciklominis sudėtingumas

Ciklominis sudėtingumas yra svarbi sudėtingumo metrika, naudojama programinės įrangos kokybei įvertinti. Ji matuoja nepriklausomų loginių kelių skaičių kode, analizuojant programos struktūrą kaip grafą, kurio mazgai atitinka vykdymo blokus, o briaunos – perėjimus tarp jų. Ši metrika buvo pristatyta Thomas J. McCabe 1976 metais ir plačiai naudojama kodo suprantamumui bei palaikymo ir testavimo sudėtingumui įvertinti [McC76].

Ciklominis sudėtingumas apskaičiuojamas pagal formulę:

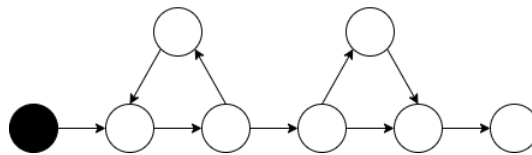
$$M = E - N + 2P \quad (1)$$

kur M – sudėtingumo reikšmė, E – briaunų skaičius, N – mazgų skaičius, o P – sujungtų komponentų skaičius [McC76]. Aukštas ciklominis sudėtingumas rodo, kad kodas yra sunkiau

modifikuojamas, perpanaudojamas ir testuojamas, nes turi daugiau loginių šakų. Praktikoje rekomenduojama, kad ciklominio sudėtingumo reikšmė viename modulyje neviršytų 10 [HWY09]. Nors šios metrikos apskaičiavimas rankiniu būdu yra sudėtingas ir reikalauja daug laiko, yra automatizuotų įrankių, tokių kaip SonarQube, kurie šį procesą atlieka greitai ir efektyviai. Ciklominio sudėtingumo metrika yra viena iš svarbiausių kodo struktūrinio sudėtingumo analizės priemonių, tačiau ji turi ir trūkumų, nes neatsižvelgia į kodo suprantamumą [LKJ23].

Toliau pateiktas 2 paveikslėlis vaizduoja grafą, kuris iliustruoja nesudėtingos programos veikimą. Programos vykdymas prasideda pradiniam, juodai nuspaltintame mazge. Tuomet programa patenka į ciklą (pirmoji trijų mazgų grupė). Po ciklo seka sąlyginis sakiny (antroji trijų mazgų grupė) ir galiausiai programa baigia darbą galutiniame mazge. Šis grafas turi 9 briaunas, 8 mazgus ir 1 komponentą, todėl ciklominis sudėtingumas apskaičiuojamas kaip:

$$M = 9 - 8 + 2 \cdot 1 = 3$$



2 pav. Ciklominio sudėtingumo grafas atvaizduojantis nepriklausomus loginius kelius programoje

2.3.1.2. Kognityvinis sudėtingumas

Kognityvinis sudėtingumas yra moderni kodo kokybės metrika, skirta įvertinti, kaip sudėtinga suprasti programos logiką. Ši metrika buvo sukurta „SonarSource“ kompanijos 2017 metais kaip alternatyva ciklominio sudėtingumo metrikai. Skirtingai nuo ciklominio sudėtingumo, kuris daugiausia dėmesio skiria kodo vykdymo kelių skaičiui, kognityvinis sudėtingumas orientuojasi į kodo skaitymo ir interpretavimo lengvumą [Son17].

Kognityvinio sudėtingumo metrika analizuoja tokius kodo veiksnus, kaip loginiai elementai ir jų tarpusavio sąveika. Sudėtingumą didina ciklai ir rekursijos, nes reikalauja programuotojo dėmesio sekant jų vykdymo eigą. Be to, ši metrika atsižvelgia į gylį – kuo giliau loginė kodo struktūra įdėta, tuo sunkiau ją suprasti [MWW20].

Kitaip nei ciklominis sudėtingumas, kognityvinis sudėtingumas nevertina visų struktūrų vienodai. Jis atsižvelgia į kodo konstrukcijas, kurios pagerina skaitomumą, pavyzdžiui, aiškiai pavadintus metodų iškvietimus ar supaprastintas operatorių formas. Be to, ši metrika apima šiuolaikines programavimo kalbų ypatybes, tokias kaip „try-catch“ blokai [Son17].

SonarQube yra vienas pagrindinių šios metrikos taikymo pavyzdžių. Jis leidžia automatiškai apskaičiuoti kognityvinį sudėtingumą ir identifikuoti sudėtingas programos dalis, kurias reikėtų optimizuoti siekiant didesnio kodo aiškumo [LKJ23].

2.3.2. Apimties metrikos

Apimties metrikos yra rodikliai, padedantys įvertinti programinę įrangą kiekybiniais ir struktūros aspektais. Jos suteikia informaciją apie kodo apimtį, kuri yra svarbi analizuojant kodo palaikomumą, skaitomumą ir bendrą kokybę [WN12].

Kodo apimtį lemia įvairūs veiksniai, tokie kaip kodo eilučių kiekis, komentarų eilučių kiekis, pasikartojančios kodo dalys. Vienos iš paprasčiausių, bet plačiausiai naudojamų apimties metrikų yra kodo, komentarų bei tuščių eilučių skaičius kode. Šios metrikos yra dažnai naudojamos vertinant kodo apimtį, kadangi yra lengvai ir greitai skaičiuojamos [NPM⁺17].

Kita plačiai naudojama metrika yra komentarų ir kodo santykis, kuris parodo, kiek dokumentacijos pateikiama kartu su kodu. Ši metrika vertina, kaip išsamiai yra aprašomas kodas ir gali padėti padaryti išvadų apie kodo suprantamumą, tačiau ji nevertina komentarų kokybės, todėl leidžia analizuoti tik kiekybinius kodo aspektus [He19].

Dar viena lengvai pritaikoma kodo apimties metrika yra pasikartojančių eilučių skaičius, kuri analizuoja kodą ieškodama pasikartojančių fragmentų. Ši metrika padeda nustatyti poreikį pertvarkyti ir gerinti kodą [MBH17].

Apimties metrikos padeda įvertinti kodą kiekybiniais aspektais, yra nesudėtingai skaičiuojamos ir lengvai pritaikomos, tačiau paprastai yra naudojamas kartu su kitomis metrikomis, kadangi jos vienos neatspindi kodo kokybės ir struktūros [WN12].

2.3.3. Objektiškai orientuotos metrikos

Objektiškai orientuotos metrikos yra kodo kokybės rodikliai, kuriais yra vertinami objektų tarpusavio ryšiai. Šios metrikos padeda nustatyti, kaip gerai yra taikomi objektinio programavimo principai ir suteikia galimybę analizuoti klasių ir metodų priklausomybes, struktūrą bei jų sąveiką, taip identifikuojant galimus trūkumus. Vienas iš svarbiausių indėlių šioje srityje yra Chidamber ir Kemerer metrikų rinkinys, kuris buvo pristatytas 1991 metais. Šis rinkinys, apima šešias pagrindines metrikas, skirtas objektiškai orientuoto programinio kodo vertinimui [CK94], [SPD⁺05].

Klasėje apibrėžtų metodų skaičius yra glaudžiai su apimties metrikomis susijusi metrika, skirta įvertinti klasės sudėtingumą, apskaičiuojant joje apibrėžtų metodų skaičių [SPD⁺05]. Didelis klasėje apibrėžtų metodų skaičius yra siejamas su didesne klaidų tikimybe ir žemesne kodo kokybe, nes tokią klasę sunkiau suprasti ir prižiūrėti. Norint palaikyti kodo prižiūrimumą nesudėtingą, rekomenduojama riboti metodų skaičių klasėje. Nors tikslios ribos nėra, dažnai rekomenduojama, kad klasėje būtų ne daugiau kaip 20–50 metodų. Klases su aukštu metodų skaičiumi, patartina pertvarkyti, suskaidant jas į mažesnes klases [CK94].

Paveldėjimo gylis yra metrika, skirta įvertinti programos paveldėjimo hierarchijos gylį, matuojant maksimalų kelią nuo klasės iki šakninės klasės. Kuo didesnis paveldėjimo gylis, tuo daugiau metodų ir kintamųjų klasė paveldi, dėl ko auga ir jos sudėtingumas. Gilios paveldėjimo hierarchijos gali padidinti kodo pakartotinį panaudojimą, tačiau jos taip pat komplikuoja kodo suprantamumą ir palaikomumą. Aukštas paveldėjimo gylis yra siejamas su didesne klaidų tikimybe.

Siekiant palaikyti optimalų programos sudėtingumą, rekomenduojama, kad paveldėjimo gylio metrikos reikšmė neviršytų 5 [CK94].

Vaikų skaičius yra metrika, vertinanti klasės vaikinių klasių skaičių [SPD⁺05]. Ji parodo paveldėjimo hierarchijos plotį, skirtingai nei paveldėjimo gylis, kuris matuoja hierarchijos gilumą. Didesnis vaikų skaičius dažnai siejamas su tėvinės klasės metodų pakartotiniu panaudojimu, tačiau tai taip pat reiškia, kad tėvinė klasė tampa sudėtingesnė [CK94].

Klasių sanglauda yra metrika, vertinanti, kiek klasių yra tiesiogiai susijusios su analizuojama klase. Dvi klasės laikomos susietomis, jei vienos klasės metodai naudoja kitos klasės metodus ar kintamuosius [BBM96]. Aukšta klasių sanglaudos reikšmė rodo stiprią priklausomybę tarp klasių, kuri apsunkina kodo moduliškumą ir pakartotinį panaudojimą. Be to, didelis klasių tarpusavio ryšių skaičius padidina jautrumą pokyčiams, nes bet kokie susijusių klasių pakeitimai gali sukelti grandininis efektus, dėl kurių atsiranda naujų defektų. Sanglaudos reikšmė, viršijanti 14, signalizuoja apie nekokybišką kodą, kadangi dėl per didelės klasių sanglaudos tokį kodą yra sudėtinga palaikyti ir prižiūrėti [CK94].

Atsakas į klasę yra metrika, kuri įvertina, kiek metodų gali būti vykdoma, kai klasė gauna užklausą. Tai apima tiek klasės viduje apibrėžtus metodus, tiek tuos, kurie yra iškviečiami iš kitų klasių. Didesnė atsako į klasę reikšmė rodo, kad klasę sunkiau suprasti, testuoti ir modifikuoti, nes kiekvienas klasės sąryšis su kitomis klasėmis didina kodo priklausomybes. Klasė, kuri iškviečia daug išorinių metodų, gali sukelti problemų keičiant ar testuojant kodą, nes net nedidelis pakeitimas vienoje iš susijusių klasių gali turėti nenumatytų pasekmių. Siekiant išlaikyti aukštą kodo kokybę, rekomenduojama riboti klasės atsakymų skaičių ir sumažinti jos priklausomybę nuo kitų klasių [CK94].

Metodų sanglaudos stoka yra metrika, skirta įvertinti, kiek metodai klasėje tarpusavyje sąveikauja per bendrus kintamuosius [BBM96]. Ji padeda nustatyti, ar klasė yra pakankamai vientisa, ar bando atlikti per daug skirtingų funkcijų. Jei daugelis klasės metodų naudoja skirtingus kintamuosius tai rodo žemą sanglaudą. Aukšta metodų sanglaudos stokos reikšmė signalizuoja, kad klasė gali būti išskaidyta į kelias mažesnes, labiau specializuotas klases, kurios būtų aiškesnės ir lengviau prižiūrimos. Toks kodo organizavimo principas padeda palaikyti vientisumą klasėse ir mažina klaidų tikimybę. Klasės su aukšta metodų sanglaudos stokos reikšme yra sunkiau testuojamos, labiau linkusios į klaidas ir sudėtingesnės suprasti, nes jų metodai nėra pakankamai susiję [CK94].

Be Chidamber ir Kemerer metrikų rinkinio, objektiškai orientuoto kodo analizėje taip pat naudojamos papildomos metrikos, leidžiančios įvertinti metodų tarpusavio ir vidines priklausomybes. Viena iš jų yra įeinančių elementų skaičius, nusakanti, kiek skirtingų priklausomybių – kitų metodų, globalių kintamųjų ir parametrų – metodas naudoja. Kita glaudžiai susijusi metrika – išėinančių elementų skaičius, apibrėžianti, kiek kitų metodų yra kviečiami iš šio metodo, bei kiek globalių kintamųjų ar parametrų jis keičia ir nustato. Didesnės šių metrikų reikšmės rodo aukštesnį priklausomumo lygį, kuris gali apsunkinti testavimą ir padidinti kodo jautrumą pokyčiams [Sci24].

2.4. Subjektyvūs kodo kokybės kriterijai

Remiantis ISO/IEC 25010:2023 standartu, apibrėžiančiu programų sistemos produkto kokybės vertinimo charakteristikas, palaikomumas yra tiesiogiai priklausomas nuo kodo kokybės. Palaikomumo kokybės charakteristika turi keturias sub-charakteristikas: analizuojamumas, modifikuojamumas, perpanaudojamumas ir testuojamumas [11]. Šie subjektyvūs kokybės kriterijai nėra tiesiogiai apskaičiuojami ar išmatuojami kaip statinės metrikos, tačiau jie koreliuoja su struktūriniais kodo kokybės rodikliais [IGM11; ST22].

Perpanaudojamumas parodo, kaip lengvai programinis kodas gali būti pakartotinai naudojamas kitose sistemose be reikšmingų pakeitimų. Šiam kriterijui įtakos turi bendras kodo paprastumas ir aiškumas. Metrikos, kurios su tuo glaudžiai susijusios, yra ciklopatinis sudėtingumas, kodo eilučių skaičius, komentarų eilučių skaičius, tuščių eilučių skaičius [IGM11]. Mažesnis ciklopatinis sudėtingumas leidžia spręsti, kad kodas yra lengviau pritaikomas kitos programos kontekste. Mažesnė kodo apimtis dažnai reiškia universalesnę logiką, o kokybiški komentarai didina jo skaitomumą ir supaprastina atkartojamumą.

Modifikuojamumas apibūdina, kaip lengva atlikti pakeitimus kode, išlaikant esamą funkcionalumą. Ši kokybės aspektą geriausiai atspindi ciklopatinis sudėtingumas, kodo gylis, kodo mazgų skaičius, kodo eilučių skaičius bei pasikartojančių eilučių skaičius [IGM11]. Didelis ciklopatinis sudėtingumas, kodo gylis ir kodo mazgų skaičius rodo sudėtingą loginę struktūrą ir stiprias vidines priklausomybes, dėl kurių pakeitimai gali turėti poveikį kitoms kodo dalims. Jeigu programoje tas pats funkcionalumas kartojasi keliose vietose, kyla rizika modifikavus vieną vietą neatnaujinti ir kitų [AM13].

Analizuojamumas atspindi, kaip lengva suprasti kodą siekiant diagnozuoti klaidas, atlikti analizę ar suprasti sistemos logiką. Šiai savybei didžiausią įtaką turi kognityvinis sudėtingumas, kodo gylis, komentarų eilučių skaičius ir pasikartojančių eilučių skaičius [HKV07]. Aukštas kognityvinis sudėtingumas reiškia, kad programuotojui sunku sekti loginę eigą, ypač kai yra daug sudėtingų struktūrų [LAL⁺23]. Kokybiški komentarai padeda greičiau suprasti kodo paskirtį. Pasikartojančios kodo dalys mažina aiškumą, nes reikia įsiminti, kur ir kaip jos naudojamos [TCM⁺18].

Testuojamumas nusako, kiek efektyviai galima testuoti programinio kodo fragmentą, ar lengva parašyti vienetinius testus ir kiek izoliuotas yra testuojamas komponentas. Tam reikšmingos ciklopatinio sudėtingumo, kodo mazgų, kodo eilučių skaičiaus, įeinančių elementų skaičiaus ir išeinančių elementų skaičiaus metrikos [IGM11]. Didesnis įeinančių elementų skaičius reiškia, kad komponentas priklauso nuo daug vidinių elementų (globalių kintamųjų, parametrų), o išeinančių elementų skaičius – kad jis pats kviečia daug kitų komponentų, todėl taip pat turi didelę priklausomybę nuo kitų kodo dalių. Abi šios metrikos padidina testavimo sudėtingumą, nes apsunkina kodo izoliaciją [ST22]. Ciklopatinis sudėtingumas ir kodo mazgų skaičius apsunkina testų logiką, o didelės apimties metodai taip pat apsunkina testavimą, nes paprastai reikalauja didesnio kiekio testų.

Žemiau pateiktoje 3 lentelėje yra apibendrinama, kurios statinės kodo kokybės metrikos stipriausiai koreliuojasi su subjektyviais ISO/IEC 25010:2023 palaikomumo kriterijais.

3 lentelė. Statinių metrikų ir ISO/IEC 25010:2023 palaikomumo subcharakteristikų tarpusavio ryšys (subjektyvūs kriterijai)

Metrika	Perpanaudojamumas	Modifikuojamumas	Analizuojamumas	Testuojamumas
Ciklomatinis sudėtingumas	x	x		x
Kognityvinis sudėtingumas			x	
Kodo gylis		x	x	
Kodo mazgų skaičius		x		x
Kodo eilučių skaičius	x		x	x
Komentarų eilučių skaičius	x		x	
Tuščių eilučių skaičius	x			
Pasikartojančių eilučių skaičius		x	x	
Įeinančių elementų skaičius				x
Išeinančių elementų skaičius				x

2.5. Metrikų pasirinkimo kriterijai kodo kokybei vertinti su dideliais kalbos modeliais

Dideli kalbos modeliai, ypač tie, kurie yra specifiskai apmokyti programiniu kodu, leidžia juos pritaikyti, užduočių susijusių su kodo rašymu ar analize, sprendimui. Tačiau jų gebėjimai yra riboti, nes modeliai nesugeba tiesiogiai vykdyti kodo ir negali analizuoti dinaminių kodo veikimo aspektų, tokių kaip našumas ar atminties naudojimas. Atsižvelgiant į modelių galimybes, svarbu įvertinti, kurios metrikos gali būti efektyviai taikomos analizuoti kodą naudojant didelius kalbos modelius.

Norint nustatyti, kurios kodo metrikos yra tinkamos didelių kalbos modelių pritaikymui kodo kokybės analizei, svarbu įvertinti esminius aspektus, kurie apibrėžia jų gebėjimus ir ribas.

Vienas svarbiausių kriterijų yra metrikų statinis pobūdis. DKM negali vykdyti kodo ar stebėti jo veikimo realiuoju laiku, todėl jie nėra tinkami dinaminių metrikų, tokių kaip našumas, atminties panaudojimas ar vykdymo laiko analizė, vertinimui. Dėl šios priežasties svarbu pasirinkti tokias metrikas, kurių analizei nėra reikalingas kodo paleidimas.

Taip pat svarbu yra atsižvelgti į didelių kalbos modelių gebėjimus analizės apimties požiūriu. DKM efektyviai analizuoja atskirus kodo fragmentus, funkcijas ar klases, tačiau jų galimybės įvertinti tarpusavio sąveiką tarp modulių ar suprasti sudėtingų programų architektūrą yra ribotos. Tai ypač aktualu jei norima vertinti didelės apimties projektus, kuriuose elementų tarpusavio priklausomybės yra itin svarbios. Todėl metrikos, kurias galima vertinti lokaliai, nepriklausomai nuo išorinių kontekstų, yra pritaikomos lengviau.

Tinkamų metrikų pasirinkimą lemia ir jų interpretacijos paprastumas. DKM pritaikymui ypač tinka metrikos, kurios turi aiškias ir lengvai interpretuojamas reikšmes. Tokios metrikos kaip ciklomatinis sudėtingumas ar eilučių skaičius yra lengvai suprantamos ne tik žmonėms, bet ir modeliams, todėl jos gali būti efektyviai naudojamos vertinant kodo kokybę pasitelkiant didelius kalbos modelius.

2.6. Skyriaus apibendrinimas ir išvados

Atsižvelgiant į atliktą literatūros analizę, sudarytą metrikų klasifikaciją ir nustatytus pasirinkimo kriterijus, buvo atrinktos kodo kokybės metrikos, kurios geriausiai dera su didelių kalbos modelių gebėjimais. Šios metrikos yra statinės, lokaliai interpretuojamos, aiškiai suprantamos ir gali būti apskaičiuojamos naudojantis automatizuotais analizės įrankiais.

Iš sudėtingumo metrikų kategorijos buvo išskirtos ciklomatinio sudėtingumo, kognityvinio sudėtingumo, kodo gylio ir mazgų skaičiaus metrikos. Ciklomatinis sudėtingumas įvertina loginių kelių skaičių programoje, kognityvinis sudėtingumas matuoja, kaip sudėtinga žmogui suprasti programos struktūrą, kodo gylis pamatuoja, kiek gyliai loginiai kodo blokai yra įdėti vienas į kitą, o mazgų skaičius parodo, kiek susikirtimų yra programos valdymo sraute. Šios metrikos padeda įvertinti tiek struktūrinį, tiek suprantamumo sudėtingumą.

Apimties metrikų kategorijoje išskiriamos kodo eilučių, komentarų eilučių, tuščių eilučių ir pasikartojančių eilučių skaičiaus metrikos. Šios metrikos leidžia įvertinti kodo apimtį, dokumentacijos lygį, struktūrinį aiškumą bei pasikartojimų kiekį. Jos yra lengvai interpretuojamos ir reikšmingos kodo analizuojamumui, perpanaudojamumui, modifikuojamumui ir testuojamumui.

Taip pat išskirtos dvi objektiškai orientuotos metrikos – įeinančių elementų skaičius ir išėinančių elementų skaičius, kurios padeda įvertinti metodo priklausomumą nuo išorinių ir vidinių komponentų.

Aptartos metrikos yra tinkamos vertinti kodo kokybę naudojantis dideliais kalbos modeliais ir leidžia atlikti visapusišką statinės kodo kokybės analizę.

3. Kitų autorių tyrimų analizė

Literatūros analizė ir šaltinių atranka buvo atlikta remiantis sistemingos literatūros analizės metodika, aprašyta straipsnyje „Systematic literature reviews in software engineering“ (Kitchenham et al., 2009) [KBB⁺09]. Šis metodas užtikrina struktūruotą ir pakartojamą mokslinių šaltinių paieškos, atrankos ir analizės procesą, leidžiantį identifikuoti aktualiausius ir patikimiausius tyrimus pasirinktoje srityje.

3.1. Literatūros šaltinių pasirinkimo aprašymas

Literatūros šaltinių atranka buvo atliekama sistemingai, siekiant identifikuoti mokslinius darbus, nagrinėjančius didelių kalbos modelių taikymą kodo kokybės vertinimui. Renkantis darbus literatūros analizei, pagrindinis dėmesys buvo skiriamas aktualiausiems ir temai tinkamiausiems šaltiniams. Siekta rasti nuo 4 iki 6 straipsnių, kurie ne tik leistų įvertinti jau esamus tyrimus apie didelių kalbos modelių pritaikymą kodo kokybės vertinimui, bet ir padėtų identifikuoti potencialias spragas ar aspektus, kurie dar nebuvo nuodugniai nagrinėti šios temos kontekste. Taip pat detalai buvo įvertintos pasirinktų darbų metodologijos ir gauti rezultatai siekiant suformuoti išsamų analizuojamų tyrimų rinkinį.

Pirminė paieška buvo vykdoma naudojant *Google Scholar* mokslinių straipsnių duomenų bazę. Tam buvo sudaryta specifinė paieškos užklausa, apibrėžianti paieškos lauką:

- **(“evaluat*” OR “assess*”) AND “code quality” AND (“large language model*” OR “LLM*”) AND “code metrics”**

Paieškos rezultatai buvo apriboti pagal publikavimo metus nuo 2023 iki 2024, siekiant identifikuoti tik naujausius tyrimus, dėl šios srities spartaus vystymosi. Po pirminės paieškos buvo gauti 95 straipsniai. Visi straipsniai buvo kruopščiai tikrinami, siekiant pašalinti pasikartojančius darbus ir patikrinti, ar straipsnių tema yra tiesiogiai susijusi su didelių kalbos modelių taikymu kodo kokybės vertinimui. Po rezultatų peržiūros buvo atrinkti 2 straipsniai, kurie geriausiai atitiko tyrimo tikslus [SV24], [DST24]. Likę 93 straipsniai buvo atmesti dėl pasikartojimo arba neatitikimo tyrimo temai. Dauguma rastų straipsnių buvo orientuoti ne į kodo kokybės vertinimą, o į didelių kalbos modelių sugeneruoto kodo kokybės analizę, todėl jie neatitiko tyrimo krypties. Atlikus detalią atrinktų darbų analizę, buvo rastas dar vienas aktualus tyrimui straipsnis, kuris nepateko į pirminę paiešką [MAH⁺23].

Kadangi rastų straipsnių skaičius, išliko nepakankamas, buvo nuspręsta išplėsti paieškos lauką įtraukiant kodo pažeidžiamumo vertinimo temą. Šiam tikslui buvo sudaryta nauja paieškos užklausa:

- **“detect” AND “security vulnerabilities in code” AND (“large language model*” OR “LLM*”) AND “metrics”**

Antroji užklausa buvo skirta rasti straipsnių apie didelių kalbos modelių pritaikymą kodo saugumo vertinimo srityje, kadangi kodo saugumas yra glaudžiai susijęs su kodo kokybe. Naujos paieškos

metu buvo gauti 17 straipsnių. Po peržiūros buvo atrinktas dar vienas straipsnis, atitinkantis darbo temą [KDL⁺23].

3.2. Literatūros šaltinių analizė ir rezultatai

Atlikus mokslinių straipsnių paiešką pagal 3.1 skyriuje aprašytą metodiką, buvo peržiūrėti 112 straipsnių, iš kurių literatūros analizei atrinkti 4 straipsniai. Šie straipsniai buvo analizuoti remiantis apibrėžtais kriterijais. Buvo išskirtas kiekvieno straipsnio pavadinimas, jį parengę autoriai, publikacijos data, tyrimo tikslas, tyrimui naudoti didelės kalbos modeliai bei jų versijos. Taip pat buvo nustatyta, kokie kodo kokybės kriterijai buvo matuojami, kokie duomenų rinkiniai naudoti tyrimui atlikti, pažingsniui aprašytas tyrime taikytas metodas ir jo rezultatai.

4 lentelė. Didelių kalbos modelių taikymo kodo kokybės vertinimui tyrimų palyginimas

Pavadinimas	Autoriai	Metai	Tyrimo tikslas	Naudoti modeliai	Kodo kokybės matavimai	Duomenų šaltiniai	Tyrimo metodas	Rezultatai
Evaluating Source Code Quality with Large Language Models: A Comparative Study	Igor Regis da Silva Simões, Elaine Venson	2024	Ivertinti didelių kalbos modelių pritaikymo, kaip statinių kodo kokybės vertinimo įrankių, efektyvumą	GPT-3.5 Turbo, GPT-4o	Skaitomumas, bendra kokybė	„Quarkus“, „Shattered Pixel Dungeon“ projektai (1,641 Java klasės)	- Atrinkti 2 Java projektai turintys skirtingus SonarCloud kodo palaikomumo įvertinimus: Quarkus (A) ir Shattered Pixel Dungeon (B). - Atrinktos 1 641 Java klasės, atitinkančios SonarQube ir pasirinktų DKM apribojimus. - Pasirinktos kodo metrikos kodo analizei su SonarQube. - Sukurtos be pavyzdžių DKM užklauskos, kodo skaitomumo ir bendros kokybės įvertinimui skalėje 0-100. - Kodas įvertintas pagal pasirinktas metrikas naudojantis SonarQube. - Kodas įvertinamas naudojant DKM. - SonarQube ir DKM rezultatų palyginimas ir analizė.	- ChatGPT-3.5 Turbo kodo vertinimo rezultatai koreliuoja su SonarQube rezultatais. - GPT-4o rezultatai skirėsi nuo GPT-3.5 Turbo ir dažniau pervertindavo prasto kodo kokybę. - SonarQube įvertino kodo kokybę išsamiau ir pagal daugiau aspektų. - DKM įvertino kodo kokybę pagal aspektus, tokius kaip kodo logika, skaitomumas, aiškumas, kurių negali įvertinti SonarQube.
Evaluation of Large Language Models for Code Maintainability	Marc Dillmann, Julien Siebert, Adam Trendowicz	2024	Ištirti didelių kalbos modelių pritaikymą kodo palaikomumui vertinti, naudojant kryžminę entropiją	CodeGPT, CodeGen, CodeLlama, Bloomz	Skaitomumas, suprantamumas, sudėtingumas, moduliškumas, palaikomumas	„SWQD“ duomenų rinkinys (304 Java klasės)	- Pasirenkamas duomenų rinkinys turintis 304 Java klases su ekspertų įvertintimais Likerto skalėje pagal skaitomumą, suprantamumą, sudėtingumą, moduliškumą, palaikomumą. - Atrenkami 10 DKM. - Kodas apdorojamas pagal DKM maksimalaus įvesties dydžio ribas. - Apskaičiuojama kiekvieno DKM kryžminė entropija ir loginių kodo eilučių skaičius kiekvienai Java klasei. - Atliekami apskaičiuotos kryžminės entropijos ir ekspertų įvertinimų palyginimai ir analizė.	- Kontroliuojant kodo ilgį, mažesnė kryžminė entropija koreliuoja su didesniu palaikomumu. - Nekontroliuojant kodo ilgio, ryšys tarp kryžminės entropijos ir palaikomumo nėra nuoseklus. - Modelių parametų skaičius neturi reikšmingos įtakos kodo palaikomumo vertinimui.
Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities	Avishree Khare, Saikat Dutta, Ziyang Li, Alaia Solko-Breslin, Rajeev Alur, Mayur Naik	2024	Ivertinti didelių kalbos modelių gebėjimą aptikti saugumo pažeidimus programiniame kode	GPT-4, GPT-3.5, CodeLlama, Llama-3.1, Gemini-1.5, Qwen-2.5, DSCoder	Kodo pažeidžiamumai pagal MITRE (Top 25 Most Dangerous Software Weaknesses)	„OWASP“, „SARD Juliet (C/C++)“, „SARD Juliet (Java)“, „CVEFixes (C/C++)“, „CVEFixes (Java)“ (5000 kodo fragmentų C/C++ ir Java kalbomis)	- Pasirenkami 5 duomenų rinkiniai su nepažeidžiamu ir pažeidžiamu kodu. - Atrenkami kodo fragmentai su pažeidžiamumo problemomis. - Pasirenkamos pažeidžiamumų aptikimo metrikos: accuracy, precision, recall, F1 score. - Pasirenkami 7 skirtingi DKM. - Sukuriamos 5 skirtingų tipų DKM užklauskos: be pavyzdžių, su dviem pavyzdžiais, specifikuojančios konkretų pažeidžiamumą, vertinimo pažingsniui, paremtos minčių grandinės metodu. - Kodas įvertinamas naudojant DKM ir CodeQL. - DKM rezultatai analizuojami pagal pasirinktas metrikas ir palyginami su CodeQL vertinimu.	- Vidutinis DKM pažeidžiamumų aptikimo tikslumas buvo 62,8%, o F1 rodiklis – 0,71. - CodeQL dažniausiai pasiekė geresnį aptikimo tikslumą, tačiau DKM lenkė CodeQL specifinių pažeidžiamumų, tokių kaip Path Traversal, OS Command Injection, NULL Pointer Dereference, Use After Free, aptikime. - DKM tikslumas, taikant užklauskas, specifikuojančias kodo vertinimą pažingsniui, pagerėjo 18% lyginant su kitais metodais.
SkipAnalyzer: A Tool for Static Code Analysis with Large Language Models	Mohammad Mahdi Mohajer, Reem Aleithan, Nima Shiri Harzevili, Moshi Wei, Alvine Bo-aye Belle, Hung Viet Pham, Song Wang	2023	Ivertinti didelių kalbos modelių veikimo principais paremtos statinės kodo analizės įrankio „SkipAnalyzer“ efektyvumą aptinkant kodo klaidas ir neteisingus įspėjimus	GPT-3.5 Turbo, GPT-4 (Integruoti į SkipAnalyzer)	Kodo klaidų kiekis (Null Dereference, Resource Leak)	10 atvirojo kodo Java projektų (beveik 2 mln. Kodo eilučių)	- Pasirenkami 10 atvirojo kodo Java projektų. - Statiniu kodo klaidų analizės įrankiu Infer išanalizuoti pasirinkti Java projektai ir gauti 222 Null Dereference ir 46 Resource Leak įspėjimai. - Rankiniu būdu patikrinti ir pažymėti Infer sugeneruoti įspėjimai kaip teisingi arba klaidingi. - Pasirenkamos klaidų aptikimo metrikos: accuracy, precision, recall, F1 score. - Pasirinkti 2 DKM. - Sukuriamos 3 skirtingų tipų DKM užklauskos: be pavyzdžių, su vienu pavyzdžiu ir su keliais pavyzdžiais. - Kodas įvertinamas naudojant SkipAnalyzer. - Analizuojami SkipAnalyzer rezultatai.	- SkipAnalyzer Null Dereference kodo klaidų aptikimo tikslumas buvo 68,37%, 12,86% geriau nei Infer. - SkipAnalyzer Resource Leak kodo klaidų aptikimo tikslumas buvo 76,95%, 43,13% geriau nei Infer.

Surinkta informacija buvo susisteminta ir pateikta 4 lentelėje, kuri išryškina pagrindines analizuotų tyrimų tendencijas ir skirtumus.

Visų tyrimų išsikelti tikslai skyrėsi vienas nuo kito: du darbai orientavosi į kodo palaikomumo ir skaitomumo vertinimą naudojant didelius kalbos modelius, o kiti du analizavo kodo klaidų ir pažeidžiamumų vertinimą su DKM.

Visuose tyrimuose buvo vertinamas bent vienas GPT šeimos modelis. Trys iš keturių darbų analizavo GPT-3.5 ir GPT-4 versijas. Pastebėta, kad modelių rezultatai skyrėsi: naujesni modeliai, kaip GPT-4, kai kuriais aspektais vertino kodo kokybę net prasčiau nei GPT-3.5. Be to, nustatyta, kad modelių parametrų skaičius neturėjo reikšmingos įtakos kodo palaikomumo vertinimui.

Rezultatų palyginimas parodė, kad tradiciniai įrankiai, kaip SonarQube, vertino kodo kokybę platesniu aspektų spektru nei DKM. Tačiau DKM pasižymėjo gebėjimu analizuoti tokias savybes kaip kodo logika, aiškumas ir skaitomumas, kurių SonarQube neapima. Pažeidžiamumų aptikimo srityje CodeQL dažniausiai pasiekė didesnę tikslumą nei DKM, tačiau DKM buvo pranašesni tam tikrų specifinių pažeidžiamumų, tokių kaip Path Traversal ir OS Command Injection, aptikime.

Visų tyrimų vertinamas kodas buvo surinktas iš skirtingų duomenų rinkinių. Visi keturi tyrimai analizavo Java kalbos kodą, o viename darbe papildomai buvo įtraukti C/C++ kalbos kodo fragmentai.

Tyrimuose naudoti užklausų formavimo metodai taip pat buvo įvairūs. Naudoti tiek paprasti užklausų tipai (be pavyzdžių), tiek pažangesni metodai, kaip minčių grandinės metodas. Viename tyrime šis metodas pagerino modelio tikslumą aptinkant kodo pažeidžiamumus iki 18% lyginant su kitais metodais.

Tyrimai taip pat pabrėžė tam tikrus iššūkius, su kuriais susiduriama taikant DKM kodo kokybės vertinime. Dėl DKM įvesties dydžio apribojimų buvo reikalingas papildomas kodo duomenų apdorojimas ir paruošimas prieš vertinimą — kodas turėjo būti išskaidomas arba trumpinamas.

4. Eksperimentinis tyrimas ir rezultatai

Šiame skyriuje aprašomas praktinis eksperimentas, kuriuo siekta įvertinti didelių kalbos modelių gebėjimą analizuoti C# programavimo kalbos kodo kokybę, metodo lygmenyje. Eksperimentas apėmė duomenų rinkinio sudarymą, analizės priemonių paruošimą, kodo kokybės metrikų vertinimą naudojantis statinės analizės įrankiais, dideliais kalbos modeliais ir ekspertų žiniomis, bei gautų rezultatų palyginimą ir analizę. Visi eksperimento metu naudoti failai, programinis kodas ir analizės rezultatai yra pateikiami viešai prieinamoje GitHub repozitorijoje: <https://github.com/TadasBa/LLM-Experiment>.

4.1. Duomenų rinkinio sudarymas

Eksperimentui atlikti buvo sudarytas C# programavimo kalbos metodų rinkinys, skirtas analizuoti jame esančio kodo kokybei. Šiame poskyryje aprašoma, kaip buvo ieškoma metodų, pagal kokius kriterijus jie buvo atrenkami, apdorojami ir paruošiami analizei.

4.1.1. Duomenų paieška ir atrankos kriterijai

C# metodai buvo surinkti rankiniu būdu iš viešai prieinamų GitHub saugyklų, naudojant GitHub paieškos sistemą ir specialias užklausas, pateiktas 5 lentelėje.

5 lentelė. Naudotos GitHub paieškos užklausos

Metodų paieškos užklausos
Algorithm Implementation language:C#
Bioinformatics Algorithms language:C#
Queue operations language:C#
bubble sort language:C#
calculator language:C#
good code examples language:C#
insertion sort language:C#
invoice processor language:C#
matrix operations language:C#
palindrome checker language:C#
string operations language:C#
tic-tac-toe language:C#

Radus tinkamą GitHub saugyklą, buvo ieškoma atskirų metodų, kuriuos būtų galima išskirti nepriklausomai nuo visos klasės konteksto. Buvo siekiama pasirinkti metodus, kurie būtų pakankamai įvairūs savo aprašomumu, struktūra ir sudėtingumu. Kiekvienas rastas metodas buvo rankiniu būdu nukopijuojamas į bendrą JSON formato failą, kuriame be kodo taip pat buvo įrašoma tikslė nuoroda į failą iš kurio paimtas metodas, panaudota paieškos užklausa ir unikalūs metodo identifikatoriai. Kodo fragmentai duomenų rinkinyje buvo saugomi formatu pateiktu 3 paveikslėlyje. Iš viso buvo surinkta 510 C# metodų, kurie apimė tiek aukštos, tiek žemos kokybės fragmentus.

```
{
  "id": 22,
  "source": "https://github.com/iis-markua/Sort/blob/master/Program.cs",
  "code": "<C# metodo kodas>",
  "query": "bubble sort language:C#"
}
```

3 pav. Duomenų rinkinio struktūra

4.1.2. Duomenų filtravimas ir paruošimas analizei

Surinkus pradinis duomenis buvo atliekamas jų apdorojimas, filtravimas ir paruošimas analizei. Naudojantis pasirašytu Python kodu, buvo sutvarkytas JSON tipo faile esantis metodų kodas pašalinant nereikalingus simbolius ir sutraukiant kodą į vienos eilutės formatą, kad atitiktų JSON sintaksės reikalavimus. Tuomet naudojantis Python kiekvienas metodas buvo automatizuotai eksportuojamas į atskirą C# tipo failą, apgaubiant jį reikiama klasės struktūros sintakse ir papildant būtinomis bibliotekomis.

Po šio parengimo, norint užtikrinti duomenų rinkinio suderinamumą su statinės analizės įrankiu SonarQube, kuris reikalauja, kad analizuojamas kodas kompiliuotusi, buvo atliekamas metodų testavimas bandant juos sukompiliuoti testiniame .NET projekte. Metodai, kurie nesi-kompiliavo dėl išorinių priklausomybių, neatpažintų tipų naudojimo ar sintaksės klaidų, buvo pašalinti.

Po filtravimo, duomenų rinkinyje liko 310 metodų kurie buvo paruošti tolimesnei analizei.

4.2. Techninės eksperimento aplinkos ir įrankių pasiruošimas

Šiame poskyryje aprašomas įrankių ir modelių paruošimas eksperimentui. C# metodai buvo vertinami trimis skirtingais būdais: naudojant statinės analizės įrankius SonarQube ir SciTools Understand, taikant didelius kalbos modelius CodeLlama, CodeGemma ir Qwen2.5-Coder, bei vertinant keturių, skirtingą programavimo patirtį, turinčių ekspertų. Kiekvienam iš šių vertinimo būdų buvo parengta atskira analizės aplinka, užtikrinanti sklandų ir tikslų rezultatų gavimą.

4.2.1. SciTools Understand konfigūracija

SciTools Understand įrankis turi pranašumą, leidžiantį analizuoti kodą be būtinybės jį kompiliuoti, todėl gali apdoroti ir neišbaigtus, klaidų turinčius ar klasėmis neapgaubtus metodus. Nepaisant šio privalumo, siekiant rezultatus sulyginti su SonarQube įrankiu bei užtikrinti vienodą visų metodų apdorojimą, buvo analizuojami tie patys 310 metodų, kurių kiekvienas buvo įtrauktas į atskirą klasę ir pateiktas CS formato failuose.

Kadangi SciTools Understand yra komercinis produktas, jo standartinė kaina svyruoja nuo 100 iki 120 JAV dolerių per mėnesį, taikant minimalų 12 mėnesių naudojimo laikotarpį. Visgi, įrankio kūrėjai palaiko akademinį bendradarbiavimą ir studentams bei dėstytojams akademiniams tikslams suteikia nemokamas licencijas. Pasinaudojus šia galimybe, buvo gauta 30 dienų edukacinė licencija, kurią įrankio komanda suteikė operatyviai, greitai reaguodama į pateiktą paraišką.

Analizės vykdymui buvo sukurtas naujas projektas, kuriame visi metodai buvo įkelti kaip atskiri failai. Nustatymuose buvo pasirinkta programavimo kalba (C#) ir nurodyti papildomi

projekto parametrai. Kodas buvo analizuojamas automatiškai vos tik sukūrus projektą, be būtinybės paleisti papildomą analizės procesą. Iš visų pateikiamų metrikų tolimesnei analizei buvo pasirinktos šios aštuonios: cikloatinis sudėtingumas, kodo gylis, kodo mazgų skaičius, kodo eilučių skaičius, komentarų skaičius, tuščių eilučių skaičius, įeinančių elementų skaičius ir išeinančių elementų skaičius. Šių metrikų įvertinimai buvo eksportuoti csv formatu, taip parengiant juos tolimesnei analizei ir palyginimui.

4.2.2. SonarQube konfigūracija

Šiame eksperimente SonarQube buvo naudojamas C# metodų kokybės vertinimui pagal penkias pasirinktas metrikas: cikloatinį sudėtingumą, kognityvinį sudėtingumą, kodo eilučių skaičių, komentarų eilučių skaičių ir pasikartojančių eilučių skaičių.

Analizės aplinka buvo sukurta naudojant Docker konteinerį, kuriame veikė SonarQube serveris. Norint atlikti analizę, buvo būtina sukurti naują projektą SonarQube sąsajoje ir sugeneruoti autentifikacijos raktą. Projekto analizė buvo vykdoma naudojant *SonarScanner for .NET*, kuris reikalauja, kad projektas būtų sukompiliuotas naudojant MSBuild įrankį.

Pagrindinis analizės procesas susidėjo iš trijų žingsnių:

1. Paleidžiamas SonarScanner, nurodant projekto pavadinimą, serverio URL ir autentifikacijos raktą.
2. Projektas sukompiliuojamas naudojant MSBuild ir sugeneruojami analizės duomenys.
3. Užbaigiama analizė ir duomenys siunčiami į SonarQube serverį.

Po sėkmingos analizės, rezultatai buvo pasiekiami per SonarQube internetinę sąsają, veikiančią numatytuoju adresu *localhost:9000*. Norint toliau apdoroti gautas metrikas, buvo parašytas Python kodas, kuris JSON formatu pateiktas metrikų reikšmės iškseportavo į Excel failą, skirtą tolimesnei analizei ir duomenų lyginimui.

4.2.3. Užklauso sudarymas

Siekiant atlikti nuoseklų C# metodų vertinimą su dideliais kalbos modeliais, buvo sukonstruotas specialus užklauso šablonas. Jis buvo sukurtas tiksliai aprašant modelio užduotį ir apimė vieną tikimosi rezultato pavyzdį, todėl gali būti priskirtas prie vienu pavyzdžiu paremtų užklauso tipo. Tokia struktūra leido modeliui tiksliai suprasti, ko iš jo tikimasi, tuo pačiu apribojant interpretacinių klaidų tikimybę.

Užklausoje modelis yra apibrėžiamas kaip patyręs C# programuotojas ir kodo kokybės ekspertas. Jam pateikiamas vienas C# metodo fragmentas ir nurodoma įvertinti jį pagal dvi metrikų grupes: statines ir subjektyvias. Kiekvienos metrikos aprašas pateikiamas tiesiogiai užklausoje, taip sumažinant riziką, kad modelis neteisingai interpretuos metrikos pavadinimą.

Statinių metrikų grupę sudarė dešimt skirtingų metrikų: cikloatinis sudėtingumas, kognityvinis sudėtingumas, kodo gylis, kodo mazgų skaičius, kodo eilučių skaičius, komentarų skaičius, tuščių eilučių skaičius, pasikartojančių eilučių skaičius, įeinančių elementų skaičius ir išeinančių

elementų skaičiu. Kiekvienos metrikos įvertinimą modelis turėjo pateikti skaitinėmis reišmėmis atsižvelgiant į užklausoje pateikto metodo struktūrą.

Subjektyviosios metrikos buvo: perpanaudojamumas, modifikuojamumas, analizuojamumas ir testuojamumas. Šios metrikos buvo paremtos ISO/IEC 25010:2023 kokybės modeliu. Kiekvieną iš jų modelis turėjo įvertinti nuo 1 iki 5 bei pagrįsti įvertinimą trumpu komentaru. Tokia struktūra buvo pasirinkta siekiant ne tik kiekybiškai įvertinti subjektyvias metrikas, bet ir gauti išsamesnį kodo kokybės apibūdinimą.

Atsakymo formatas buvo griežtai apibrėžtas – modelis turėjo grąžinti sintaksiškai teisingą JSON objektą. Modeliui taip pat buvo nurodyta neįtraukti jokio papildomo teksto ar paaiškinimų už JSON ribų. Tai buvo būtina sąlyga, leidžianti po analizės automatizuotai apdoroti visus atsakymus be papildomos interpretacijos ar korekcijų. Užklaustos pabaigoje taip pat pateiktas pavyzdinis atsakymas, padedantis modeliui geriau suprasti, kaip turi atrodyti rezultatas.

4.2.4. Didelių kalbos modelių paruošimas

Didelių kalbos modelių taikymui šiame eksperimente buvo sukurta speciali analizės aplinka, paremta debesijos infrastruktūra. Kodo kokybės vertinimas buvo atliekamas naudojant tris atvirojo kodo didelių kalbos modelius: 7 milijardų parametrų CodeLlama Instruct modelio versiją, 7 milijardų parametrų CodeGemma Instruct modelio versiją ir 7 milijardų parametrų Qwen2.5-Coder modelio versiją. Visi šie modeliai yra pritaikyti darbui su programiniu kodu, palaiko instrukcijomis paremtą bendravimą ir yra optimizuoti atsakymams, susijusiems su kodo analize bei generavimu.

Kiekvienas modelis buvo vertinamas du kartus: vieną iteraciją su numatytaisiais modelio paleidimo parametrais, ir vieną iteraciją su pakoreguotais parametrais (temperature = 0.1, top_k = 20, top_p = 0.5). Taip iš viso buvo atliktos šešios vertinimo sesijos po 310 metodų. Visų bandymų metu buvo naudojamas identiškas užklaustos šablonas, taip užtikrinant rezultatų palyginamumą ir eksperimento nuoseklumą.

Modelių paleidimo parametrai tiesiogiai veikia tai, kaip modelis generuoja atsakymus. Eksperimento metu buvo keisti trys pagrindiniai parametrai:

- Temperature (0.1) — reguliuoja atsakymų „kūrybiškumą“. Kuo ši reikšmė mažesnė, tuo labiau modelis linkęs rinktis saugiausią ir tiksliausią atsakymą. 0.1 reikšmė reiškia labai konservatyvų modelio elgesį – jis stengiasi pateikti kuo labiau nuspėjamą ir nuoseklų rezultatą.
- Top-k (20) — riboja modelio pasirinkimų kiekį kiekviename žingsnyje. Vietoj to, kad rinktųsi iš visų galimų išvesčių, modelis sprendimą priima tik iš 20 dažniausiai tikėtinų variantų. Tai padeda išvengti atsitiktinių ar netikslių atsakymų pasirinkimo.
- Top-p (0.5) — veikia kaip papildomas filtras. Modelis renkasi atsakymą iš mažiausio žodžių rinkinio, kurio bendra tikimybė sudaro bent 50%. Tai leidžia modeliams sutelkti dėmesį į tik labiausiai tikėtinius variantus, ignoruojant mažiau prasmingus.

Šių parametrų derinys užtikrina, kad modelio atsakymai būtų kuo labiau stabilūs, logiški ir nuoseklūs.

Modelių paleidimui buvo sukurta virtuali mašina naudojant Google Cloud Console paslaugą.

Pasirinktas mašinos tipas – g2-standard-32, turintis 32 virtualius procesorius (vCPU), 128 GB operatyviosios atminties ir vieną NVIDIA L4 grafinę plokštę. Ši virtuali mašina veikė x86/64 architektūros pagrindu ir buvo skirta dirbtinio intelekto modelių paleidimui, užtikrinant pakankamą skaičiavimo resursų pajėgumą atliekant eksperimentą.

Po mašinos paleidimo buvo įdiegta Ollama programinė įranga, leidžianti lokaliai paleisti didelius kalbos modelius per komandinės eilutės sąsają. Kiekvieno modelio vertinimui buvo naudojamas Python programinis kodas, kuris įkeldavo C# metodus iš JSON formato, pritaikydavo jiems paruoštą užklauso šabloną ir siųsdavo į pasirinktą modelį per Ollama API.

Python kodas siųsdavo pasirinktam didelės kalbos modeliui po vieną užklausa ir modeliui sugeneravus atsakymą išsaugodavo jį masyve struktūrizuotu JSON formatu. Surinkus visus modelio atsakymus, duomenys buvo apdorojami naudojant Pandas biblioteką ir eksportuojami į Excel formato failus tolimesnei analizei.

4.2.5. Ekspertų profilis ir įtraukimas

Subjektyvioms kodo kokybės metrikoms įvertinti buvo pasirinkti keturi programuotojai, turintys skirtingą profesinę patirtį. Prieš atlikdami vertinimus, visi ekspertai užpildė anketą, kurioje pateikė informaciją apie savo dabartines pareigas, pagrindinę veiklos sritį, bendrą darbo patirtį su programavimu, patirtį dirbant su C# kalba ir patirtį vertinant kitų programuotojų kodą.

Remiantis surinktais atsakymais, ekspertų grupė pasižymi įvairiapusiška, bet su eksperimentu aiškiai susijusia patirtimi:

- Visi keturi ekspertai patvirtino turintys realios patirties vertinant arba peržiūrint kitų programuotojų kodą.
- Du ekspertai turėjo daugiau nei 10 metų bendros programavimo patirties, o kiti du – nuo 3 iki 6 metų.
- Visi keturi ekspertai nurodė turintys nuo 1 iki daugiau nei 10 metų patirties dirbant su C# kalba, todėl yra susipažinę su šios kalbos ypatumais.
- Ekspertų pareigos apima platų spektrą – nuo IT analitiko iki programuotojo, o pagrindinės veiklos sritys: programinės įrangos kūrimas, moksliniai tyrimai bei IT infrastruktūros administravimas.

Atsižvelgiant į jų darbo stažą, techninį pasirengimą ir pareigas, ekspertai buvo suskirstyti į dvi kategorijas: **vyresnieji ekspertai** (2 dalyviai), turintys daugiau nei 10 metų programavimo patirties ir aukštesnes pareigas, bei **jaunesnieji ekspertai** (2 dalyviai), kurių patirtis siekia iki 6 metų, o dabartinės pareigos labiau susijusios su jaunesnių ar vidutinių programuotojų veikla. Toks ekspertų pasiskirstymas užtikrina skirtingų požiūrių įtraukimą vertinant subjektyvius kodo kokybės kriterijus ir sumažina galimą vertinimų šališkumą.

Ekspertams buvo pateikta atskira „Google Forms“ vertinimo forma, kurioje jie turėjo įvertinti 10 skirtingų C# metodų. Kiekvienas metodas buvo pateiktas kaip pilnas kodo fragmentas, o ekspertai turėjo įvertinti jį pagal keturis subjektyvius ISO/IEC 25010:2023 pagrindu paremtus kriterijus: perpanaudojamumą, modifikuojamumą, analizuojamumą ir testuojamumą. Kiekvienas

kriterijus turėjo būti įvertintas balu nuo 1 iki 5 ir papildytas trumpu komentaru, pagrindžiančiu pasirinkimą.

Surinkti vertinimai buvo eksportuoti į Excel formato failą tolimesnei analizei ir palyginimui su didelių kalbos modelių pateiktais subjektyviais atitinkamų metodų vertinimais.

4.3. Statinių kodo kokybės metrikų vertinimo rezultatai ir analizė

4.3.1. Vertinimo rezultatų struktūra ir apdorojimas

Statinių kodo kokybės metrikų analizės rezultatai buvo apdorojami pasitelkiant vieną bendrą Excel darbaknygę, kurioje buvo sujungti duomenys iš dviejų statinės analizės įrankių (SonarQube ir SciTools Understand) bei trijų didelių kalbos modelių – Qwen2.5-Coder, CodeGemma ir CodeLlama – kiekvienas iš jų papildomai turėjo po vieną versiją su pakeistais parametrais (Qwen2.5-CoderP, CodeGemmaP, CodeLlamaP).

Duomenys buvo suskirstyti pagal dešimt pasirinktų statinių metrikų: ciklopatinis sudėtingumas, kognityvinis sudėtingumas, kodo gylis, kodo mazgų skaičius, kodo eilučių skaičius, komentarų skaičius, tuščių eilučių skaičius, pasikartojančių eilučių skaičius, įeinančių elementų skaičius ir išeinančių elementų skaičius. Kiekvienai metrikai buvo pateikti visi šešių DKM vertinimai bei vieno ar dviejų analizės įrankių rezultatai, priklausomai nuo to, kuris įrankis palaikė konkrečią metriką. Tokiu būdu įrankiai ne tik atkartoją vienas kito funkcionalumą, bet ir papildė bendrą vertinimo spektrą.

Kai kuriais atvejais statinių analizės įrankių rezultatai nesutapo dėl nevienodų jų skaičiavimo taisyklių ir apibrėžimų. Ciklopatiniam sudėtingumui SciTools Understand ir SonarQube pateikė skirtingas reikšmes, kadangi jie naudoja skirtingus skaičiavimo metodus. Scitools Understand remiasi klasikine NIST formulės interpretacija, pagal kurią kiekvienas valdymo sprendimo taškas padidina sudėtingumą vienetu. Tuo tarpu SonarQube skaičiavimo metodiką aprašo oficiali dokumentacija, kurioje yra pateiktos aiškos formulės ir taikymo scenarijai [Sonnd]. Kadangi SonarQube metodika tiksliau atitinka McCabe ciklopatinio sudėtingumo apibrėžimą, SonarQube įrankio įvertinimai buvo pasirinkti kaip pagrindinis šaltinis šiai metrikai lyginti.

Kodo eilučių skaičiaus metrika SonarQube buvo skaičiuojama susumuojant visas eilutes, įskaitant eilutes už metodo apibrėžimo ir komentarus, tuo tarpu SciTools fiksavo tik pačiame metode esančias eilutes neįskaitant komentarų ir tuščių eilučių. Kadangi DKM užklausoje buvo nurodyta vertinti tik patį metodo turinį, SciTools rezultatai buvo laikomi šios metrikos etalonu.

Komentarų eilučių skaičiaus metrikos atveju SonarQube įskaitė ir komentarus virš metodo deklaracijos, o SciTools Understand skaičiavo tik pačiame metode esančias komentarų eilutes. Kadangi dideliems kalbos modeliams buvo pateikiami metodai su komentarais virš metodo apibrėžimo, SonarQube interpretacija buvo tinkamesnė ir todėl šio įrankio reikšmės buvo pasirinktos kaip palyginimo pagrindas šiai metrikai.

Kitos metrikos – kognityvinis sudėtingumas, kodo gylis, kodo mazgų skaičius, tuščių eilučių skaičius, pasikartojančių eilučių skaičius, įeinančių elementų skaičius ir išeinančių elementų skaičius buvo pateiktos tik vieno iš įrankių, todėl jos automatiškai buvo laikomos etaloningomis palyginimui su DKM vertinimu.

4.3.2. Duomenų paruošimas statistinei analizei

Išsamesniam ir gilesniam įvertinimų palyginimui tarp DKM ir statinės analizės įrankių buvo taikoma klaidų klasifikacija. Kiekvienai metrikai buvo pritaikytas sąlyginis Excel formatavimas, kuris leido vizualiai atskirti vertinimų tikslumo lygius:

- Žalia spalva – DKM reikšmė tiksliai atitinka lyginamo įrankio vertę;
- Geltona spalva – nuokrypis lygus ± 1 ;
- Raudona spalva – nuokrypis lygus ± 2 .

Tokiu būdu buvo galima ne tik fiksuoti, kada modeliai pateikė tikslų rezultatą, bet ir įvertinti kaip arti jie buvo prie teisingos reikšmės, net jeigu suskaičiavo neteisingai. Šis sprendimas leido taikyti tolerantiškesnius tikslumo vertinimus, svarbius analizuojant DKM gebėjimą apytiksliai numatyti metrikas. Be to, žymėjimas spalvomis padėjo išryškinti įvairias tendencijas ir anomalijas.

Taip pat buvo parengta papildoma lentelė, kurioje spalviniai skirtumai buvo konvertuoti į skaitines vertes. Tam naudota tokia formulė:

$$f(a, b) = \begin{cases} 0, & a = b \\ 1, & |a - b| = 1 \\ 2, & |a - b| = 2 \\ x, & |a - b| > 2 \end{cases} \quad (2)$$

kur a – modelio įvertinimas, b – statinės analizės įrankio įvertinimas.

Ši lentelė buvo parengta ir užpildyta atitinkamomis reikšmės tolimesniems statistiniams skaičiavimams ir analizei, užtikrinant galimybę kiekybiniam DKM ir statinių analizės įrankių palyginimui visų metrikų mastu.

4.3.3. Rezultatų analizė ir palyginimas

Apdorojus surinktus duomenis ir juos paruošus analizei, buvo pereita prie kiekybinės analizės ir DKM rezultatų palyginimo su statiniais įrankiais.

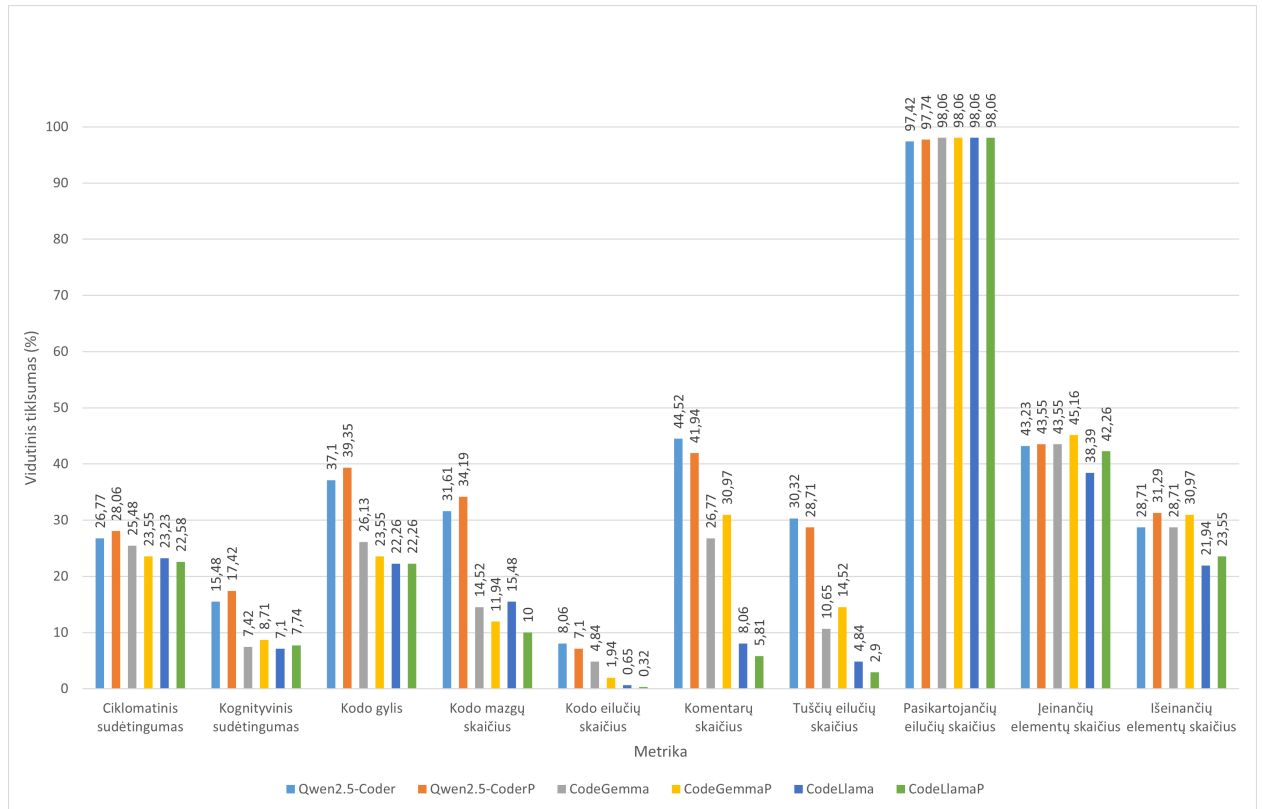
4.3.3.1. Modelių vertinimo tikslumo analizė

Analizuojant modelių vertinimo tikslumą esant skirtingoms paklaidų riboms, pastebėta tendencija, kad visų modelių tikslumas didėja augant toleruojamai paklaidai. 4 diagramoje pateikiamas vidutinis modelių vertinimo tikslumas be paklaidos, apskaičiuotas kaip teisingų modelio įvertinimų ir bendro vertintų metodų skaičiaus santykis:

$$\text{Vidutinis modelio tikslumas} = \frac{T}{N} \cdot 100\%, \quad (3)$$

kur T – teisingų įverčių skaičius, o N – visų vertintų metodų skaičius. Tuo atveju, kai taikoma paklaida (± 1 arba ± 2), rezultatas laikomas teisingu, jei modelio įvertis skiriasi nuo tikrosios reikšmės ne daugiau kaip viena arba dviem reikšmėmis atitinkamai.

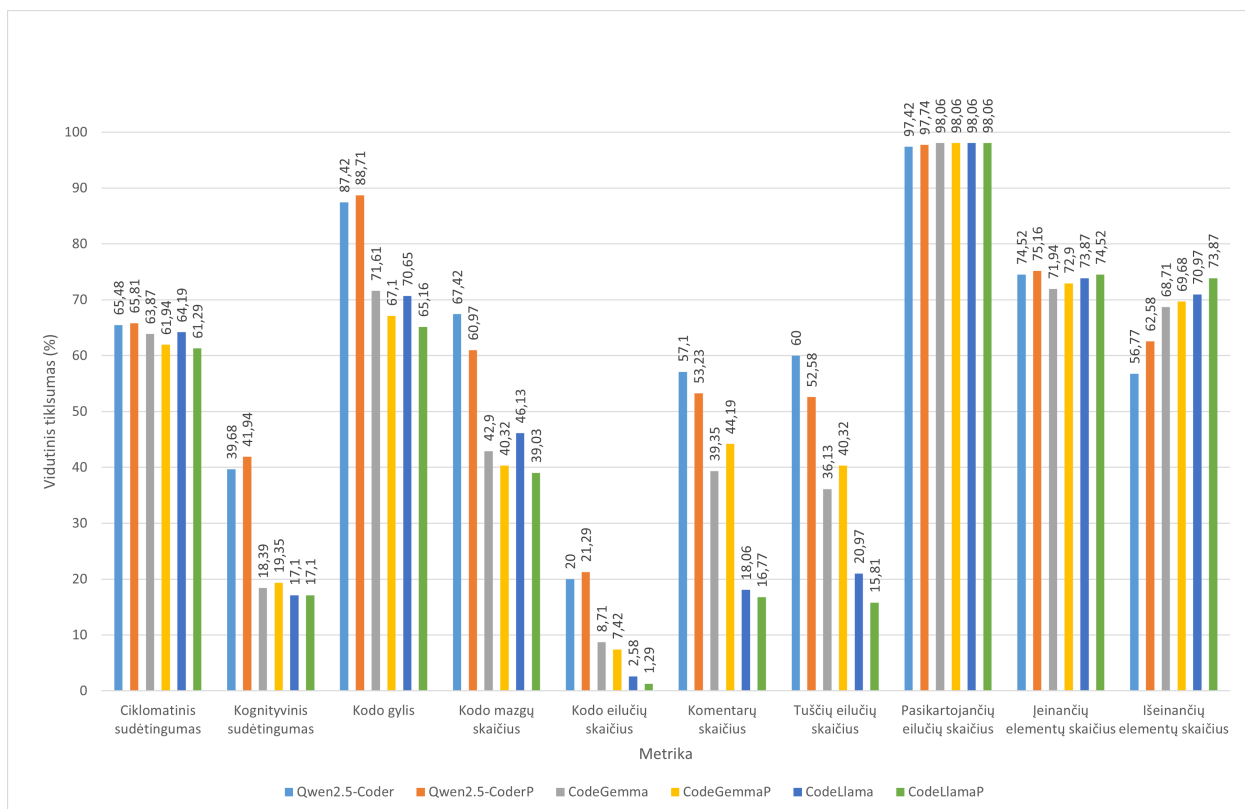
5 ir 6 diagramose atvaizduojamas modelių vertinimo tikslumas esant atitinkamai ± 1 ir ± 2 paklaidai. Qwen2.5-Coder modelio ciklomatinio sudėtingumo vertinimų tikslumas siekė tik 26,77 %, tačiau su ± 1 paklaida jis išaugo iki 65,48 %, o su ± 2 – iki 86,45 %. Toks pat šuolis pastebimas ir kitose metrikose: kodo gylio metrikai tikslumas nuo 37,1 % padidėjo iki 87,42 % (± 1) ir 98,71 % (± 2).



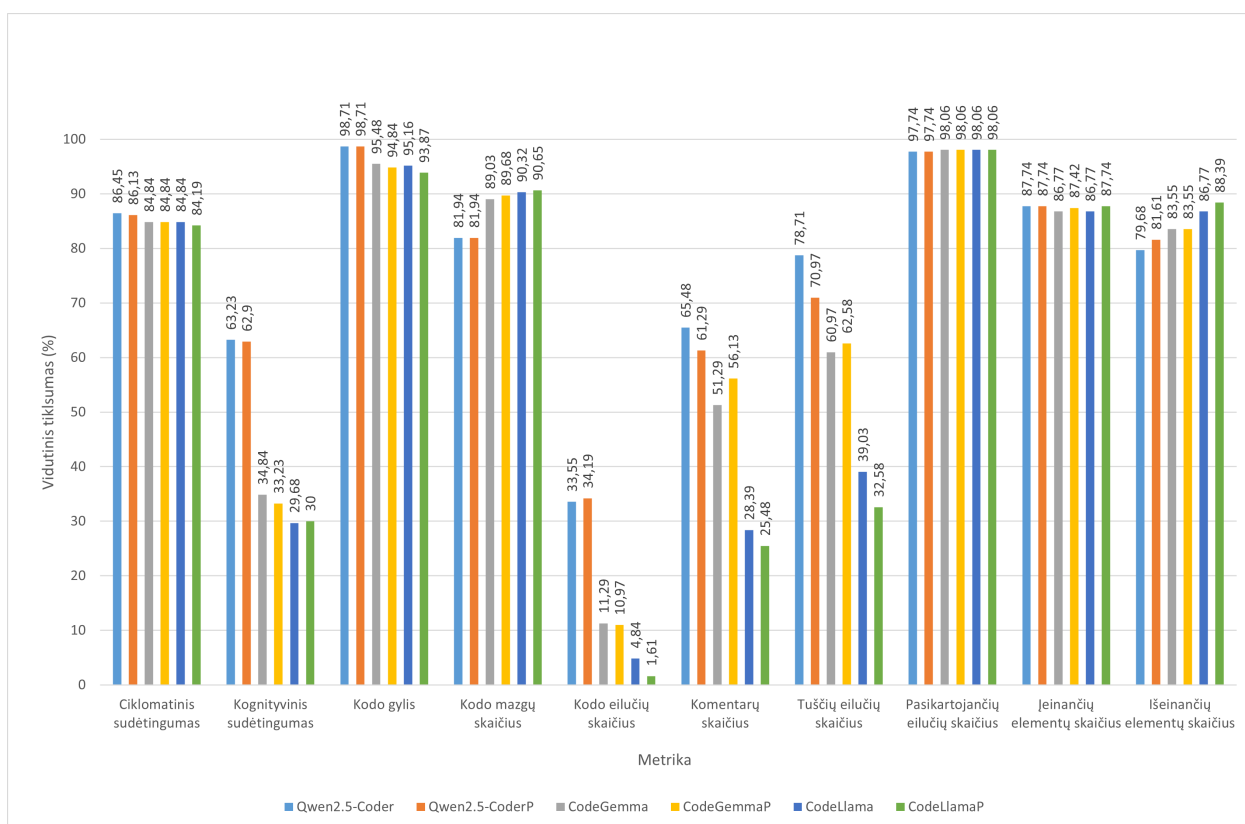
4 pav. Vidutinio modelių tikslumo palyginimas

Lyginant skirtingus modelius, aiškiai išsiskiria Qwen2.5-Coder ir jo versija su pakeistais parametrais Qwen2.5-CoderP, kurie visose metrikose ir visomis paklaidų ribomis pasiekė aukščiausius arba labai artimus aukščiausiams tikslumo rezultatus. Kodo gylio metrikai Qwen2.5-CoderP modelis pasiekė 88,71 % tikslumą su ± 1 paklaida ir 98,71 % su ± 2 . Taip pat itin aukštas tikslumas pasiektas įšeinančių elementų skaičiaus metrikai, kur su ± 2 paklaida abu Qwen2.5-Coder modeliai viršijo 87 %.

Mažiausi tikslumo rezultatai fiksuoti CodeLlama ir CodeLlamaP modeliams. Jie pasiekė tik 0,65 % ir 0,32 % tikslumą kodo eilučių skaičiaus metrikai, ir tik 8,06 % bei 5,81 % komentarų skaičiaus metrikai. Net ir taikant ± 2 paklaidą, šių metrikų tikslumas neviršijo 33 %.



5 pav. Vidutinio modelių tikslumo palyginimas esant ± 1 paklaidai



6 pav. Vidutinio modelių tikslumo palyginimas esant ± 2 paklaidai

Pastebėta, kad patobulintos modelių versijos ne visada pasižymėjo geresniais rezultatais. CodeGemmaP modelio tikslumas kodo eilučių skaičiaus metrikai buvo tik 1,94 %, kai originali

versija pasiekė 4,84 %. Kita vertus, Qwen2.5-CoderP daugelyje metrikų šiek tiek lenkė originalų modelį – pavyzdžiui, išeinančių elementų skaičiaus ± 2 tikslumas buvo 81,61 %, palyginti su 79,68 %.

Metrikos, kurių vertinimus modeliai atliko tiksliausiai, buvo įeinančių elementų skaičius, išeinančių elementų skaičius ir kodo gylis – daugumos modelių tikslumas su paklaida ± 2 šiose metrikose viršijo 80 %. Sudėtingiausiai modeliams buvo įvertinti apimties metrikas tokias kaip kodo eilučių ir komentarų eilučių skaičius, kurioms tikslumas paprastai nesiekė net 10 %.

4.3.3.2. Statistinių paklaidų ir koreliacijos analizė

Be tikslumo, taip pat buvo analizuojami modelių vertinimo paklaidų dydžiai ir koreliaciją su statinių įrankių rezultatais. Tam buvo naudojamos vidutinės absoliučios paklaidos (MAE) [WM05], vidutinio kvadratinio nuokrypio (RMSE)[WM05] bei Pearson koreliacijos [LN88] koeficientai.

MAE ir RMSE reikšmės pateiktos 6 ir 7 lentelėse ir leido įvertinti kaip stipriai modelių įvertinimai buvo nutolę nuo statinių analizės įrankių vertinimų. Mažiausios MAE ir RMSE reikšmės daugumai metrikų fiksuotos Qwen2.5-Coder modeliui ir šio modelio variantui su pakeistais parametrais Qwen2.5-CoderP. Ciklomatinio sudėtingumo metrikai Qwen2.5-CoderP MAE buvo tik 1,529, o RMSE – 3,097. Tuo tarpu CodeGemmaP ir CodeLlamaP tai pačiai metrikai pasiekė MAE 1,732 ir 1,761 bei RMSE 3,893 ir 3,908. Panašiai, kodo gylio metrikai Qwen2.5-CoderP MAE buvo tik 0,735, kai CodeLlamaP – 1,187.

6 lentelė. Modelių vidutinės absoliučiosios paklaidos reikšmės kiekvienai metrikai (MAE)

Metrika	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
Ciklomatinis sudėtingumas	1,532	1,529	1,642	1,732	1,710	1,761
Kognityvinis sudėtingumas	2,797	2,732	4,303	4,432	4,500	4,623
Kodo gylis	0,771	0,735	1,068	1,145	1,119	1,187
Kodo mazgų skaičius	1,665	1,703	1,977	2,023	1,942	2,045
Kodo eilučių skaičius	12,048	17,123	29,116	28,171	43,371	48,310
Komentarų skaičius	2,374	2,542	3,161	3,013	4,768	5,074
Tuščių eilučių skaičius	1,500	1,716	2,342	2,239	3,184	3,326
Pasikartojančių eilučių skaičius	0,923	0,974	0,955	0,955	0,955	0,955
Įeinančių elementų skaičius	1,010	1,003	1,052	1,013	1,084	1,023
Išeinančių elementų skaičius	1,584	1,429	1,365	1,332	1,361	1,284

Didžiausios paklaidos buvo fiksuotos kodo eilučių skaičiaus metrikai. Čia RMSE reikšmės siekė nuo 20,838 (Qwen2.5Coder) iki 49,558 (CodeLlamaP), o MAE – atitinkamai nuo 12,048 iki 48,31. Tai rodo, kad modelių apskaičiuoti šios metrikos įvertinimai itin stipriai skirėsi nuo statinių analizės įrankių gautų reikšmių.

Komentarų eilučių skaičiaus ir tuščių eilučių skaičiaus metrikų paklaidos taip pat buvo aukštos, ypač CodeLlama modelio vertinimuose. CodeLlamaP MAE įvertis komentarų eilučių metrikai siekė 5,074, o RMSE – 6,038.

7 lentelė. Modelių vidutinio kvadratinio nuokrypio reikšmės kiekvienai metrikai (RMSE)

Metrika	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
Ciklopatinis sudėtingumas	3,103	3,097	3,402	3,893	3,838	3,908
Kognityvinis sudėtingumas	6,268	6,165	7,067	7,394	7,266	7,505
Kodo gylis	1,046	1,016	1,348	1,418	1,378	1,459
Kodo mazgų skaičius	3,334	3,358	3,418	3,438	3,471	3,440
Kodo eilučių skaičius	20,838	25,953	36,284	35,168	46,806	49,558
Komentarų skaičius	4,023	4,247	4,638	4,657	5,766	6,038
Tuščių eilučių skaičius	2,135	2,338	2,836	2,828	3,731	3,700
Pasikartojančių eilučių skaičius	7,446	7,947	8,102	8,102	8,102	8,102
Įeinančių elementų skaičius	1,571	1,571	1,632	1,592	1,628	1,581
Išeinančių elementų skaičius	2,258	2,085	2,006	1,981	1,943	1,833

Pearson koreliacijos reikšmės pateiktos lentelėje 8 leido nustatyti kaip modelių vertinimas koreliavo su statinių analizės įrankių vertinimais. Pagal Pearson koreliacijos rezultatus, matyti, kad aukščiausi koreliaciniai ryšiai fiksuoti Qwen2.5-Coder modeliui. Kodo eilučių skaičiaus metrikai Qwen2.5-Coder koreliacija siekė net 0,805, o ciklopatinio sudėtingumo – 0,75. Tuo tarpu CodeLlamaP koreliacijos rezultatai daugelyje metrikų siekė vos 0,003–0,099, o pasikartojančių eilučių skaičiaus – apskritai buvo neįmanoma apskaičiuoti, nes visi vertinimai buvo vienodi.

Modelių koreliacijos vertės kognityvinio sudėtingumo metrikai svyravo nuo 0,003 (CodeLlamaP) iki 0,549 (Qwen2.5-CoderP), kas rodo, kad dauguma modelių nesugebėjo nuosekliai įvertinti šios metrikos. Koreliaciniai ryšiai įeinančių ir išeinančių elementų skaičiaus metrikoms taip pat buvo žemi visiems modeliams.

8 lentelė. Pearson koreliacijos koeficientai (r) tarp modelių įvertinimų ir statinės analizės įrankių vertinimų

Metrika	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
Ciklopatinis sudėtingumas	0,75	0,805	0,682	0,114	0,229	0,019
Kognityvinis sudėtingumas	0,512	0,549	0,364	0,112	0,268	0,003
Kodo gylis	0,427	0,499	0,252	0,181	0,170	0,080
Kodo mazgų skaičius	0,369	0,308	0,108	0,053	0,011	0,018
Kodo eilučių skaičius	0,805	0,722	0,633	0,422	0,123	0,099
Komentarų skaičius	0,586	0,499	0,435	0,399	0,172	0,092
Tuščių eilučių skaičius	0,547	0,539	0,385	0,362	0,076	0,079
Pasikartojančių eilučių skaičius	0,566	0,283	–	–	–	–
Įeinančių elementų skaičius	0,080	0,003	0,038	0,028	0,070	0,055
Išeinančių elementų skaičius	0,292	0,302	0,166	0,166	–0,050	0,031

4.3.3.3. Vertinimų anomalijos ir pastebėjimai

Rezultatų analizės metu buvo pastebėtas reikšmių pasikartojimo reiškinys, kuris skirtinguose modeliuose pasireiškė skirtingu intensyvumu. Ypač intensyviai šis reiškinys išsiskyrė CodeGemma ir CodeLlama modeliuose.

CodeGemma modelio atsakymuose identiška vertė buvo kartojama nuo 173 iki 310 kartų vienai metrikai. Tai reiškia, kad modelis dažnai generavo tą pačią reikšmę daugeliui skirtingų vertinamų metodų. Pakeistų parametrų versija (CodeGemmaP) pasižymėjo dar didesniu pasikartojimų dažniu – kai kurių metrikų atveju identiški atsakymai sudarė beveik visą rezultatų rinkinį.

Panašus elgesys buvo pastebėtas CodeLlama modelyje. Reikšmių pasikartojimų skaičius šiuo atveju svyravo tarp 281 ir 310. CodeLlamaP modelio atsakymai beveik visais atvejais buvo identiški – pasikartojimų kiekis siekė nuo 303 iki 310 kartų.

Tuo tarpu Qwen2.5-Coder modelį šis reiškinys paveikė mažiau. Nors reikšmių pasikartojimai taip pat buvo fiksuojami, jų dažnis buvo žymiai mažesnis nei kituose modeliuose. Pakeistų parametrų versijoje (Qwen2.5-CoderP) pasikartojimų padaugėjo, tačiau jie vis tiek liko retesni nei CodeGemma ar CodeLlama modeliuose.

Pasikartojimai buvo fiksuoti kiekvienai metrikai atskirai, skaičiuojant, kiek kartų kiekvienas modelis pateikė vienodą vertę visiems vertintiems metodams. Gauti rezultatai pateikiami 7 pav., kuris rodo tų pačių reikšmių pasikartojimų skaičių kiekviename modelyje.

Ciklo matinis sudėtingumas						Kognityvinis sudėtingumas					
Reikšmė 3 pasikartojo (kartus)						Reikšmė 8 pasikartojo (kartus)					
Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
89	124	274	296	256	303	56	119	272	296	267	303
Kodo gylis						Kodo mazgų skaičius					
Reikšmė 3 pasikartojo (kartus)						Reikšmė 2 pasikartojo (kartus)					
Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
96	135	269	296	262	304	73	120	270	296	264	304
Kodo eilučių skaičius						Komentarų skaičius					
Reikšmė 67 pasikartojo (kartus)						Reikšmė 7 pasikartojo (kartus)					
Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
44	105	173	173	256	303	42	96	174	173	260	303
Tuščių eilučių skaičius						Pasikartojančių eilučių skaičius					
Reikšmė 5 pasikartojo (kartus)						Reikšmė 0 pasikartojo (kartus)					
Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
48	101	173	173	263	303	306	308	310	310	310	310
Įeinančių elementų skaičius						Išeinančių elementų skaičius					
Reikšmė 1 pasikartojo (kartus)						Reikšmė 2 pasikartojo (kartus)					
Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP	Qwen2.5-Coder	Qwen2.5-CoderP	CodeGemma	CodeGemmaP	CodeLlama	CodeLlamaP
305	309	274	288	284	305	46	104	179	177	281	304

7 pav. Pasikartojimų skaičiaus palyginimas tarp modelių

Dar vienas svarbus pastebėjimas buvo susijęs su pasikartojančių eilučių metrika. Tik 6 iš visų 310 analizuotų metodų turėjo pasikartojančių kodo eilučių, o likusiuose 304 metoduose ši metrika buvo lygi 0. Visi modeliai pateikė neteisingą pasikartojimų skaičių visuose 6 metoduose, kuriose pasikartojimai iš tiesų egzistavo.

Qwen2.5-Coder modelis aptiko, kad pasikartojančios eilutės egzistuoja 2 iš 6 metodų, tačiau abiem atvejais pateikė neteisingą pasikartojimų skaičių. Šis modelis taip pat du kartus aptiko pasikartojimus ten kur jų nebuvo (angl. false positives). Modelio versija su pakeistais parametrais (Qwen2.5-CoderP) vieną kartą aptiko pasikartojimus metode teisingai, bet taip pat pateikė netikslių skaičių ir vieną kartą klaidingai identifikavo pasikartojimus ten, kur jų nebuvo.

Tuo tarpu CodeGemma ir CodeLlama modeliai ir jų versijos su pakeistais parametrais, visuose 310 vertintų metodų grąžino 0 ir nei karto neaptiko pasikartojimų. Šis rezultatas rodo, kad jie „teisingai“ įvertino 304 metodus be pasikartojančių eilučių, tačiau nesugebėjo aptikti nė vieno tikro atvejo, kuriame pasikartojimai buvo. Atsižvelgiant į anksčiau pastebėtą vienodų reikšmių kartojimo anomaliją šiuose modeliuose, galima manyti, kad jie tiesiog nuosekliai generavo vieną iš anksto pasirinktą atsakymą (0), nepriklausomai nuo kodo turinio pokyčių. Dėl šios priežasties, nors modelių pasikartojančių eilučių aptikimo tikslumas atrodo aukštas, jis neatspindi realių gebėjimų

atpažinti pasikartojimus. Tikslesniam vertinimui reikėtų naudoti labiau subalansuotą duomenų rinkinį, kuriame būtų daugiau metodų su pasikartojančiomis eilutėmis.

4.4. Subjektyvių kodo kokybės metrikų vertinimo rezultatai ir analizė

4.4.1. Vertinimo rezultatų struktūra ir apdorojimas

Subjektyviųjų kodo kokybės metrikų vertinimui buvo atrinkta 10 skirtingų C# metodų. Kiekvienas metodas buvo vertinamas didelių kalbos modelių ir ekspertų pagal keturis ISO/IEC 25010:2023 programinės įrangos palaikomumo kriterijus: perpanaudojamumą, modifikuojamumą, analizuojamumą ir testuojamumą. Metodai buvo vertinami penkiabalėje skalėje (1 – labai žema kokybė, 5 – labai aukšta kokybė).

Vertinimus pateikė keturi ekspertai, kurie pagal patirtį buvo suskirstyti į dvi grupes – jaunesnieji ekspertai turintys iki 6 metų darbo su programiniu kodu patirties ir vyresnieji ekspertai turintys daugiau nei 10 metų patirties. Kiekvienas metodas taip pat buvo įvertintas šešis kartus trijų didelių kalbos modelių Qwen2.5-Coder, CodeGemma, CodeLlama, kurių kiekvienas turėjo po vieną versiją su pakeistais parametrais (Qwen2.5-CoderP, CodeGemmaP, CodeLlamaP). Visiems modeliams buvo pateikiama ta pati užklausa su tokiomis pat instrukcijomis, užtikrinant vertinimo sąlygų vienodumą.

Surinkti ekspertų ir didelių kalbos modelių vertinimo duomenys buvo struktūrizuoti pateikti vienoje Excel lentelėje, kuri buvo naudojama kaip pagrindinis duomenų šaltinis tolimesniai analizei. Lentelėje kiekvienam kokybės kriterijui buvo sukurta atskira sekcija, kurioje pateikti modelių ir ekspertų įvertinimai, apskaičiuoti ekspertų grupių vertinimų vidurkiai ir standartiniai nuokrypiai. Ši struktūra leido tiksliai įvertinti DKM atitikimą ekspertų vertinimams, taikant tokius statistinius metodus kaip vidutinė absoliuti paklaida, Pearson ir Spearman koreliacijos.

4.4.2. Rezultatų analizė ir palyginimas

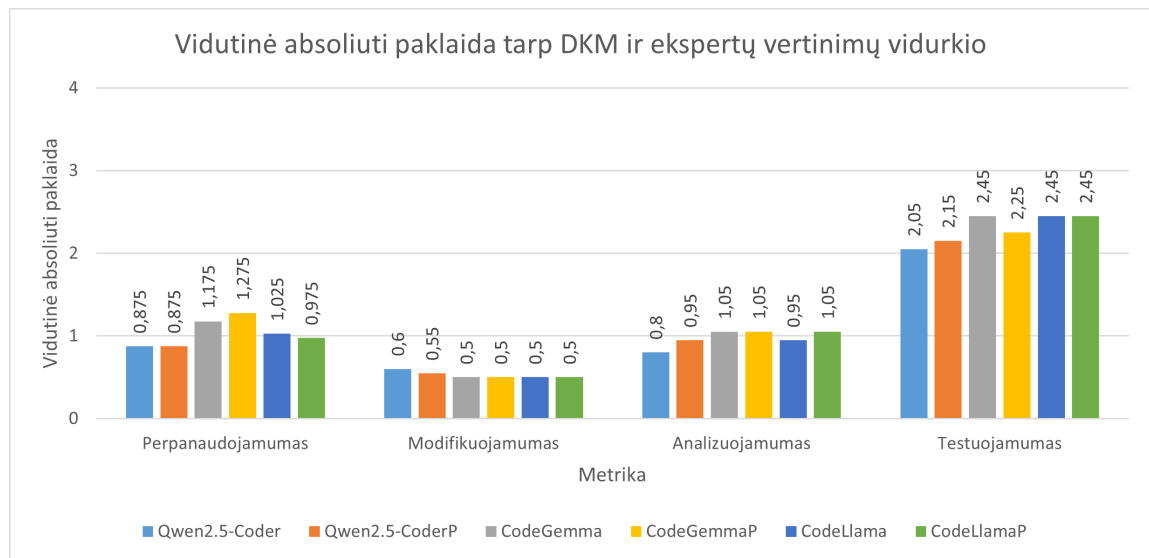
Šiame poskyryje aprašoma subjektyvių kodo kokybės metrikų vertinimo rezultatų analizė, įvertinant didelių kalbos modelių atitikimą ekspertų vertinimams. Analizė apima modelių paklaidų įvertinimą (MAE) bei modelių vertinimų koreliacijas su ekspertų vertinimais. Atskirai analizuojami kiekvienas iš keturių ISO/IEC 25010:2023 palaikomumo kriterijų, taip pat atskiriant jaunesniųjų ir vyresniųjų ekspertų grupes.

4.4.2.1. Statistinių paklaidų analizė

Modelių vertinimo tikslumui nustatyti buvo apskaičiuota vidutinė absoliuti paklaida (MAE) tarp modelių sugeneruotų balų ir ekspertų įvertinimų. Ši statistinė metrika parodo, kiek vidutiniškai modelio įvertinimas skyrėsi nuo ekspertų vertinimo, kadangi kriterijų vertinimai buvo pateikiami skalėje nuo 1 iki 5, didžiausia MAE reikšmė galėjo būti 4 ir mažiausia 0. Analizė buvo atlikta visų ekspertų atžvilgiu, taip pat atskirai vertinant jaunesniųjų ir vyresniųjų ekspertų grupes.

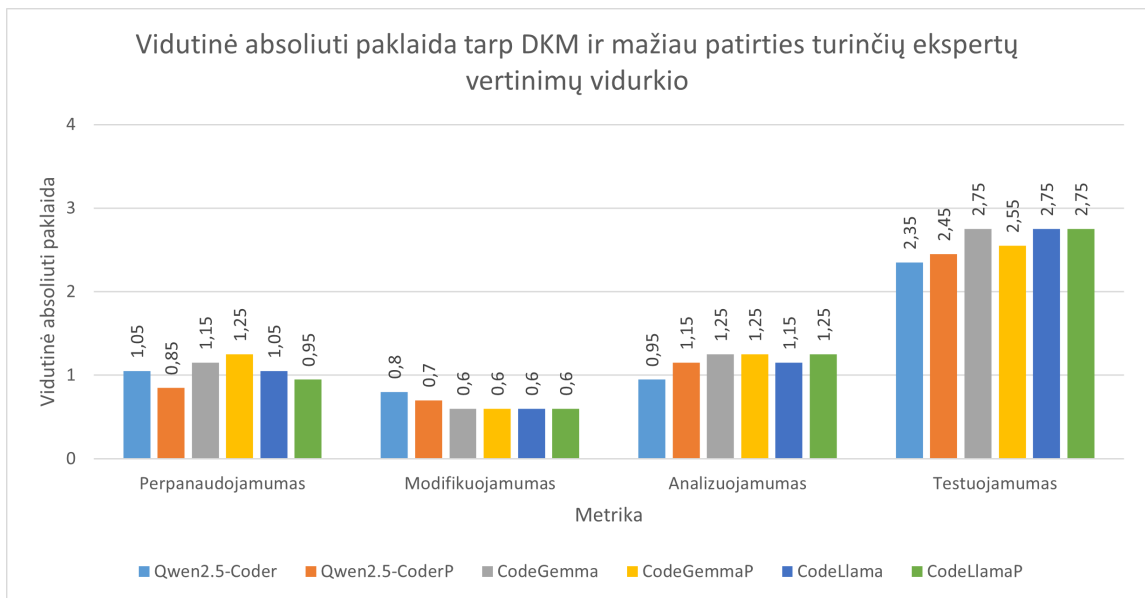
Analizuojant visų keturių ekspertų vertinimų vidurkio MAE su didelių kalbos modelių įvertinimais (žr. 8 pav.), paaiškėjo, kad modifikuojamumo vertinimas visiems modeliams buvo

lengviausias – visų modelių MAE siekė tik 0,5–0,6, o tai rodo gana tikslių gebėjimą vertinti šią subjektyvią metriką. Perpanaudojamumo metrikai geriausius rezultatus pateikė Qwen2.5-Coder ir Qwen2.5-CoderP (MAE = 0,875), o prasčiausius – CodeGemmaP (MAE = 1,275). Analizuojamumo metrikai mažiausią paklaidą pasiekė taip pat Qwen2.5-Coder (MAE = 0,8), o CodeGemma ir CodeGemmaP vėl išsiskyrė aukščiausiais MAE (iki 1,05). Sudėtingiausiai sekėsi visiems modeliams įvertinti testuojamumą. Visų modelių MAE šiame kriterijuje viršijo 2, o CodeGemma ir CodeLlama modelių paklaida siekė net 2,45.



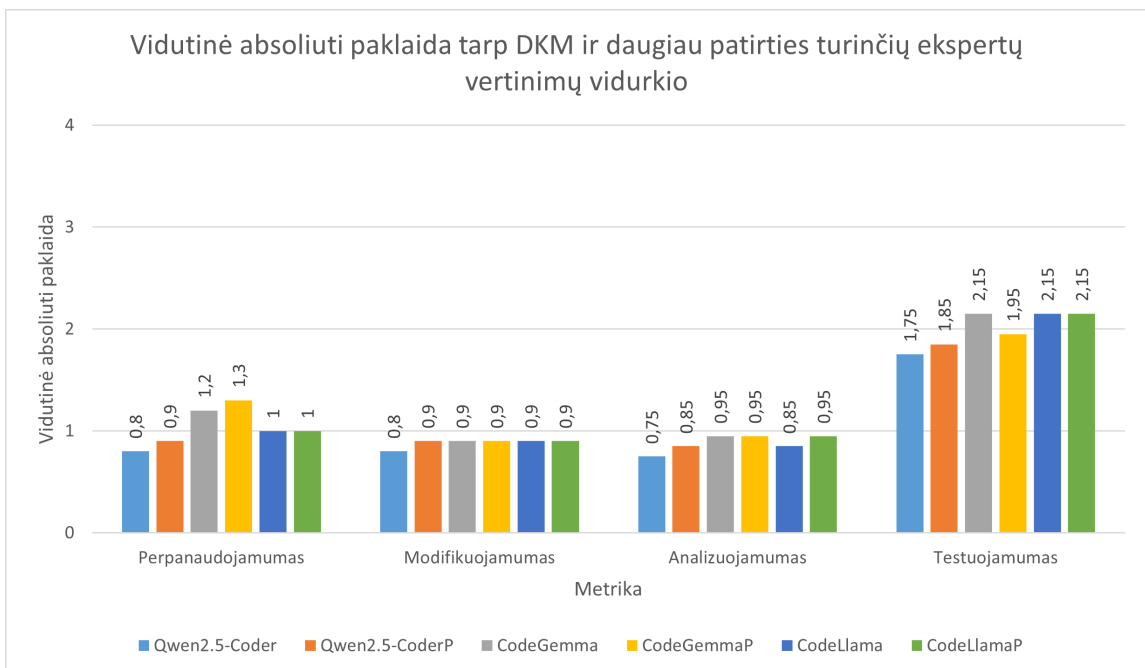
8 pav. Vidutinė absoliuti paklaida tarp DKM ir ekspertų vertinimų vidurkio

Vertinant rezultatus pagal jaunesniųjų ekspertų vertinimus (žr. 9 pav.), daugumos modelių MAE reikšmės buvo didesnės nei bendroje analizėje, ypač testuojamumo ir analizuojamumo kriterijams. Qwen2.5-Coder MAE vertinant testuojamumą išaugo nuo 2,05 iki 2,35, o perpanaudojamumo metrikai – nuo 0,875 iki 1,05. CodeGemma ir CodeGemmaP pasiekė ypač aukštus MAE testuojamumo metrikai – atitinkamai 2,75 ir 2,55, o analizuojamumo – net 1,25. Vienintelė išimtis – modifikuojamumo kriterijus, kuriame MAE reikšmės išliko žemos (0,6–0,8) visiems modeliams, o geriausiai pasirodė CodeGemma ir CodeGemmaP (MAE = 0,6).



9 pav. Vidutinė absoliuti paklaida tarp DKM ir mažiau patirties turinčių ekspertų vertinimų vidurkio

Tuo tarpu vyresniųjų ekspertų atveju (žr. 10 pav.) MAE reikšmės sumažėjo visuose kriterijuose, išskyrus modifikuojamumą, kuriame jos išaugo (MAE = 0,8–0,9) visiems modeliams. Qwen2.5-Coder perpanaudojamumo MAE sumažėjo iki 0,8, analizuojamumo – iki 0,75, o testuojamumo – iki 1,75, palyginti su atitinkamai 0,875, 0,8 ir 2,05 bendroje analizėje. Geriausią rezultatą testuojamumo metrikai pasiekė būtent šis modelis, nors visų modelių MAE šiam kriterijui vis dar viršijo 1,75, visos reikšmės buvo mažesnės nei analizėje su jaunesniaisiais ekspertais.



10 pav. Vidutinė absoliuti paklaida tarp DKM ir daugiau patirties turinčių ekspertų vertinimų vidurkio

4.4.2.2. Koreliacijų analizė

Siekiant pamatuoti, kaip stipriai modelių vertinimai atitiko ekspertų vertinimo tendencijas, buvo apskaičiuoti Pearson (r) ir Spearman (ρ) koreliacijos koeficientai tarp kiekvieno modelio įvertinimų ir ekspertų vertinimų vidurkių. Pearson koreliacija matuoja linijinę priklausomybę, o Spearman – monotoninių reitingų sutapimą. Analizė buvo atlikta visų ekspertų, jaunesniųjų ir vyresniųjų grupių atžvilgiu. Rezultatai pateikti 9 ir 10 lentelėse.

9 lentelė. Pearson koreliacijos koeficientai tarp modelių ir ekspertų vertinimų

Metrika	Modelis	Visi ekspertai	Jaunesnieji	Vyresnieji
Perpanaudojamumas	Qwen2.5-Coder	-0.116	-0.417	0.133
	Qwen2.5-CoderP	0.186	0.029	0.260
	CodeGemma	0.043	-0.044	0.098
	CodeGemmaP	-0.272	-0.058	-0.369
	CodeLlama	0.674	0.558	0.608
	CodeLlamaP	–	–	–
Modifikuojamumas	Qwen2.5-Coder	0.352	0.129	0.407
	Qwen2.5-CoderP	0.179	0.147	0.155
	CodeGemma	–	–	–
	CodeGemmaP	–	–	–
	CodeLlama	0.157	0.098	0.155
	CodeLlamaP	–	–	–
Analizuojamumas	Qwen2.5-Coder	0.470	0.294	0.426
	Qwen2.5-CoderP	0.302	0.163	0.291
	CodeGemma	–	–	–
	CodeGemmaP	–	–	–
	CodeLlama	0.134	0.163	0.067
	CodeLlamaP	–	–	–
Testuojamumas	Qwen2.5-Coder	0.352	0.327	0.302
	Qwen2.5-CoderP	0.295	0.181	0.298
	CodeGemma	–	–	–
	CodeGemmaP	0.295	0.181	0.298
	CodeLlama	–	–	–
	CodeLlamaP	–	–	–

10 lentelė. Spearman koreliacijos koeficientai tarp modelių ir ekspertų vertinimų

Metrika	Modelis	Visi ekspertai	Jaunesnieji	Vyresnieji
Perpanaudojamumas	Qwen2.5-Coder	-0.140	-0.465	0.085
	Qwen2.5-CoderP	0.296	0.060	0.298
	CodeGemma	0.177	-0.045	0.045
	CodeGemmaP	-0.387	-0.078	-0.352
	CodeLlama	0.532	0.537	0.537
	CodeLlamaP	–	–	–
Modifikuojamumas	Qwen2.5-Coder	0.387	0.116	0.352
	Qwen2.5-CoderP	0.296	0.119	0.060
	CodeGemma	–	–	–
	CodeGemmaP	–	–	–
	CodeLlama	0.355	0.119	0.239
	CodeLlamaP	–	–	–
Analizuojamumas	Qwen2.5-Coder	0.442	0.292	0.421
	Qwen2.5-CoderP	0.351	0.188	0.311
	CodeGemma	–	–	–
	CodeGemmaP	–	–	–
	CodeLlama	0.176	0.188	0.186
	CodeLlamaP	–	–	–
Testuojamumas	Qwen2.5-Coder	0.428	0.417	0.315
	Qwen2.5-CoderP	0.418	0.215	0.362
	CodeGemma	–	–	–
	CodeGemmaP	0.418	0.215	0.362
	CodeLlama	–	–	–
	CodeLlamaP	–	–	–

Remiantis visų ekspertų vidurkiu, didžiausią tiek Pearson, tiek Spearman koreliaciją perpanaudojamumo metrikai demonstravo CodeLlama modelis – atitinkamai $r = 0,674$, $q = 0,532$. Qwen2.5-Coder pasiekė aukščiausias koreliacijas analizuojamumo kriterijui: $r = 0,47$, $q = 0,442$, o modifikuojamumo vertinime koreliacijos visiems modeliams buvo silpnos ($r < 0,407$). Testuojamumo kriterijuje koreliacijos taip pat buvo vidutinės – didžiausia Spearman reikšmė siekė 0,428 (Qwen2.5-Coder). Modeliai CodeGemma ir CodeGemmaP perpanaudojamumo metrikai turėjo neigiamas arba labai žemas koreliacijos koeficientus, CodeGemmaP Pearson $r = -0,272$ ir Spearman $q = -0,387$, o tai rodo esminį nesutapimą tarp jų vertinimų ir ekspertų vertinimo.

Analizuojant koreliacijas tik su jaunesniųjų ekspertų grupe, pastebimas ryškus nuoseklumo sumažėjimas daugeliui modelių. Qwen2.5-Coder koreliacija perpanaudojamumo kriterijui tapo didesnė į neigiamą pusę (Pearson $r = -0,42$, Spearman $q = -0,47$). Tuo tarpu CodeLlama išlaikė palyginti aukštą koreliaciją su jaunesniaisiais ekspertais perpanaudojamumo kriterijui ($r = 0,56$, $q = 0,54$), o kitų modelių reikšmės dažniausiai buvo žemos. Analizuojamumo Spearman koreliacijos svyravo apie 0,19–0,29, o testuojamumo – nuo 0,21 (CodeGemmaP) iki 0,42 (Qwen2.5-Coder).

Koreliacijos su vyresniųjų ekspertų vertinimais buvo ženkliai aukštesnės. Qwen2.5-Coder analizuojamumo kriterijui pasiekė Pearson koreliaciją $r = 0,43$ ir Spearman $q = 0,42$, o modifikuojamumo – atitinkamai 0,41 ir 0,35. CodeLlama vėl pasižymėjo aukšta perpanaudojamumo

koreliacija – $r = 0,61$, $\rho = 0,54$. Šie rezultatai patvirtina tendenciją, pastebėtą ir MAE analizėje – modelių vertinimai labiau sutampa su patyrusių ekspertų vertinimų rezultatais.

Kai kurioms kombinacijoms koreliacijos nebuvo apskaičiuotos, nes visi ekspertų arba modelių vertinimai buvo vienodi, t. y. neturėjo dispersijos. Tokiu atveju koreliacijos matematiškai negalima apskaičiuoti, nes standartinis nuokrypis lygus nuliui.

4.5. Eksperimento rezultatai ir apibendrinimas

Atliktas eksperimentinis tyrimas leido įvertinti didelių kalbos modelių galimybes analizuojant C# kalbos programinio kodo kokybę, pasitelkiant tiek statines metrikas, tiek subjektyvius palaikomumo aspektus, paremtus ISO/IEC 25010:2023 standartu.

Vertinant modelių tikslumą statinių metrikų kontekste, geriausius rezultatus parodė Qwen2.5-Coder modelis. Šio modelio vidutinis tikslumas su ± 2 paklaida 6 iš 9 metrikų (atmetant pasikartojančių eilučių metriką) viršijo 75 %. Šis rezultatas rodo, kad Qwen2.5-Coder modelis geba įvertinti metrikų reikšmes pakankamai tiksliai, jei yra toleruojamas nedidelis nuokrypis. Vis dėlto, kaip ir kiti modeliai šis modelis itin netiksliai vertino kodo eilučių skaičiaus metriką, kur tikslumas siekė tik 8 %.

Pastebėta tendencija, kad modelių tikslumas auga proporcingai augant toleruojamai paklaidai, o tai rodo, jog modeliai pateikia metrikų įvertinimus, artimus statinės analizės įrankiams ir geba suprasti metrikas. Ši tendencija ypač ryškiai buvo matoma CodeGemma ir CodeLlama modelių vertinimuose, kurie metrikų vertinime pasirodė prasčiausiai, tačiau su ± 2 paklaida kai kurių metrikų vertinimu sugebėdavo pasiekti 90 % tikslumą ir prisivyti Qwen2.5-Coder modelį, o kartai ir pralenkti.

Buvo pastebėta ir modelių vertinimo anomalijų: kai kurie modeliai dažnai generavo identiškus įvertinimus skirtingiems metodams. CodeGemma metrikoms nurodė tą pačią reikšmę net 173–310 kartų, o CodeLlamaP – 303–310 kartų iš 310 vertinimų, kam galėjo turėti įtakos pakeisti parametrai, kadangi buvo parinkti siekiant padidinti modelių nuoseklumą. Taip pat buvo pastebėtas trūkumas sudarytame duomenų rinkinyje. Nors pasikartojančių eilučių metrikos vertinimo rezultatai yra pateikiami, kaip itin aukšti, tačiau dėl duomenų rinkinio išbalansuotumo šios metrikos prasme, neatspindi realių modelių vertinimo gebėjimų, todėl negali būti objektyviai vertinami.

Subjektyviųjų metrikų analizė atskleidė, kad modelių vertinimai labiau sutapo su daugiau patirties turinčių ekspertų nuomone. Tai buvo pastebima tiek vertinant vidutinę absoliučiąją paklaidą (MAE), tiek Pearson ir Spearman koreliacijos koeficientus. Šis rezultatas leidžia teigti, kad modeliai labiau atspindi vertinimo būdą, artimesnį patyrusių programuotojų mąstymui.

Rezultatai ir išvados

Rezultatai

1. Išnagrinėta mokslinė literatūra apie didelių kalbos modelių sampratą, architektūras, apmokymą užklausų formavimo metodus, atlikta septynių didelių kalbos modelių – GPT-4, Codex, CodeT5, StarCoder, CodeLlama, CodeGemma ir Qwen2.5-Coder – detali analizė bei palyginimas pagal pasirinktus techninius kriterijus, tinkamumą kodo kokybės vertinimui ir pasirinkta atlikti kodo kokybės vertinimo tyrimą su trimis, pagal apsibrėžtus kriterijus, tinkamiausiais dideliais kalbos modeliais – CodeLlama, CodeGemma ir Qwen2.5-Coder.
2. Išanalizuota mokslinė literatūra apimanti kodo kokybės vertinimo metodus, metrikas, įrankius ir tyrime pasirinkta dideliais kalbos modeliais analizuoti trims kategorijoms priskiriamas (sudėtingumo, apimties, objektiškai orientuotos) dešimt statinių kodo kokybės metrikų ir keturias ISO/IEC 25010:2023 standartu apibrėžiamas palaikomumo subcharakteristikas.
3. Remiantis Kitchenham et al., 2009 metodika, darbe išanalizuoti kitų autorių tyrimai, kodo kokybės vertinimo taikant didelius kalbos modelius tema.
4. Sudarytas duomenų rinkinys iš 310, C# programavimo kalbą parašytų, metodų, surinktų iš realių projektų prieinamų per GitHub saugyklą.
5. Atliktas dviejų etapų eksperimentas, kuriame pirmo etapo metu įvertinta sudarytame duomenų rinkinyje esančių metodų kokybė pasirinktais trimis dideliais kalbos modeliais su standartiniais ir pakoreguotais parametrais, pagal pasirinktas statinės kodo kokybės metrikas, o gauti rezultatai palyginti su dviem statinės analizės įrankiais (SonarQube, SciTools Understand) ir antro etapo metu įvertintas didelių kalbos modelių kodo kokybės vertinimo atitikimas ekspertų vertinimams pagal keturias ISO/IEC 25010:2023 standartu apibrėžiamas palaikomumo subcharakteristikas.

Išvados

1. Atlikus susijusių darbų literatūros analizę, nustatyta, kad didelių kalbos modelių taikymas kodo kokybės vertinimui vis dar yra mažai ištirta sritis – pagal 3.1 skyriuje apibrėžtus kriterijus per 2023–2025 m. rasti tik 4 aktualūs tyrimai. Juose buvo analizuojami GPT šeimos modeliai, taikomi skirtingi užklausų formavimo metodai ir vertinami tokie aspektai kaip kodo palaikomumas, skaitomumas ar saugumo pažeidžiamumai, tačiau nei viename tyrime nebuvo atliktas kiekybinis statinių metrikų vertinimas ar rezultatų palyginimas su statinės analizės įrankiais ir ekspertų vertinimais.
2. Atlikus statinių kodo kokybės metrikų vertinimo eksperimentą su trimis dideliais kalbos modeliais ir palyginus jų įvertinimus su statinės kodo analizės įrankių rezultatais nustatyta, kad Qwen2.5-Coder modelis buvo tiksliausias 8 iš 10 vertintų metrikų. Žemiausias šio modelio tikslumas fiksuotas kodo eilučių skaičiaus metrikai (8,06%), o aukščiausias – įeinančių elementų skaičiaus metrikai (45,16%). Qwen2.5-Coder taip pat turėjo mažiausią vidutinės absoliučiosios paklaidos reikšmę 9 iš 10 metrikų ir stipriausią koreliaciją su

statinės analizės įrankių vertinimais 9 iš 10 metrikų. Tuo tarpu prasčiausiai pasirodė modelis CodeLlama – jis buvo netiksliausias 9 iš 10 metrikų, pasiekė tik 0,32% tikslumą kodo eilučių skaičiaus metrikai ir 38,39% tikslumą įeinančių elementų skaičiui. Šis modelis turėjo didžiausias vidutinės absoliučiosios paklaidos reikšmes 8 iš 10 metrikų ir žemiausią koreliaciją su analizės įrankiais 8 iš 10 metrikų.

3. Atlikus ISO/IEC 25010:2023 standartu apibrėžiamų palaikomumo subcharakteristikų vertinimo eksperimentą, lyginant modelių įverčius su ekspertų nuomone, nustatyta, kad Qwen2.5-Coder modelis turėjo mažiausią vidutinę absoliučią paklaidą 3 iš 4 subcharakteristikų bendroje analizėje. Šio modelio mažiausia paklaida fiksuota modifikuojamumo subcharakteristikai (0,6), o didžiausia – testuojamumui (2,05), šis modelis taip pat pasiekė aukščiausią koreliaciją su ekspertų vertinimais 3 iš 4 metrikų pagal Pearson ir Spearman koeficientus. CodeGemma modelis pasirodė prasčiausiai ir turėjo didžiausią vidutinės absoliučiosios paklaidos reikšmę 3 iš 4 metrikų bendroje analizėje, įskaitant 1,275 perpanaudojamumui ir 2,45 testuojamumui.

Šaltiniai

- [11] ISO/IEC 25010:2011 *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models* [<https://www.iso25000.com/index.php/en/iso-25000-standards/iso-25010>]. International Organization for Standardization, 2011. Accessed May 2025.
- [AAA⁺23] J. Achiam, S. Adler, S. Agarwal, L. Ahmad ir kiti. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*. 2023.
- [AAM23] E. A. AlOmar, S. A. AlOmar, M. W. Mkaouer. On the use of static analysis to engage students with software quality improvement: An experience with PMD. Iš: *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. 2023, p. 179–191. Prieiga per internetą: <https://doi.org/10.1109/ICSE-SEET58685.2023.00023>.
- [AEX⁺24] A. Albalak, Y. Elazar, S. M. Xie, S. Longpre ir kiti. A Survey on Data Selection for Language Models. 2024. Prieiga per internetą: <http://arxiv.org/abs/2402.16827>.
- [AM13] Y. Ayalew, K. Mguni. An assessment of changeability of open source software. *Computer and Information Science*. 2013, tomas 6, numeris 3, p. 68.
- [Baa24] S. Baack. A Critical Analysis of the Largest Source for Generative AI Training Data: Common Crawl. Iš: *The 2024 ACM Conference on Fairness, Accountability, and Transparency*. 2024, p. 2199–2208.
- [BB13] A. Bacchelli, C. Bird. Expectations, outcomes, and challenges of modern code review. Iš: *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, p. 712–721.
- [BBM96] V. R. Basili, L. C. Briand, W. L. Melo. A validation of object-oriented design metrics as quality indicators. *IEEE Transactions on software engineering*. 1996, tomas 22, numeris 10, p. 751–761.
- [BD23] J. A. Baktash, M. Dawodi. Gpt-4: A review on advancements and opportunities in natural language processing. *arXiv preprint arXiv:2305.03195*. 2023.
- [CGR⁺20] J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, A. Lopez. A comprehensive survey on support vector machine classification: Applications, challenges and trends. *Neurocomputing*. 2020, tomas 408, p. 189–215. ISSN 0925-2312. Prieiga per internetą: <https://doi.org/https://doi.org/10.1016/j.neucom.2019.10.118>.
- [CK94] S. R. Chidamber, C. F. Kemerer. A metrics suite for object oriented design. *IEEE Transactions on software engineering*. 1994, tomas 20, numeris 6, p. 476–493.
- [CKP⁺15] M. Crawford, T. Khoshgoftaar, J. Prusa, A. Richter, H. Al-Najada. Survey of review spam detection using machine learning techniques. *Journal of Big Data*. 2015, tomas 2, p. 23. Prieiga per internetą: <https://doi.org/10.1186/s40537-015-0029-9>.

- [CND⁺22] A. Chowdhery, S. Narang, J. Devlin, M. Bosma ir kiti. PaLM: Scaling Language Modeling with Pathways. 2022. Prieiga per internetą: <http://arxiv.org/abs/2204.02311>.
- [DST24] M. Dillmann, J. Siebert, A. Trendowicz. Evaluation of large language models for assessing code maintainability. *arXiv preprint arXiv:2401.12714*. 2024.
- [EBM⁺24] Y. Elazar, A. Bhagia, I. Magnusson, A. Ravichander ir kiti. *What's In My Big Data?* 2024. Prieiga per internetą: <https://arxiv.org/abs/2310.20707>.
- [EOA24] N. Eziamaka, T. Odonkor, A. Akinsulire. Advanced strategies for achieving comprehensive code quality and ensuring software reliability. *Computer Science & IT Research Journal*. 2024, tomas 5, numeris 8, p. 1751–1779.
- [Gao23] A. Gao. Prompt engineering for large language models. *Available at SSRN 4504303*. 2023.
- [GBB⁺20] L. Gao, S. Biderman, S. Black, L. Golding ir kiti. The Pile: An 800GB Dataset of Diverse Text for Language Modeling. 2020. Prieiga per internetą: <http://arxiv.org/abs/2101.00027>.
- [GC08] V. Gupta, J. K. Chhabra. Measurement of dynamic metrics using dynamic analysis of programs. Iš: *Proceedings of the Applied Computing Conference*. 2008, p. 81–86.
- [GD]⁺24] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey ir kiti. The Llama 3 Herd of Models. 2024. Prieiga per internetą: <http://arxiv.org/abs/2407.21783>.
- [GEG19] M. García-Valls, J. Escribano-Barreno, J. García-Muñoz. An extensible collaborative framework for monitoring software quality in critical systems. *Information and Software Technology*. 2019, tomas 107, p. 3–17. ISSN 0950-5849. Prieiga per internetą: <https://doi.org/https://doi.org/10.1016/j.infsof.2018.10.005>.
- [Gha15] A. Ghazarian. A theory of software complexity. Iš: *2015 IEEE/ACM 4th SEMAT Workshop on a General Theory of Software Engineering*. IEEE, 2015, p. 29–32.
- [GHW⁺23] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, F. Wei. In-context autoencoder for context compression in a large language model. *arXiv preprint arXiv:2307.06945*. 2023.
- [Git21] GitHub. *GitHub Copilot: Your AI pair programmer*. 2021. Prieiga per internetą: <https://github.com/features/copilot>. Accessed: 2024-12-11.
- [He19] H. He. Understanding source code comments at large-scale. Iš: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019, p. 1217–1219.
- [HYC⁺24] B. Hui, J. Yang, Z. Cui, J. Yang ir kiti. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*. 2024.

- [HKV07] I. Heitlager, T. Kuipers, J. Visser. A practical model for measuring maintainability. Iš: *6th international conference on the quality of information and communications technology (QUATIC 2007)*. IEEE, 2007, p. 30–39.
- [HQS⁺23] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili ir kiti. A survey on large language models: Applications, challenges, limitations, and practical usage. *Authorea Preprints*. 2023.
- [HTS⁺24] M. U. Hadi, Q. A. Tashi, A. Shah, R. Qureshi ir kiti. *Large Language Models: A Comprehensive Survey of its Applications, Challenges, Limitations, and Future Prospects*. 2024. Prieiga per internetą: <https://doi.org/10.36227/techrxiv.23589741.v6>.
- [HWY09] T. Honglei, S. Wei, Z. Yanan. The research on software metrics and software complexity metrics. Iš: *2009 International Forum on Computer Science–Technology and Applications*. IEEE, 2009, tomas 1, p. 131–136.
- [YFZ⁺24] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, E. Chen. A Survey on Multimodal Large Language Models. *National Science Review*. 2024, nwae403. ISSN 2095–5138. Prieiga per internetą: <https://doi.org/10.1093/nsr/nwae403>.
- [IGM11] E. Irrazábal, J. Garzás, E. Marcos. Alignment of Open Source Tools with the New ISO 25010 Standard–Focus on Maintainability. Iš: *International Conference on Software and Data Technologies*. SCITEPRESS, 2011, tomas 2, p. 111–116.
- [YYZ⁺24] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*. 2024, tomas 36.
- [IS24] B. Idrisov, T. Schlippe. Program Code Generation with Generative AIs. *Algorithms*. 2024, tomas 17, numeris 2, p. 62.
- [YSD⁺24] Z. Yang, J. Shi, P. Devanbu, D. Lo. Ecosystem of Large Language Models for Code. *arXiv preprint arXiv:2405.16746*. 2024.
- [JSS25] I. Jaoua, O. B. Sghaier, H. Sahraoui. Combining Large Language Models with Static Analyzers for Code Review Generation. *arXiv preprint arXiv:2502.06633*. 2025.
- [JWL⁺24] S. Jin, Y. Wang, S. Liu, Y. Zhang, W. Gu. *Optimizing Dataset Creation: A General Purpose Data Filtering System for Training Large Language Models*. 2024. Prieiga per internetą: <https://doi.org/10.22541/au.172832800.00216917/v1>.
- [JWS⁺24] J. Jiang, F. Wang, J. Shen, S. Kim, S. Kim. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515*. 2024.
- [Kal24] K. S. Kalyan. A survey of GPT-3 family large language models including ChatGPT and GPT-4. *Natural Language Processing Journal*. 2024, tomas 6, p. 100048. ISSN 2949-7191. Prieiga per internetą: <https://doi.org/https://doi.org/10.1016/j.nlp.2023.100048>.

- [KBB⁺09] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman. Systematic literature reviews in software engineering—a systematic literature review. *Information and software technology*. 2009, tomas 51, numeris 1, p. 7–15.
- [KDL⁺23] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, M. Naik. Understanding the effectiveness of large language models in detecting security vulnerabilities. *arXiv preprint arXiv:2311.16169*. 2023.
- [KFS23] J. Y. Koh, D. Fried, R. R. Salakhutdinov. Generating Images with Multimodal Language Models. Iš: A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, S. Levine (sudarytojai). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023, tomas 36, p. 21487–21506. Prieiga per internetą: https://proceedings.neurips.cc/paper_files/paper/2023/file/43a69d143273bd8215578bde887bb552-Paper-Conference.pdf.
- [KGR⁺22] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*. 2022, tomas 35, p. 22199–22213.
- [KLA⁺22] D. Kocetkov, R. Li, L. B. Allal, J. Li ir kiti. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*. 2022.
- [LAL⁺23] L. Lavazza, A. Z. Abualkishik, G. Liu, S. Morasca. An empirical evaluation of the “Cognitive Complexity” measure as a predictor of code understandability. *Journal of Systems and Software*. 2023, tomas 197, p. 111561.
- [LAZ⁺23] R. Li, L. B. Allal, Y. Zi, N. Muennighoff ir kiti. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*. 2023.
- [LCL⁺24] Y. Liu, J. Cao, C. Liu, K. Ding, L. Jin. Datasets for large language models: A comprehensive survey. *arXiv preprint arXiv:2402.18041*. 2024.
- [Li17] H. Li. Deep learning for natural language processing: advantages and challenges. *National Science Review*. 2017, tomas 5, numeris 1, p. 24–26. ISSN 2095-5138. Prieiga per internetą: <https://doi.org/10.1093/nsr/nwx110>.
- [LIN⁺22] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, N. Carlini. Deduplicating Training Data Makes Language Models Better. Iš: S. Muresan, P. Nakov, A. Villavicencio (sudarytojai). *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022, p. 8424–8445. Prieiga per internetą: <https://doi.org/10.18653/v1/2022.acl-long.577>.
- [LJL⁺20] C.-C. Lin, A. Jaech, X. Li, M. R. Gormley, J. Eisner. Limitations of autoregressive models and their alternatives. *arXiv preprint arXiv:2010.11939*. 2020.
- [LKJ23] V. Lenarduzzi, T. Kilamo, A. Janes. Does Cyclomatic or Cognitive Complexity Better Represents Code Understandability? An Empirical Investigation on the Developers Perception. *An Empirical Investigation on the Developers Perception*. 2023.

- [LLA22] I. Lauriola, A. Lavelli, F. Aiolli. An introduction to Deep Learning in Natural Language Processing: Models, techniques, and tools. *Neurocomputing*. 2022, tomas 470, p. 443–456. ISSN 0925–2312. Prieiga per internetą: <https://doi.org/https://doi.org/10.1016/j.neucom.2021.05.103>.
- [LN88] J. Lee Rodgers, W. A. Nicewander. Thirteen ways to look at the correlation coefficient. *The American Statistician*. 1988, tomas 42, numeris 1, p. 59–66.
- [MAH⁺23] M. M. Mohajer, R. Aleithan, N. S. Harzevili, M. Wei, A. B. Belle, H. V. Pham, S. Wang. *SkipAnalyzer: A Tool for Static Code Analysis with Large Language Models*. 2023. Prieiga per internetą: <https://arxiv.org/abs/2310.18532>.
- [MBH17] M. A. A. Mamun, C. Berger, J. Hansson. Correlations of software code metrics: an empirical study. Iš: *Proceedings of the 27th international workshop on software measurement and 12th international conference on software process and product measurement*. 2017, p. 255–266.
- [McC76] T. J. McCabe. A complexity measure. *IEEE Transactions on software Engineering*. 1976, numeris 4, p. 308–320.
- [Mes23] B. Meskó. The Impact of Multimodal Large Language Models on Health Care’s Future. *J Med Internet Res*. 2023, tomas 25, e52865. ISSN 1438–8871. Prieiga per internetą: <https://doi.org/10.2196/52865>.
- [Met23] Meta. *Integration Guide: Meta Code Llama*. 2023. Prieiga per internetą: <https://www.llama.com/docs/integration-guides/meta-code-llama/>. Accessed: 2024-12-12.
- [MWW20] M. Muñoz Barón, M. Wyrich, S. Wagner. An Empirical Validation of Cognitive Complexity as a Measure of Source Code Understandability. Iš: *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. Bari, Italy: Association for Computing Machinery, 2020. ESEM ’20. ISBN 9781450375801. Prieiga per internetą: <https://doi.org/10.1145/3382494.3410636>.
- [NKQ⁺23] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, A. Mian. A comprehensive overview of large language models. *arXiv preprint arXiv:2307.06435*. 2023.
- [NNN24] D. Naik, I. Naik, N. Naik. Large Data Begets Large Data: Studying Large Language Models (LLMs) and Its History, Types, Working, Benefits and Limitations. Iš: *The International Conference on Computing, Communication, Cybersecurity & AI (The C3AI 2024)*. Springer. 2024.
- [NPM⁺17] A. S. Nuñez-Varela, H. G. Pérez-Gonzalez, F. E. Martínez-Perez, C. Soubervielle-Montalvo. Source code metrics: A systematic mapping study. *Journal of Systems and Software*. 2017, tomas 128, p. 164–197. ISSN 0164–1212. Prieiga per internetą: <https://doi.org/https://doi.org/10.1016/j.jss.2017.03.044>.

- [NR13] M. S. Neethu, R. Rajasree. Sentiment analysis in twitter using machine learning techniques. Iš: *2013 Fourth International Conference on Computing, Communications and Networking Technologies (ICCCNT)*. 2013, p. 1–5. Prieiga per internetą: <https://doi.org/10.1109/ICCCNT.2013.6726818>.
- [OMS⁺18] S. Omri, P. Montag, C. Sinz ir kiti. Static analysis and code complexity metrics as early indicators of software defects. *Journal of software engineering and applications*. 2018, tomas 11, numeris 04, p. 153.
- [Ope22] OpenAI. *Powering next generation applications with OpenAI Codex*. 2022. Prieiga per internetą: <https://openai.com/index/codex-apps/>. Accessed: December 7, 2024.
- [PKC21] E. Perez, D. Kiela, K. Cho. True Few-Shot Learning with Language Models. Iš: M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, J. W. Vaughan (sudarytojai). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2021, tomas 34, p. 11054–11070. Prieiga per internetą: https://proceedings.neurips.cc/paper_files/paper/2021/file/5c04925674920eb58467fb52ce4ef728-Paper.pdf.
- [Res21] S. Research. *CodeT5: Pre-trained Encoder-Decoder Model for Programming Languages*. 2021. Prieiga per internetą: <https://github.com/salesforce/CodeT5/tree/main/CodeT5>. Accessed: 2024-12-11.
- [Res24] S. Research. *Salesforce AI Research*. 2024. Prieiga per internetą: <https://www.salesforceairesearch.com/>. Accessed: 2024-12-12.
- [RGG⁺23] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla ir kiti. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*. 2023.
- [RSR⁺19] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. 2019. Prieiga per internetą: <http://arxiv.org/abs/1910.10683>.
- [RSW⁺24] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, P. Abrahamsson. Ai-powered code review with llms: Early results. *arXiv preprint arXiv:2404.18496*. 2024.
- [SBK⁺24] M. Shao, A. Basit, R. Karri, M. Shafique. Survey of different Large Language Model Architectures: Trends, Benchmarks, and Challenges. *IEEE Access*. 2024. ISSN 21693536. Prieiga per internetą: <https://doi.org/10.1109/ACCESS.2024.3482107>.
- [Sci24] SciTools, Inc. *SciTools Metrics Definitions*. 2024. Prieiga per internetą: <https://docs.scitools.com/metrics/dist/index.html>. Accessed 2024-05-01.

- [SDL⁺24] M. Shoeybi, M. Dehghani, S. T. Luccioni ir kiti. *Introducing CodeGemma: Code models for lightweight code completion and code chat* [<https://huggingface.co/blog/codegemma>]. 2024. Prieiga per internetą: <https://huggingface.co/blog/codegemma>. Hugging Face Blog, April 29, 2024.
- [SGS⁺22] B. Sorscher, R. Geirhos, S. Shekhar, S. Ganguli, A. S. Morcos. Beyond neural scaling laws: beating power law scaling via data pruning. Iš: A. H. Oh, A. Agarwal, D. Belgrave, K. Cho (sudarytojai). *Advances in Neural Information Processing Systems*. 2022. Prieiga per internetą: <https://openreview.net/forum?id=UmvSlP-PyV>.
- [SJS⁺17] A. Sriram, H. Jun, S. Satheesh, A. Coates. Cold fusion: Training seq2seq models together with language models. *arXiv preprint arXiv:1708.06426*. 2017.
- [SLM⁺24] S. Shahriar, B. D. Lund, N. R. Mannuru, M. A. Arshad, K. Hayawi, R. V. K. Bevara, A. Mannuru, L. Batool. Putting gpt-4o to the sword: A comprehensive evaluation of language, vision, speech, and multimodal proficiency. *Applied Sciences*. 2024, tomas 14, numeris 17, p. 7782.
- [Son17] SonarSource. *Cognitive Complexity*. 2017. Prieiga per internetą: <https://www.sonarsource.com/resources/cognitive-complexity/>. Accessed: 2024-12-20.
- [Son24] SonarSource. *Code Metrics Definition*. 2024. Prieiga per internetą: <https://docs.sonarsource.com/sonarqube-server/10.6/user-guide/code-metrics/metrics-definition/>. Accessed: 2024-12-14.
- [Sonnd] SonarSource. *Cyclomatic Complexity* [<https://www.sonarsource.com/learn/cyclomatic-complexity>]. n.d. Prieiga per internetą: <https://www.sonarsource.com/learn/cyclomatic-complexity>. Accessed May 2025.
- [SPD⁺05] G. Succi, W. Pedrycz, S. Djokic, P. Zuliani, B. Russo. An empirical exploration of the distributions of the chidamber and kemerer object-oriented metrics suite. *Empirical Software Engineering*. 2005, tomas 10, p. 81–104.
- [SSC⁺18] C. Sadowski, E. Söderberg, L. Church, M. Sipko, A. Bacchelli. Modern code review: a case study at google. Iš: *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*. Gothenburg, Sweden: Association for Computing Machinery, 2018, p. 181–190. ICSE-SEIP '18. ISBN 9781450356596. Prieiga per internetą: <https://doi.org/10.1145/3183519.3183525>.
- [SSS⁺24] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, A. Chadha. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 2024. Prieiga per internetą: <https://arxiv.org/abs/2402.07927>.
- [ST22] T. Software, TÜViT. *TIOBE TUViT Trusted Product Maintainability* [https://www.tiobe.com/files/TIOBETUViTTrustedProductMaintainability_v1_2.pdf]. 2022. Versija 1.2. Accessed May 2025.

- [SV24] I. R. da Silva Simões, E. Venson. *Evaluating Source Code Quality with Large Language Models: a comparative study*. 2024. Prieiga per internetą: <https://arxiv.org/abs/2408.07082>.
- [TB25] K. Tamberg, H. Bahsi. Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study. *IEEE Access*. 2025.
- [TCM⁺18] A. Trockman, K. Cates, M. Mozina, T. Nguyen, C. Kästner, B. Vasilescu. "Automatically assessing code understandability" reanalyzed: combined metrics matter. Iš: *Proceedings of the 15th International Conference on Mining Software Repositories*. 2018, p. 314–318.
- [Tea24] Q. Team. *Qwen2.5-Coder: Large Language Models for Code Understanding and Generation* [<https://github.com/QwenLM/Qwen2.5-Coder>]. 2024. Prieiga per internetą: <https://github.com/QwenLM/Qwen2.5-Coder>. GitHub repository, accessed May 2025.
- [THG21] A. Trautsch, S. Herbold, J. Grabowski. *Are automated static analysis tools worth it? An investigation into relative warning density and external software quality*. 2021. Prieiga per internetą: <https://arxiv.org/abs/2111.09188>.
- [TLI⁺23] H. Touvron, T. Lavril, G. Izacard, X. Martinet ir kiti. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. 2023.
- [TSA⁺23] K. Tirumala, D. Simig, A. Aghajanyan, A. S. Morcos. *D4: Improving LLM Pretraining via Document De-Duplication and Diversification*. 2023. Prieiga per internetą: <https://arxiv.org/abs/2308.12284>.
- [TZH⁺24] C. Team, H. Zhao, J. Hui, J. Howland ir kiti. Codegemma: Open code models based on gemma. *arXiv preprint arXiv:2406.11409*. 2024.
- [VBS⁺17] A. Vaswani, G. Brain, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. *Attention Is All You Need*. 2017.
- [VD24] S. Vatsal, H. Dubey. *A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks*. 2024. Prieiga per internetą: <https://arxiv.org/abs/2407.12994>.
- [Ves15] A. Vesra. A study of various static and dynamic metrics for open source software. *International Journal of Computer Applications*. 2015, tomas 122, numeris 10.
- [WCD⁺24] Z. Wang, Z. Chu, T. V. Doan, S. Ni, M. Yang, W. Zhang. History, development, and principles of large language models: an introductory survey. *AI and Ethics*. 2024, p. 1–17.
- [WEA⁺24] M. Weber, D. Fu, Q. Anthony, Y. Oren ir kiti. RedPajama: an Open Dataset for Training Large Language Models. 2024. Prieiga per internetą: <http://arxiv.org/abs/2411.12372>.
- [WGC⁺23] J. Wu, W. Gan, Z. Chen, S. Wan, P. S. Yu. Multimodal Large Language Models: A Survey. 2023. Prieiga per internetą: <http://arxiv.org/abs/2311.13165>.

- [WH21] D. R. Wijendra, K. Hewagamage. Analysis of cognitive complexity with cyclomatic complexity metric of software. *Int. J. Comput. Appl.* 2021, tomas 174, p. 14–19.
- [WLX⁺23] Z. Wang, C. Long, S. Xu, B. Gan, W. Shi, Z. Cao, T.-S. Chua. LGM3A '23: 1st Workshop on Large Generative Models Meet Multimodal Applications. Iš: *Proceedings of the 31st ACM International Conference on Multimedia*. Ottawa ON, Canada: Association for Computing Machinery, 2023, p. 9744–9745. MM '23. ISBN 9798400701085. Prieiga per internetą: <https://doi.org/10.1145/3581783.3610953>.
- [WM05] C. J. Willmott, K. Matsuura. Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate research*. 2005, tomas 30, numeris 1, p. 79–82.
- [WN12] A. Wirotiakun, P. Netisopakul. Improving software maintenance size metrics A case study: Automated report generation system for particle monitoring in Hard Disk Drive Industry. Iš: *2012 Ninth International Conference on Computer Science and Software Engineering (JCSSE)*. IEEE, 2012, p. 334–339.
- [WW]⁺21] Y. Wang, W. Wang, S. Joty, S. C. H. Hoi. *CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation*. 2021. Prieiga per internetą: <https://arxiv.org/abs/2109.00859>.
- [WWL⁺24] J. Wang, Z. Wu, Y. Li, H. Jiang ir kiti. Large Language Models for Robotics: Opportunities, Challenges, and Perspectives. 2024. Prieiga per internetą: <http://arxiv.org/abs/2401.04334>.
- [WZW⁺23] Z. Wang, W. Zhong, Y. Wang, Q. Zhu, F. Mi, B. Wang, L. Shang, X. Jiang, Q. Liu. Data Management For Training Large Language Models: A Survey. 2023. Prieiga per internetą: <http://arxiv.org/abs/2312.01700>.
- [XAN⁺22] F. F. Xu, U. Alon, G. Neubig, V. J. Hellendoorn. A systematic evaluation of large language models of code. Iš: *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. San Diego, CA, USA: Association for Computing Machinery, 2022, p. 1–10. MAPS 2022. ISBN 9781450392730. Prieiga per internetą: <https://doi.org/10.1145/3520312.3534862>.
- [XLH⁺24] J. Xiong, G. Liu, L. Huang, C. Wu ir kiti. Autoregressive Models in Vision: A Survey. *arXiv preprint arXiv:2411.05902*. 2024.
- [ZFX⁺23] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, Z. Chen. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223*. 2023.
- [ZNW⁺23] Z. Zheng, K. Ning, Y. Wang, J. Zhang, D. Zheng, M. Ye, J. Chen. A survey of large language models for code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372*. 2023.